T3000 Engineering

Get started with GitHub · built Nov 13, 2026

Get started with GitHub

The long version of the same subject: installing Git and TortoiseGit, generating an SSH key, signing up, cloning, and the workflow used for the T3000 repository. It is the document a new developer reads first.

Table of Contents

Introduction to Git

Installing Tools

Installation of Git

Installation of Github for Windows

Procedire to Merge using Git Bash

Compilation on Microsoft Visual Studio 2010

Git commands summary

Cloning a repo from github.com using Git Bash Shell

TortoiseGit

Installation of msysgit, the windows version of Git

Installation of TortoiseGit

Sign up for a github account

Setting up SSH key

Working with repository

TortoiseGit andGithub workflow for T3000_Building_Automation_System project

Introduction to Git

While developing software where developers are spread across the globe, issues of version management become critical. What if two developers simultaneously work on the same module and try

to upload their changes back? Situations like these are not uncommon in real-world. To solve problems such as these there exist Version Control Systems.

What is Git?

Git is a distributed version control system (dvcs) written in the programming language C. A distributed version control system keeps track of software revisions and allows many developers to work on a given project without necessarily being connected to a common network.

There is no central server which stores the data. Every local copy contains full history of source code.

Git keeps track of all versions. Therefore, you can revert to any point in your source code history.

The logo of git looks like below:



Git is a distributed revision control and source code management (SCM) system with an emphasis on speed. Git was initially designed and developed by Linus Torvalds for Linux kernel development; it has since been adopted by many other projects. Every Git working directory is a full-fledged repository with complete history and full revision tracking capabilities, not dependent on network access or a central server. Git is free software distributed under the terms of the GNU General Public License version 2.

Local and Remote Repositories

In a distributed version control system everyone has a complete copy of the source code (including the complete history of the source code) and can perform version control operations against this local copy. The use of a dvcs does not require a central code repository.

Git commits file changes to your local repository and you can synchronize your repository with other (remote) repositories. Git allows you to clone repositories, e.g. create an exact copy of a repository including the complete history of the source code. Owners of repositories can synchronize changes via push (transferring changes to a remote repository) or pull (getting changes from a remote repository).

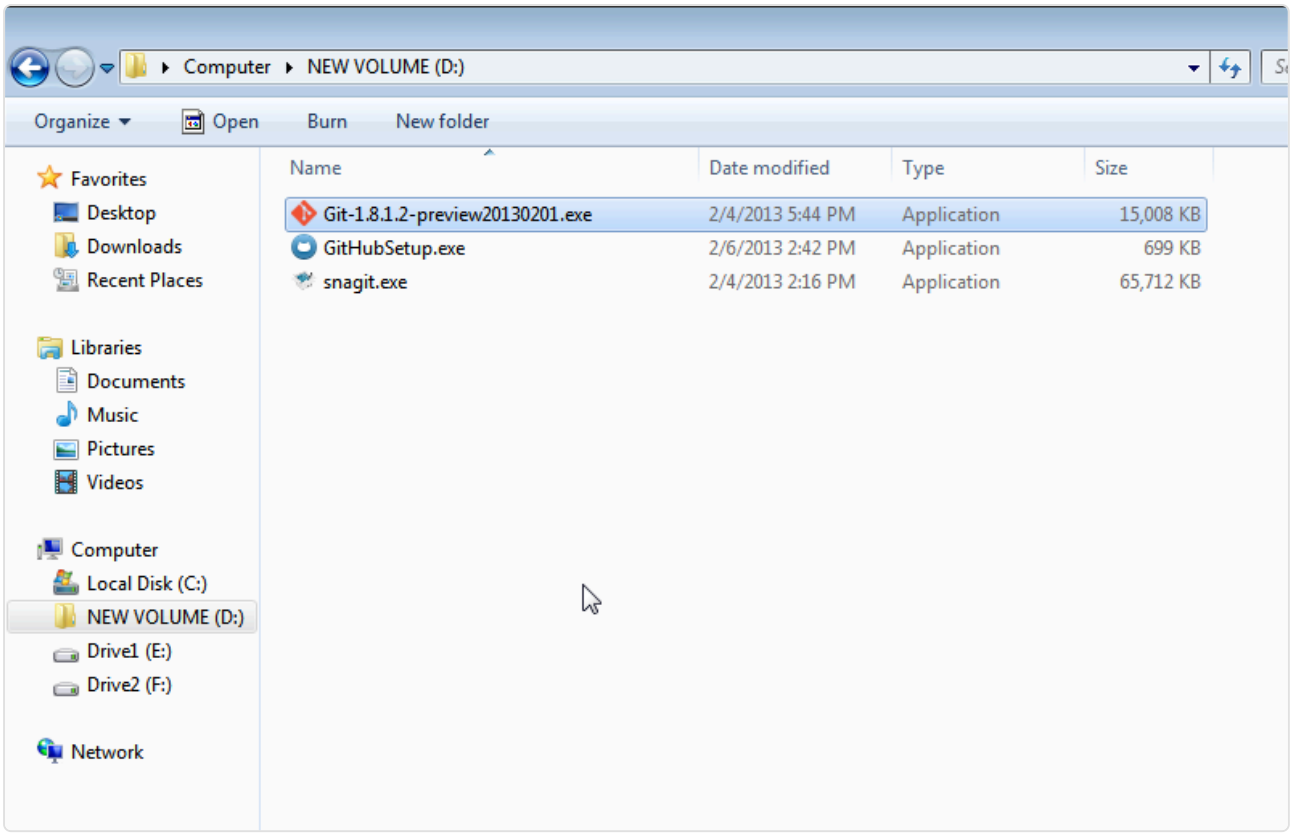
Refer to http://git-scm.com/book for detailed help on understanding git.

Installing Tools

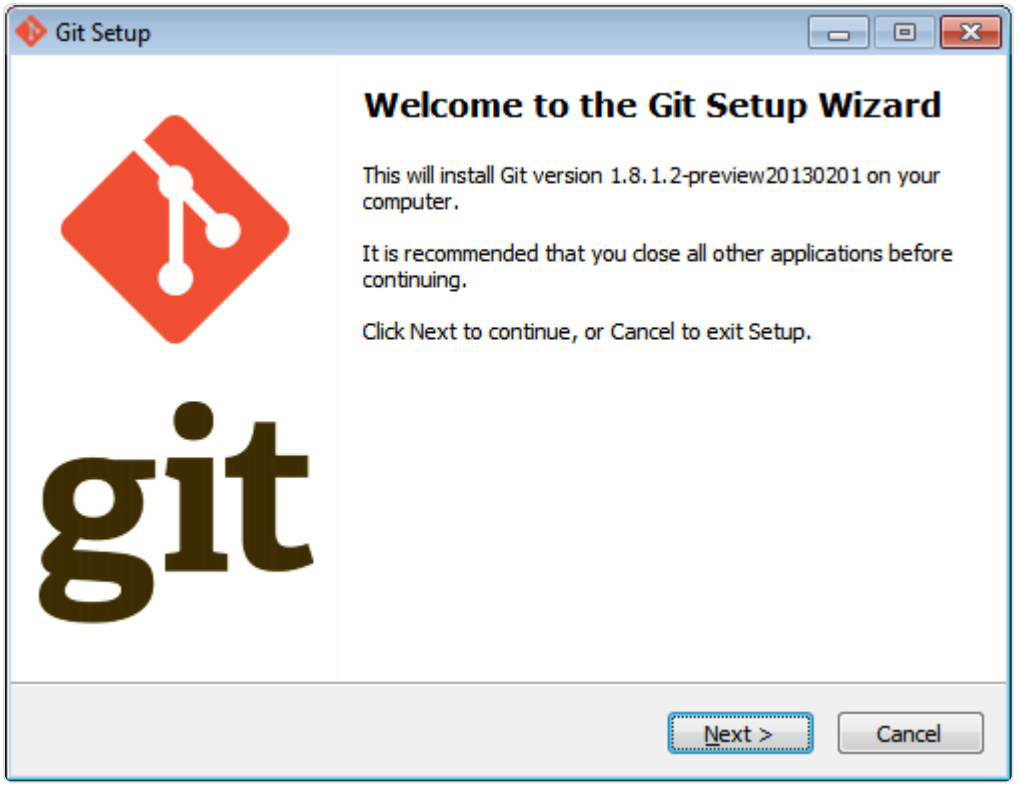
Installation of git

Download git installer.

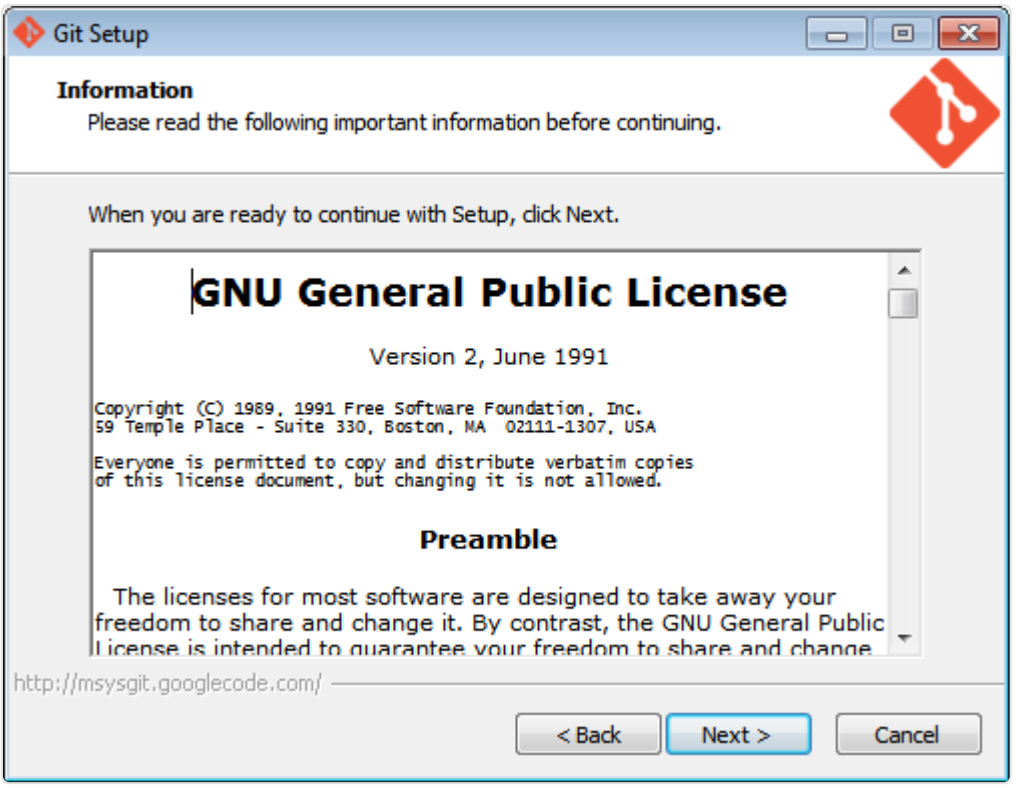
Double click on the setup file.



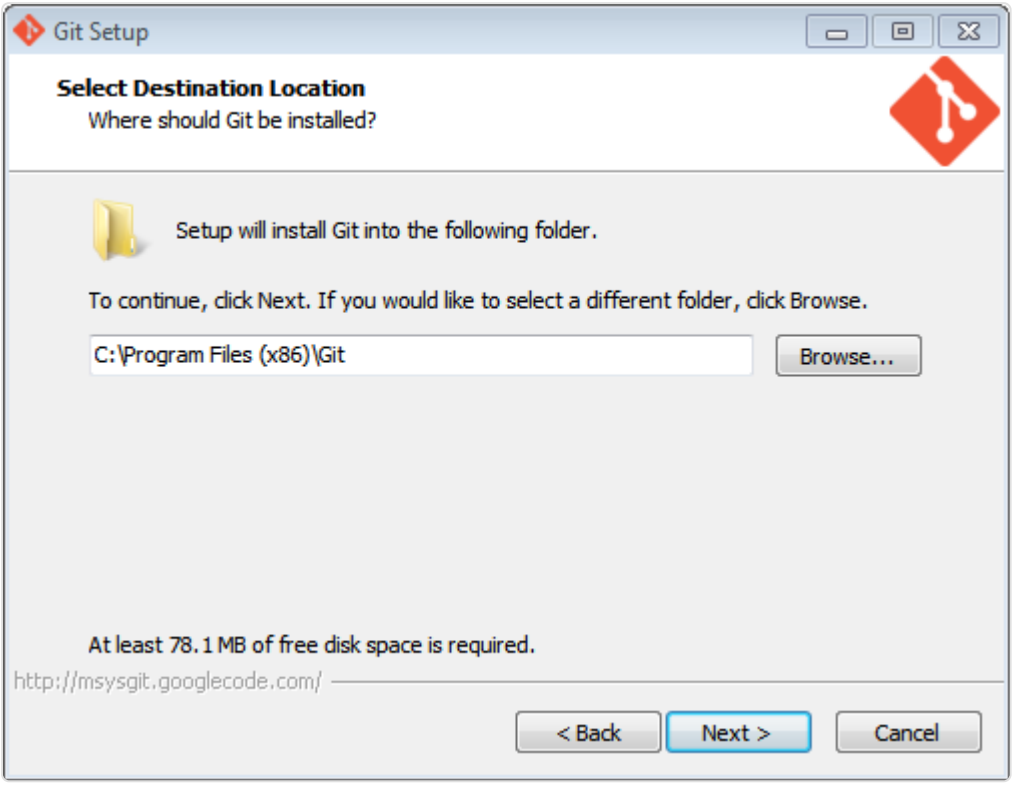
Following dialog box opens. Click Next to continue.



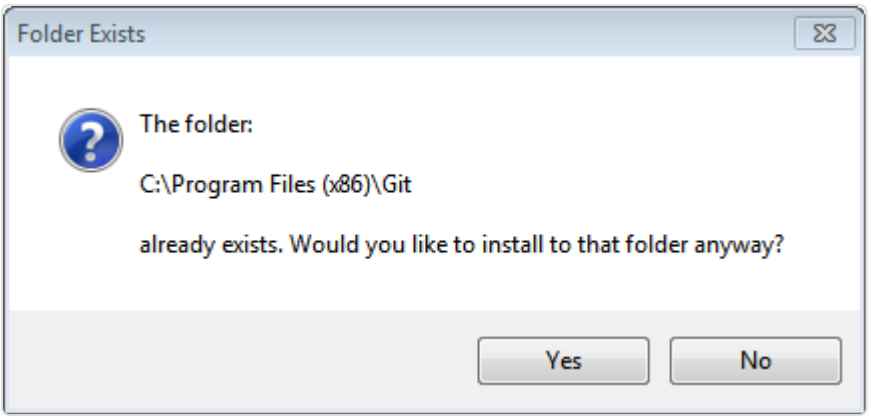
Read GNU General Public License. Click Next.



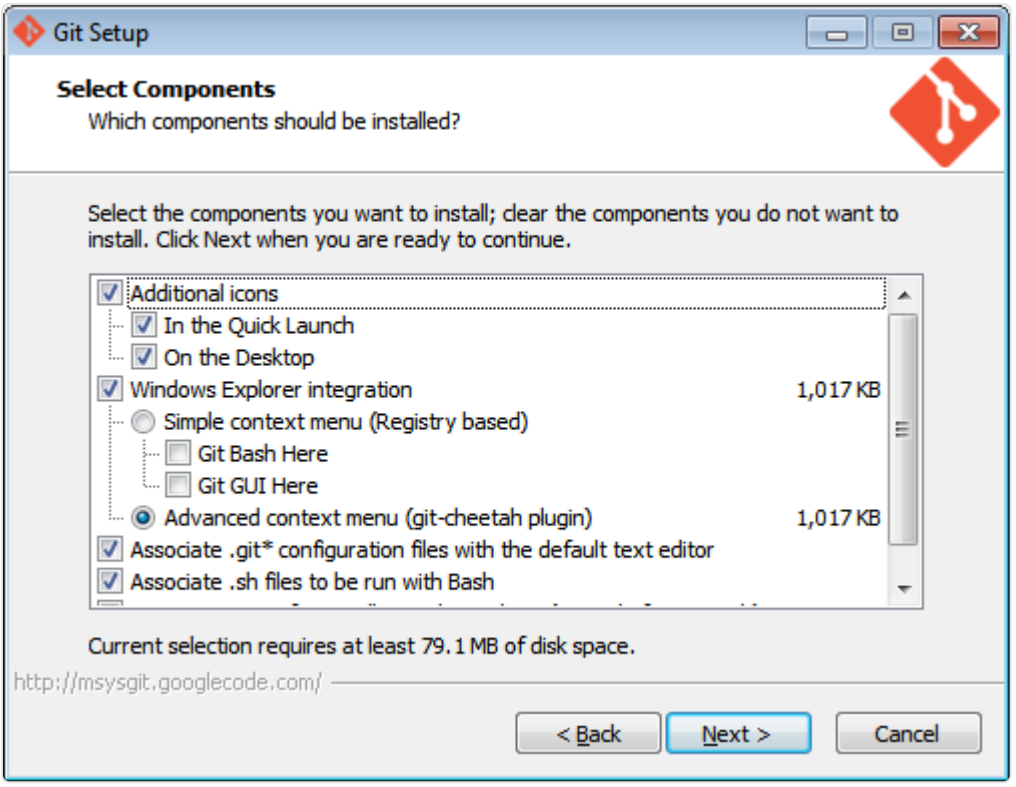
Select destination location. This is the directory where you want to install git. Normally, default directory can be kept as it is. If you wish, you can change the directory by clicking on Browse button. Here, we will install git in the default directory. (C:\Program Files (x86)\Git). Click Next.



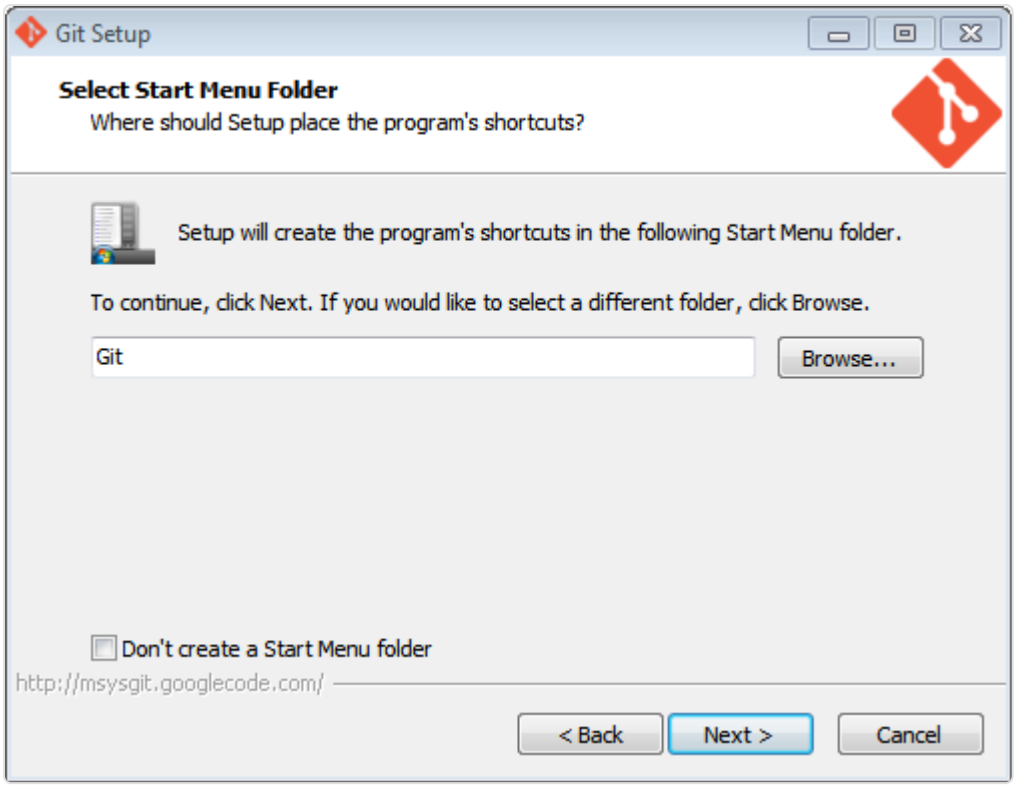
If following dialog box appears, click Yes.



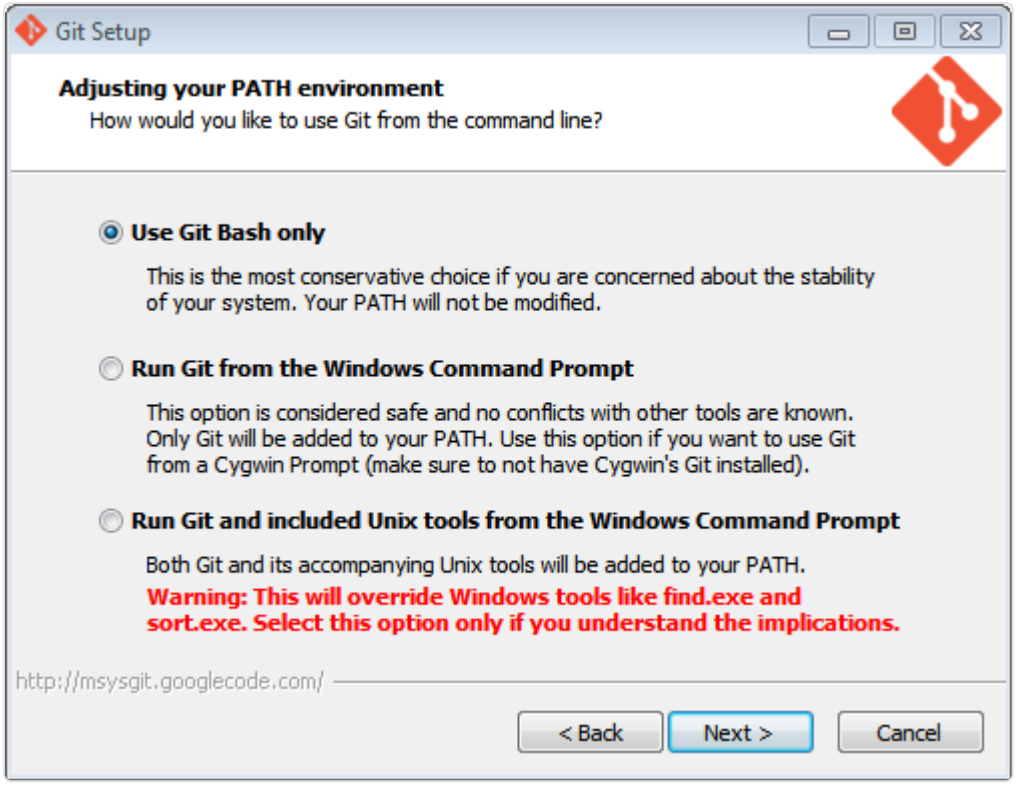
Following screen appears. Leave all the default options as is and click Next.



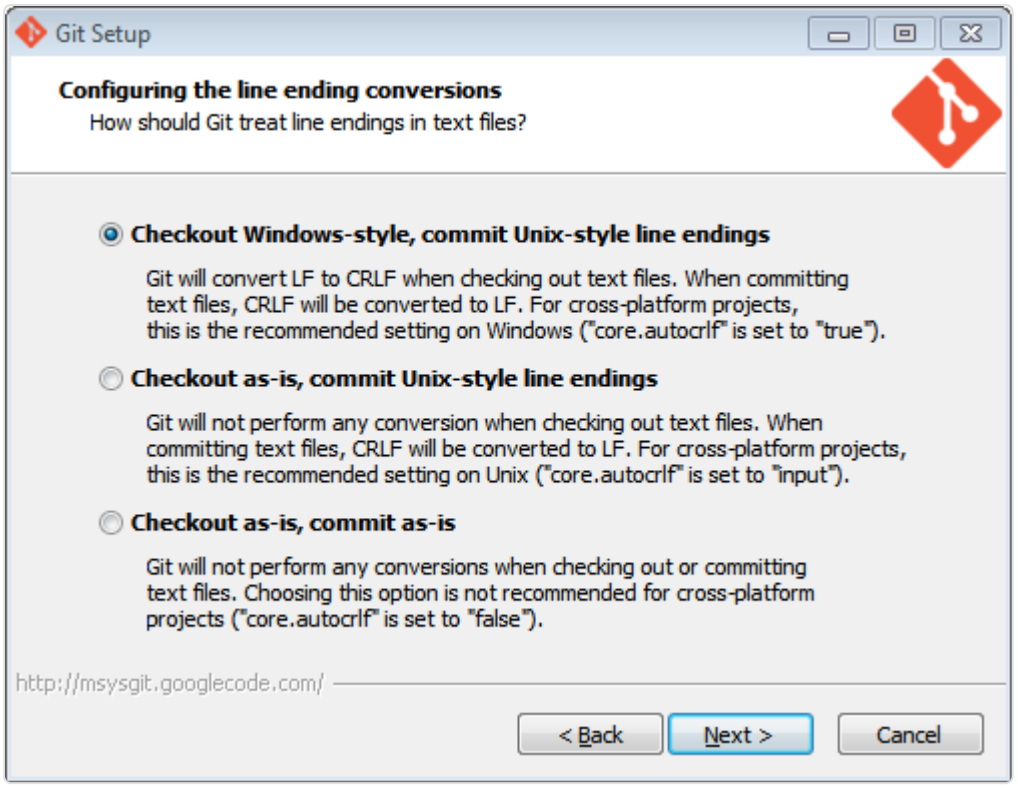
Following screen appears. Click Next.



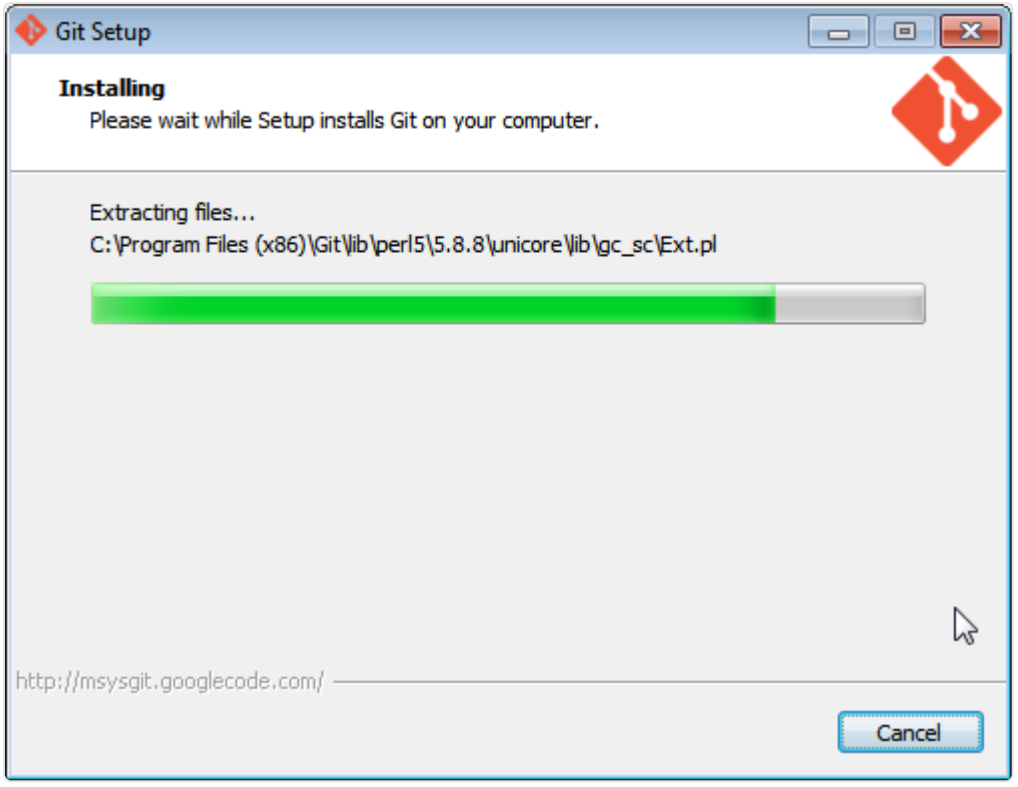
Following screen appears. Click Next to continue.



Following screen appears. Click Next to continue.



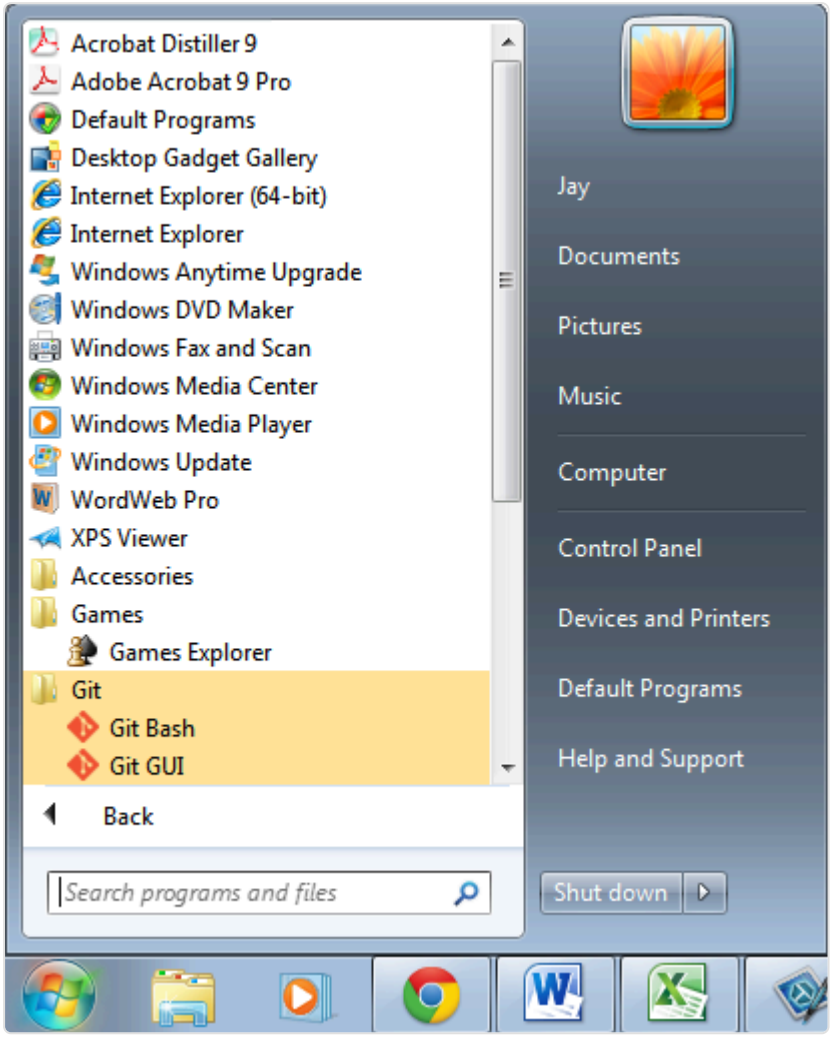
Setup will progress.



Click Finish to complete the setup.



Check installation in Start Menu.



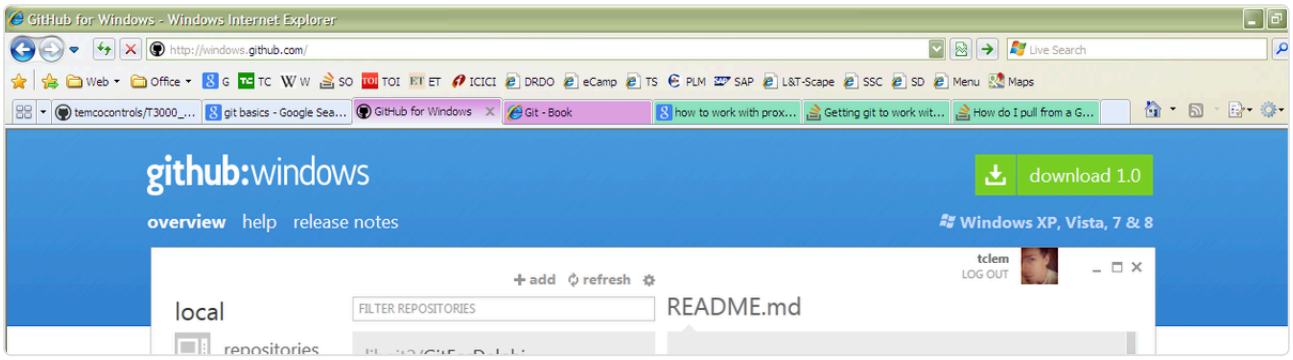
As can be seen, there are two sub menus under Git menu.

Git Bash

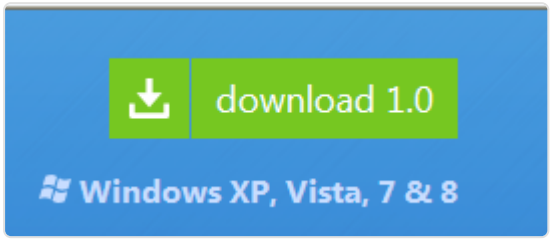
Git GUI

Installation of github for windows

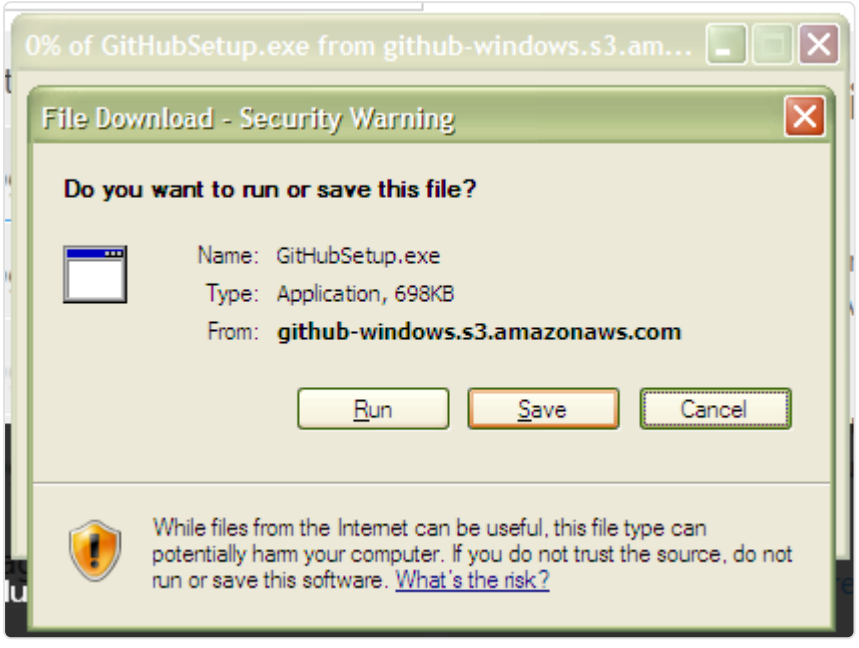
Download github for windows from http://windows.github.com/



Click on the link given for download.

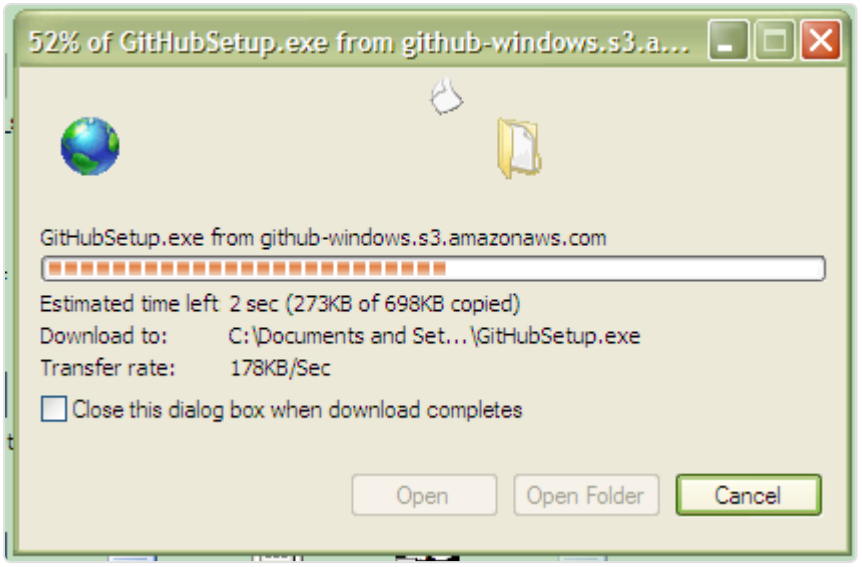


If any confirmation is asked by OS, click Save.



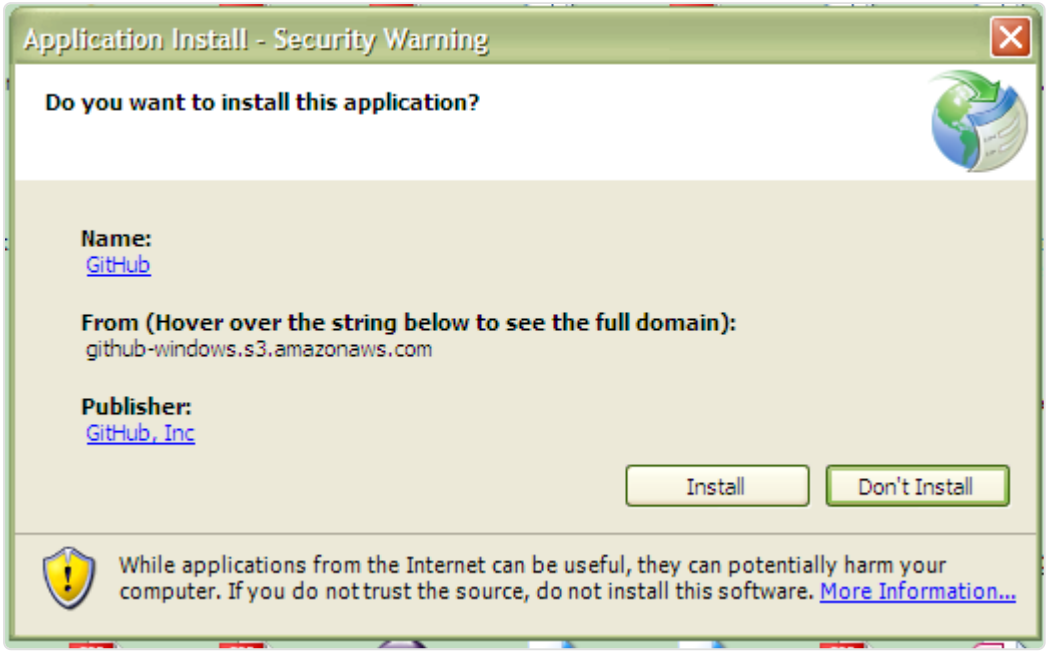
Save the file on your computer at a chosen location.

File download progress bar would be shown.

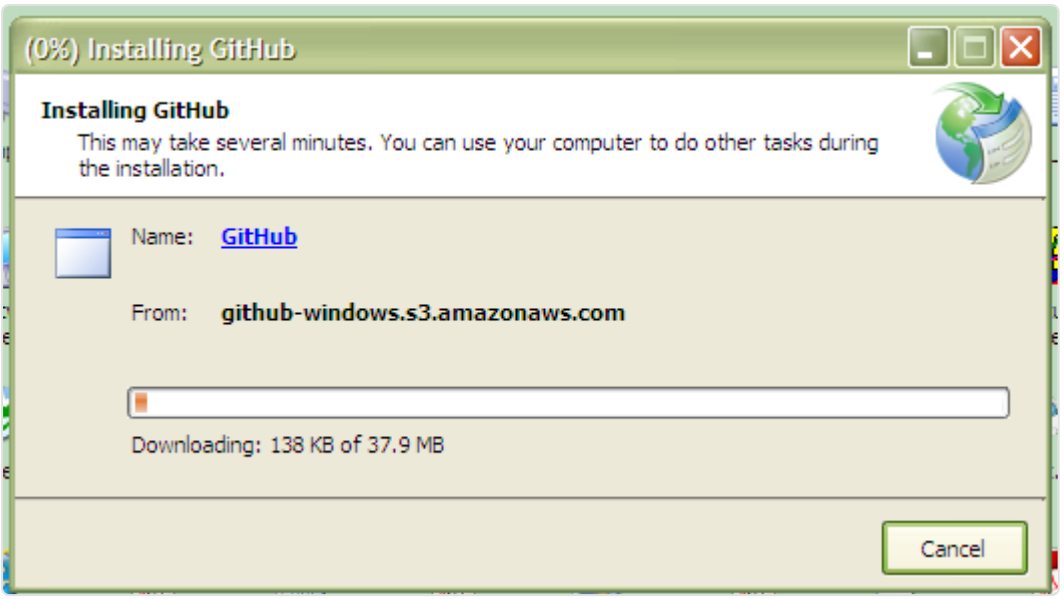


Once the download is complete, double-click on the same to install github for windows on your PC.

A dialog box for Security warning may appear.

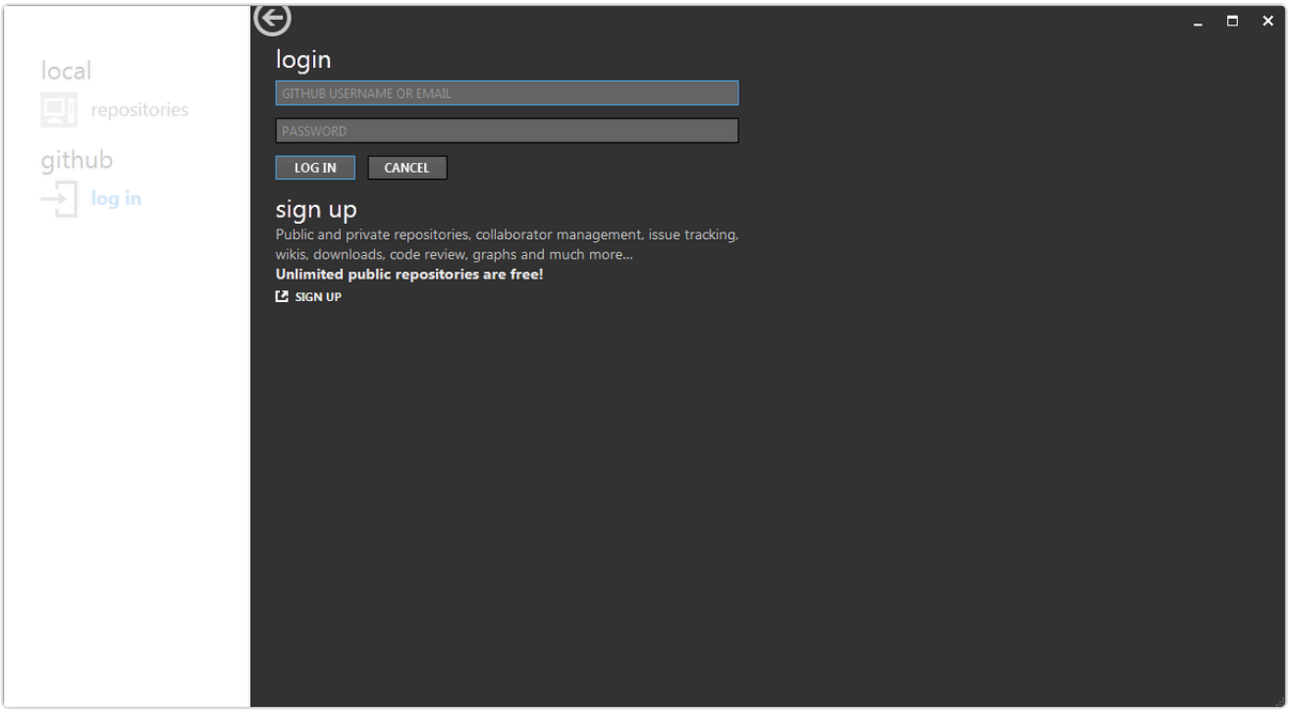


Github for windows starts installing.

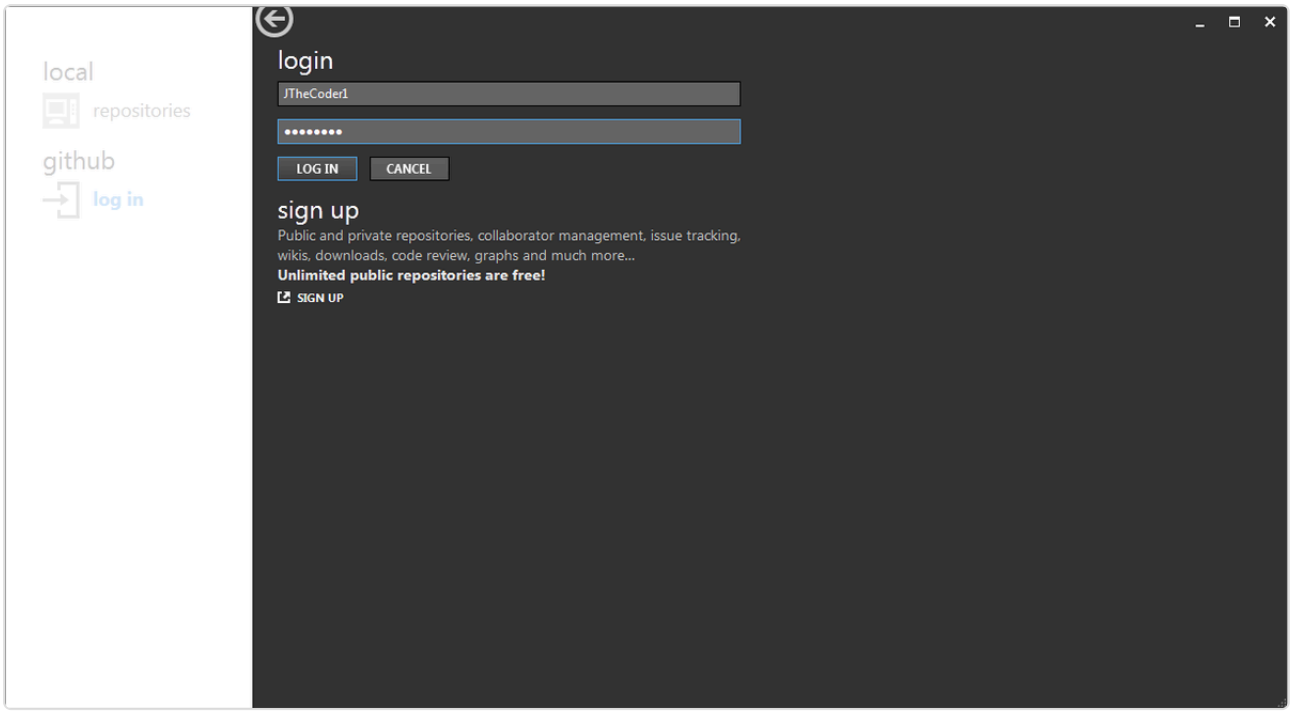


Istallation eventually gets over. Application starts.

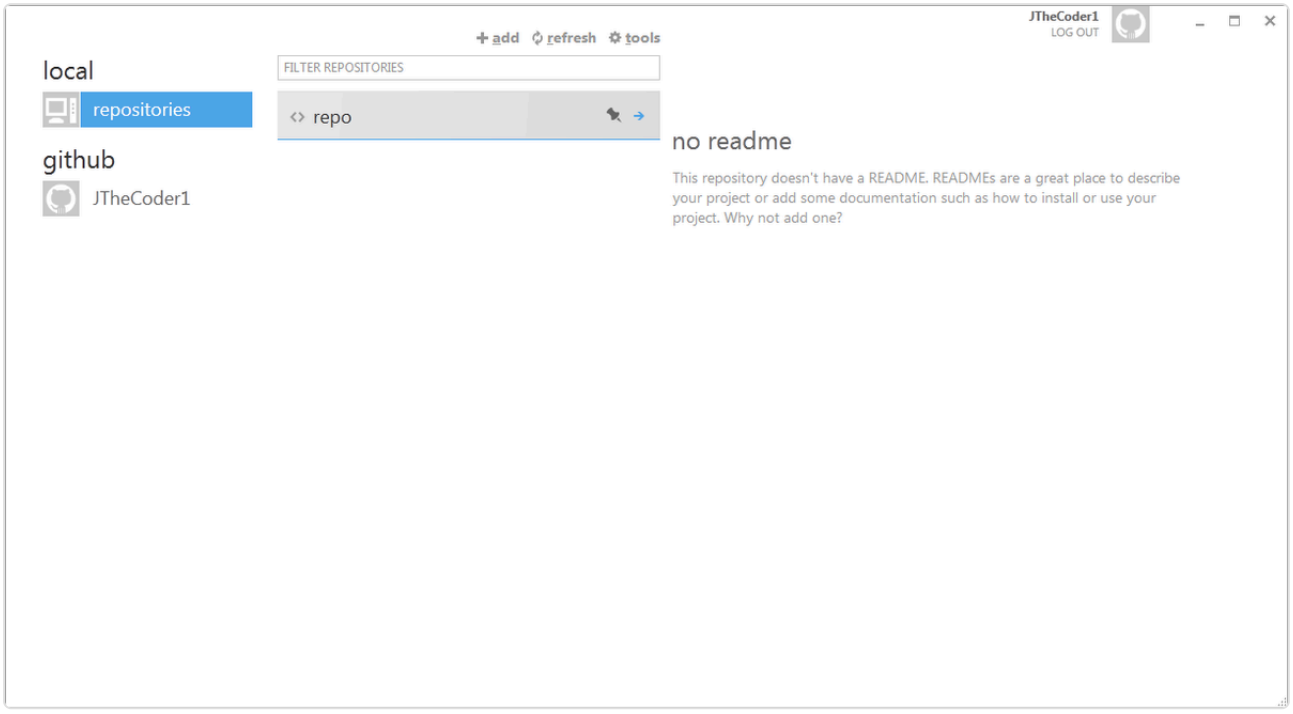
The first screen in the application Github for Windows is as follows.



Enter username and password in the given fields. You have to Sign Up with github to have these information.

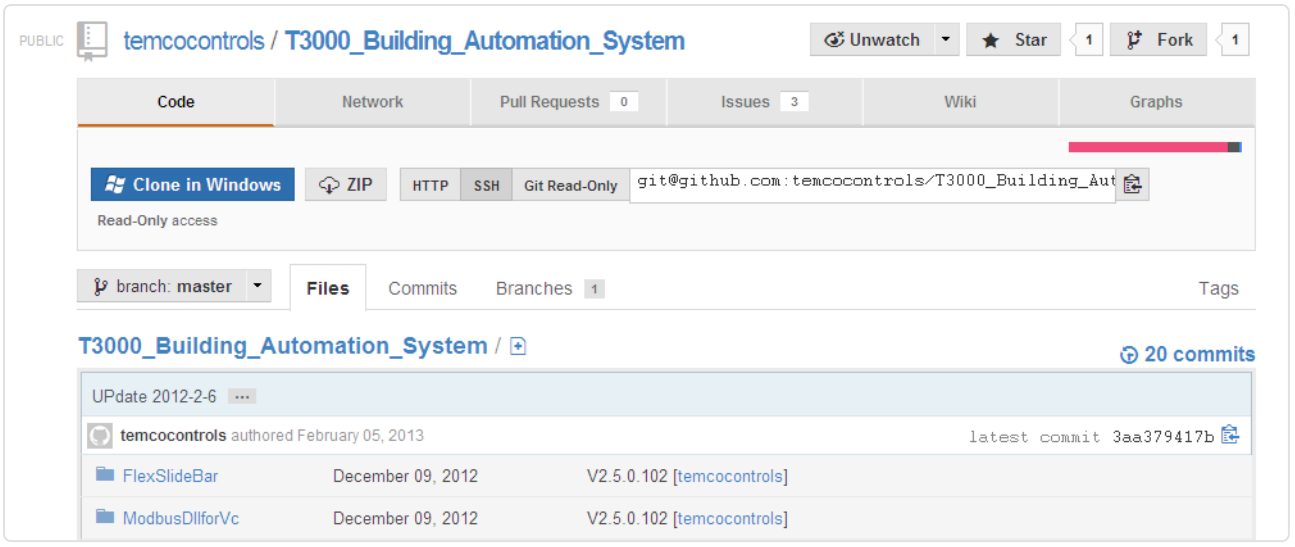


Application screen looks as follows.

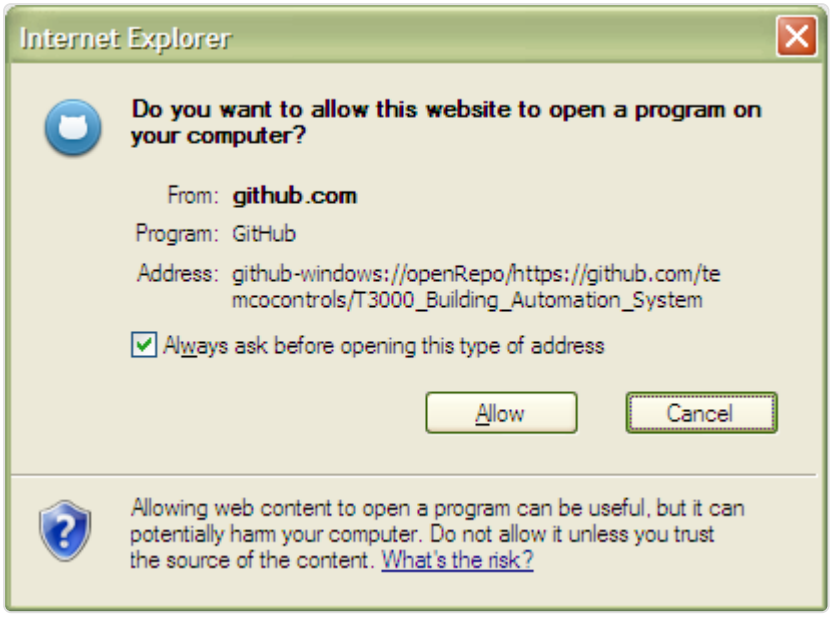


Once the application is installed on the PC, go to github website. Open the project you want to work on. For example, in my case, I am opening repository of T3000 Building Automation System.

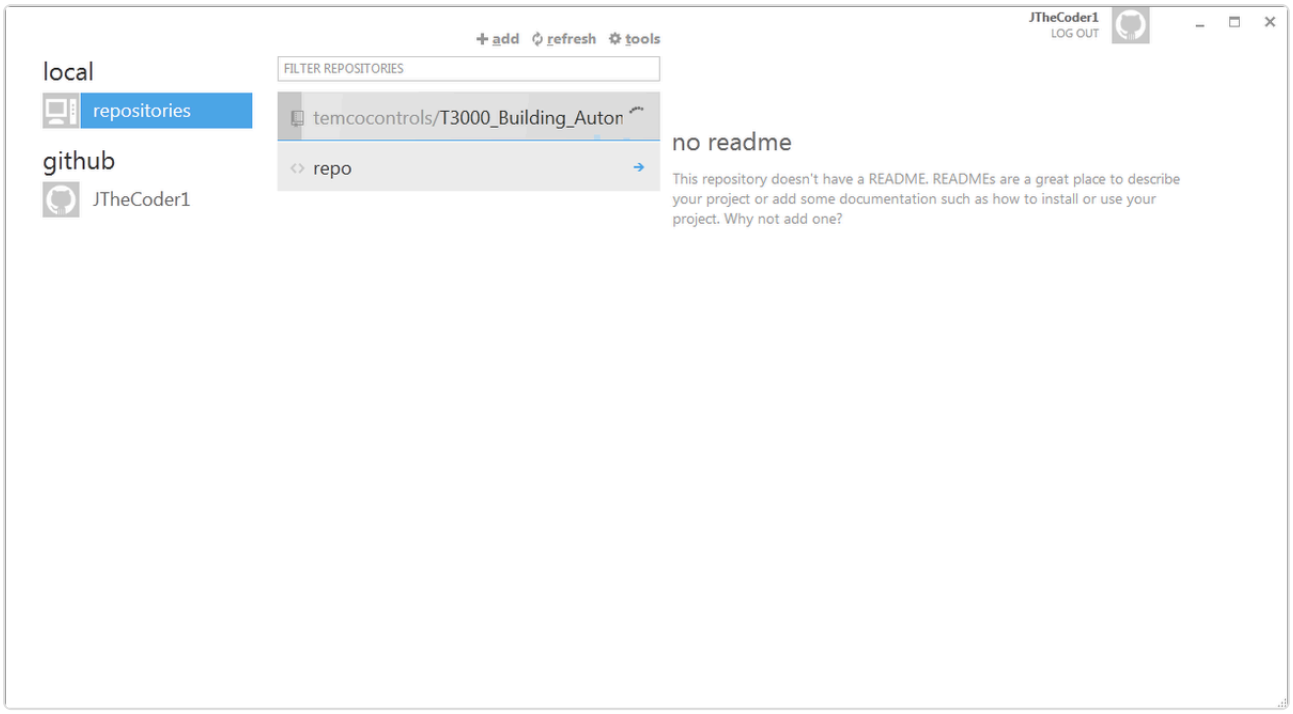
In order to clone the repository from github to local PC, click on Clone in Windows button as shown below.



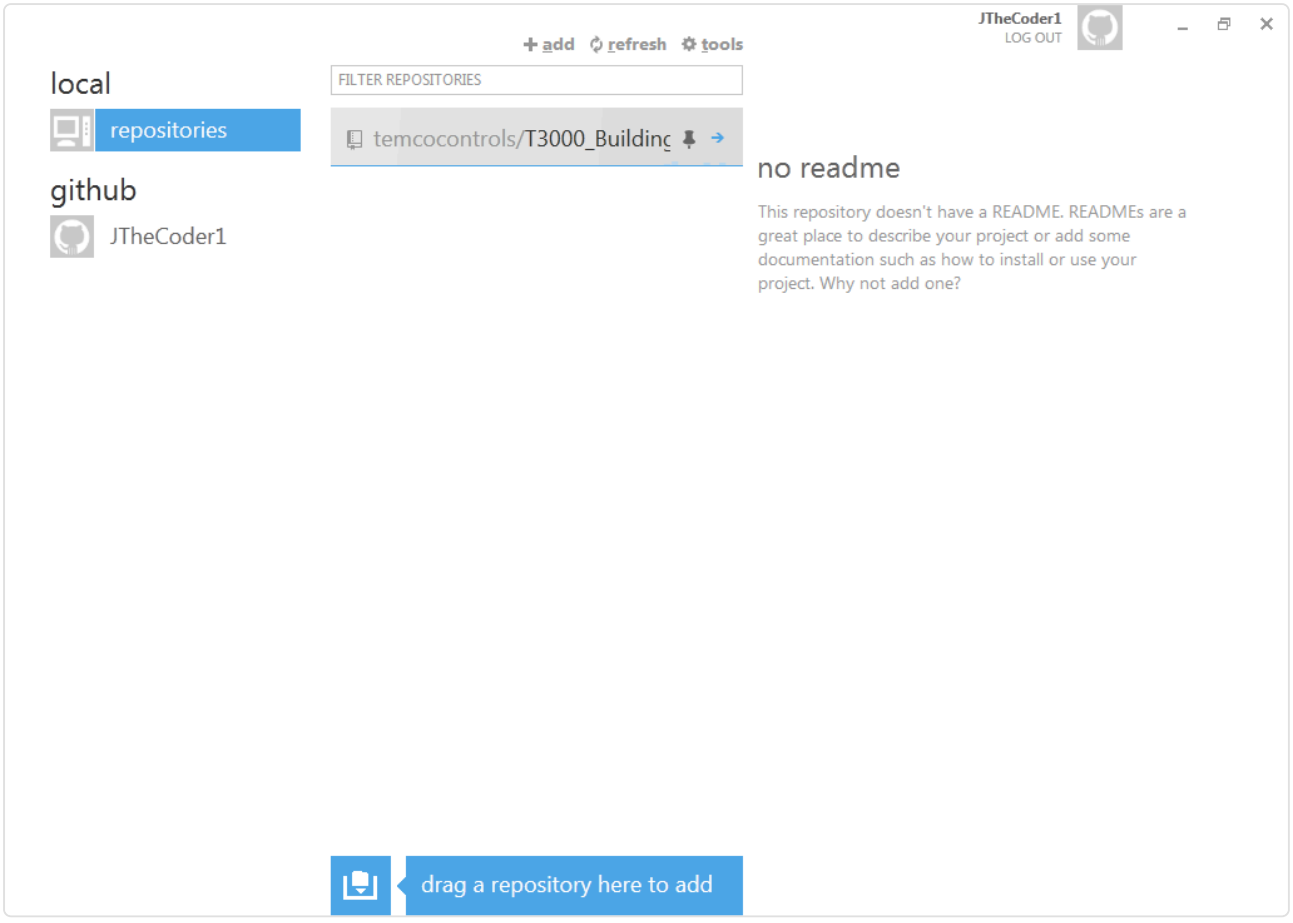
You may get following warning message. Click Allow.



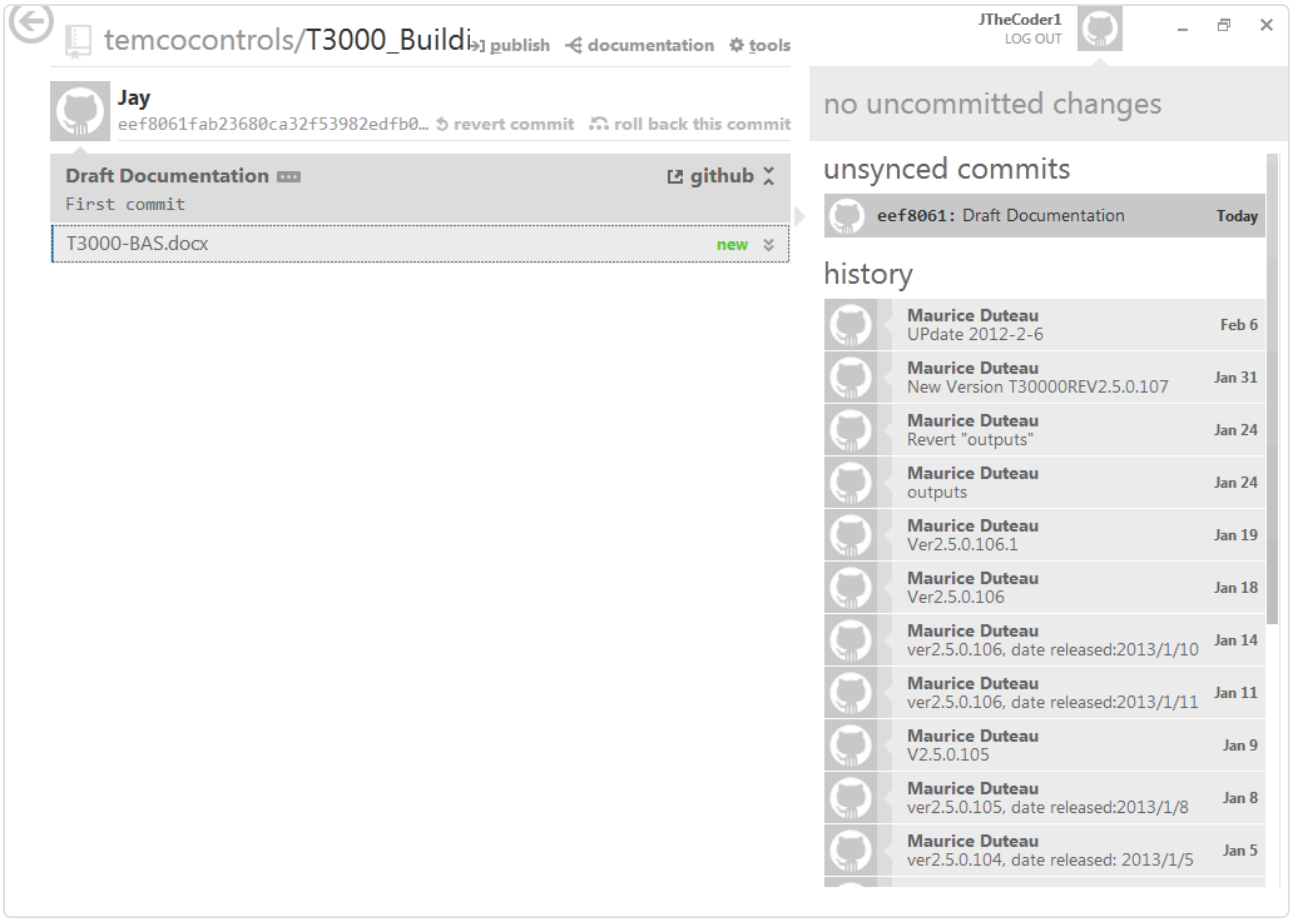
The repository starts getting cloned on local PC. The application shows the progress of the same. Be patient. The repository, depending on its size, takes few minutes to download.



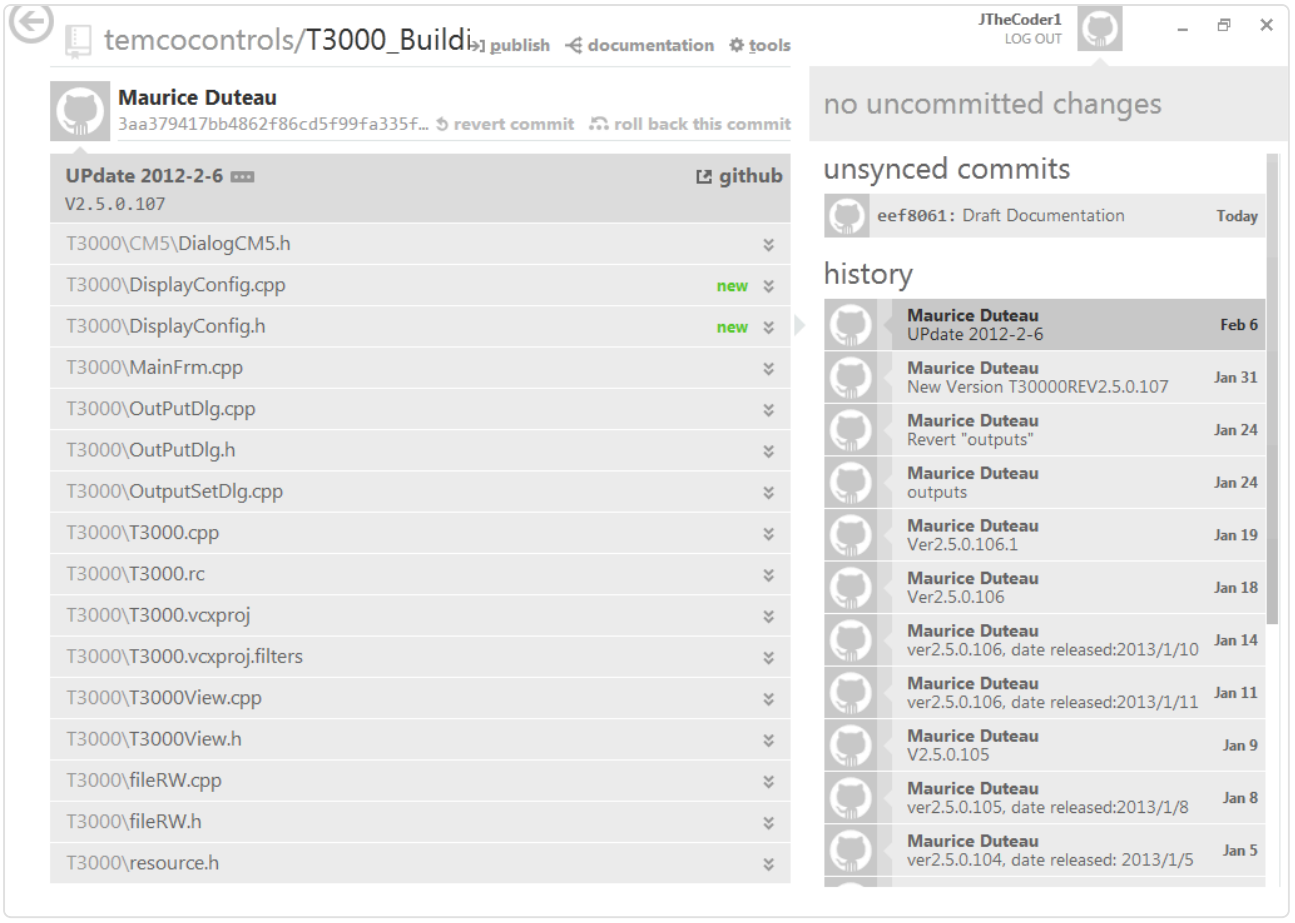
Once cloning is over, the repositories look like this.



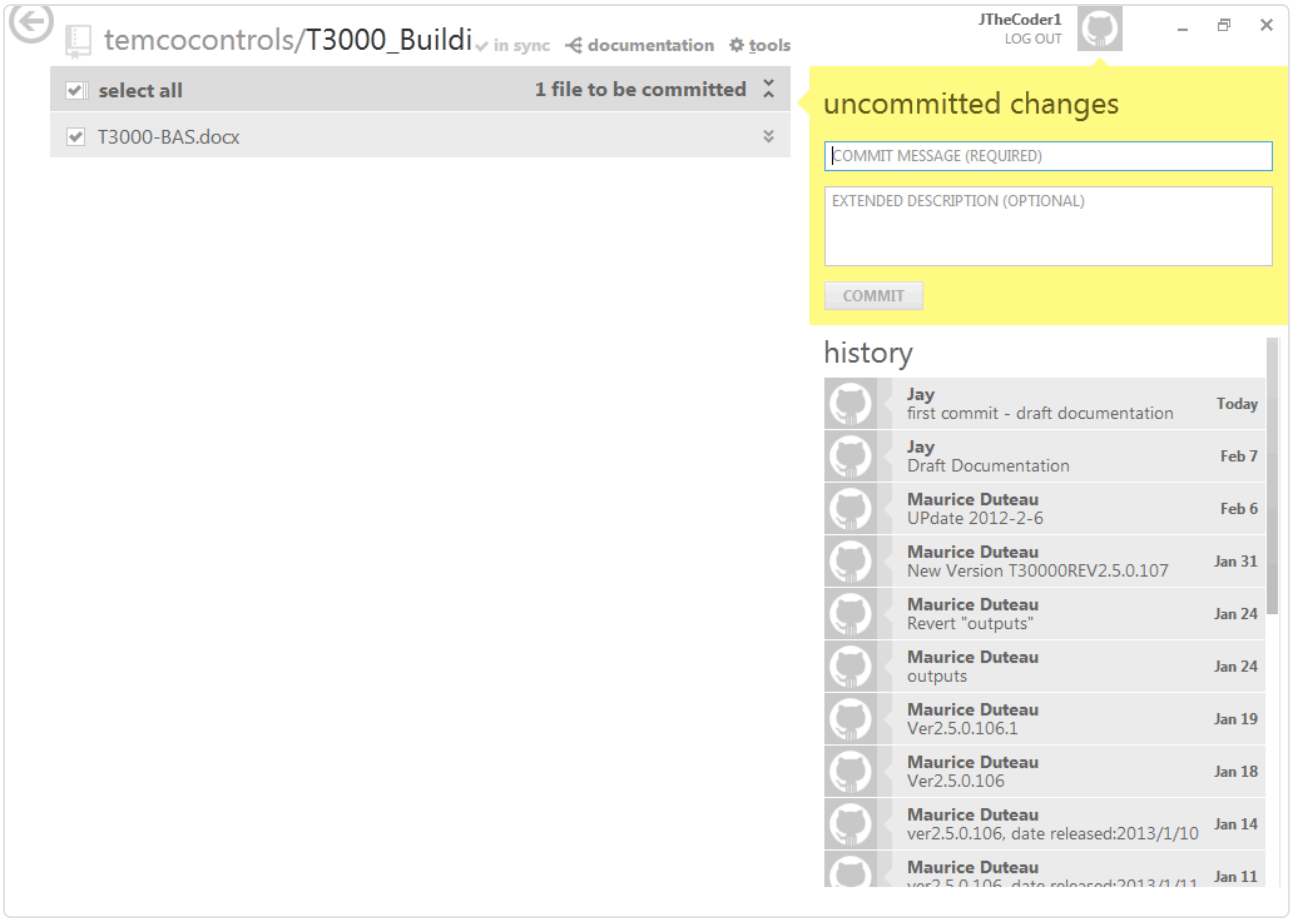
If you click on the small blue arrow on the repository, you would be taken to the following screen.



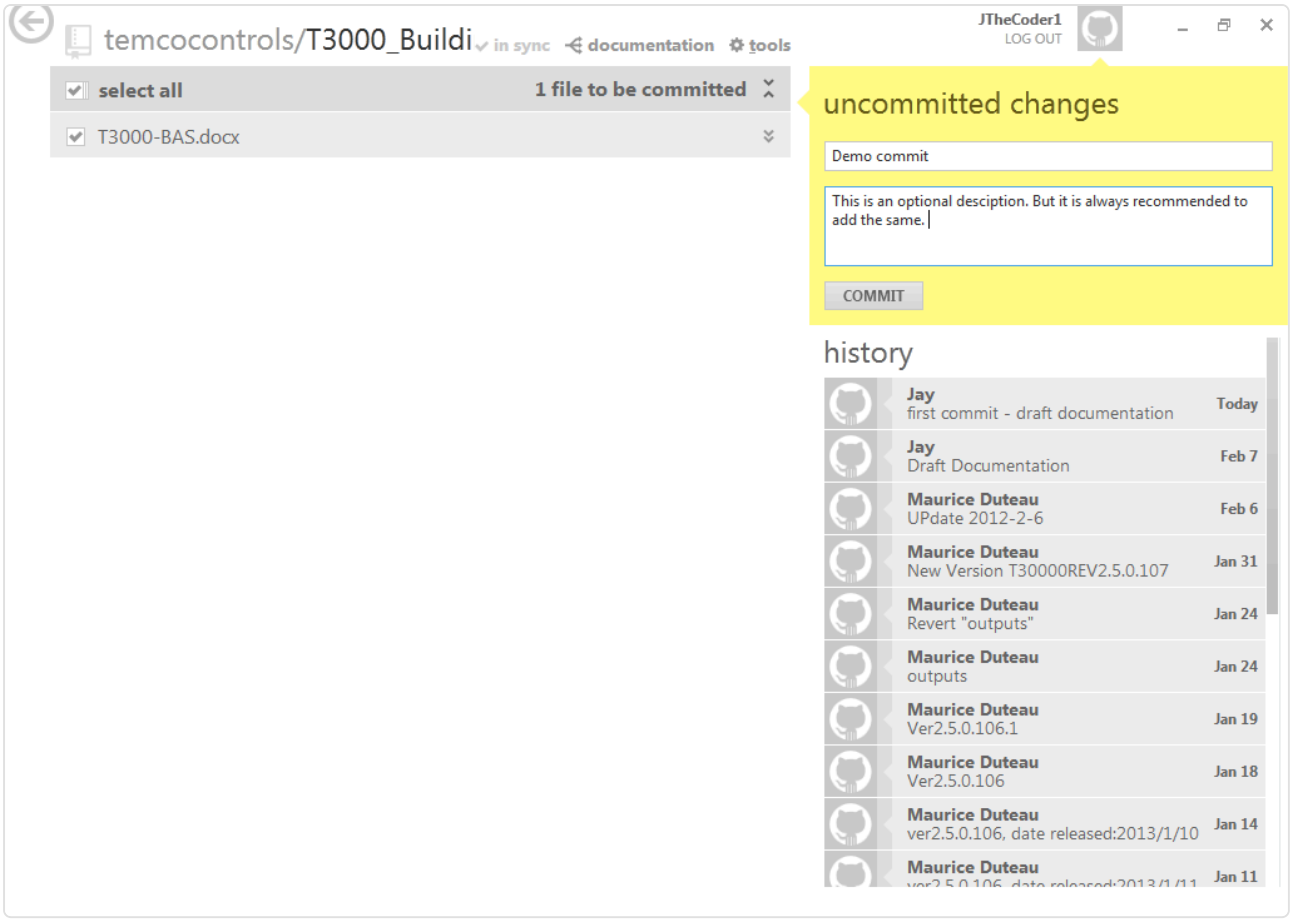
If you click on any of the links on history pane on right side, you get all the files that are there in the repository for that version.



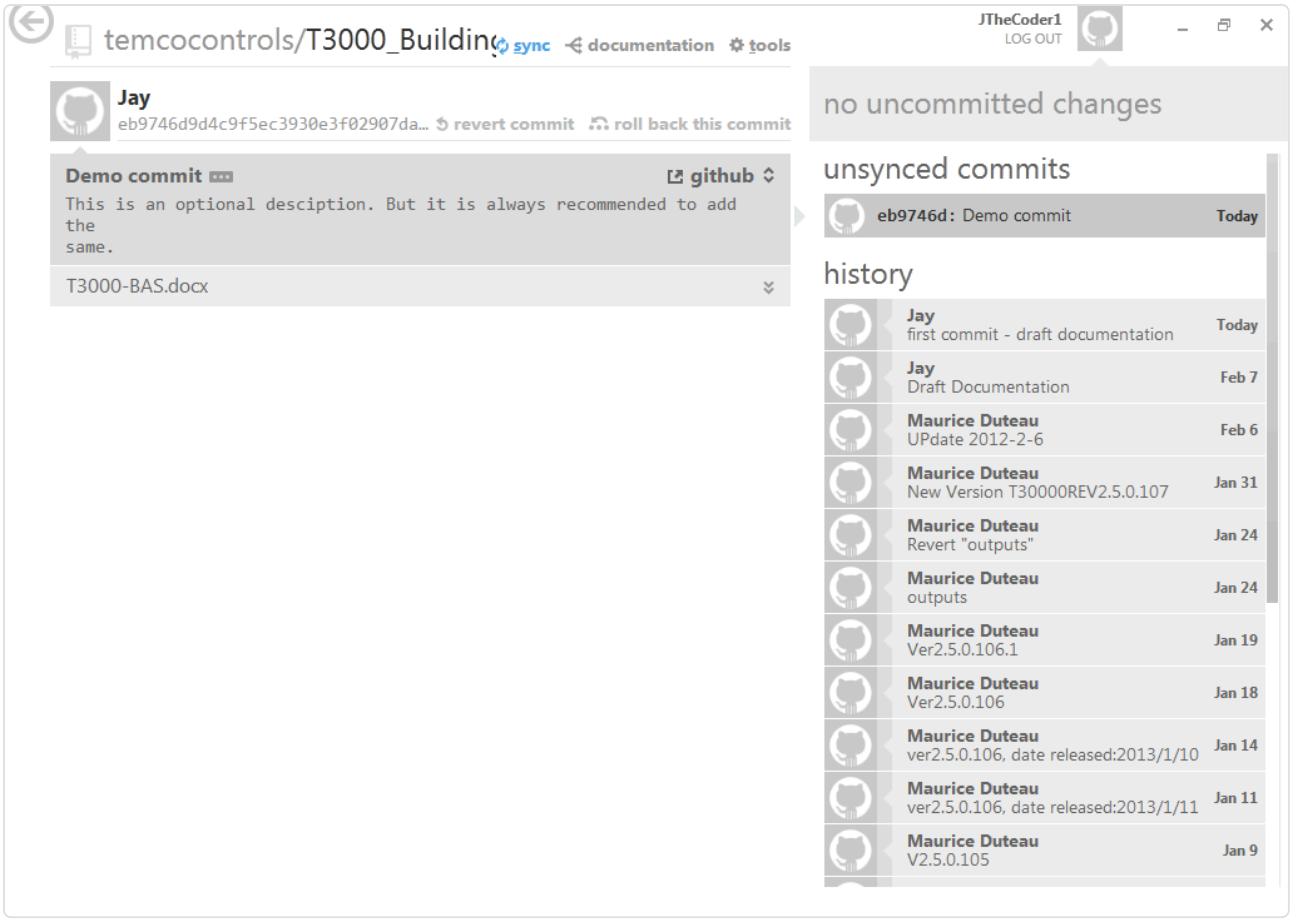
Once the file is modified in the repository, it appears on the screen as follows. I have the current file, which I am typing write now as the one which is to be committed to the repository. The same is shown on the screen.



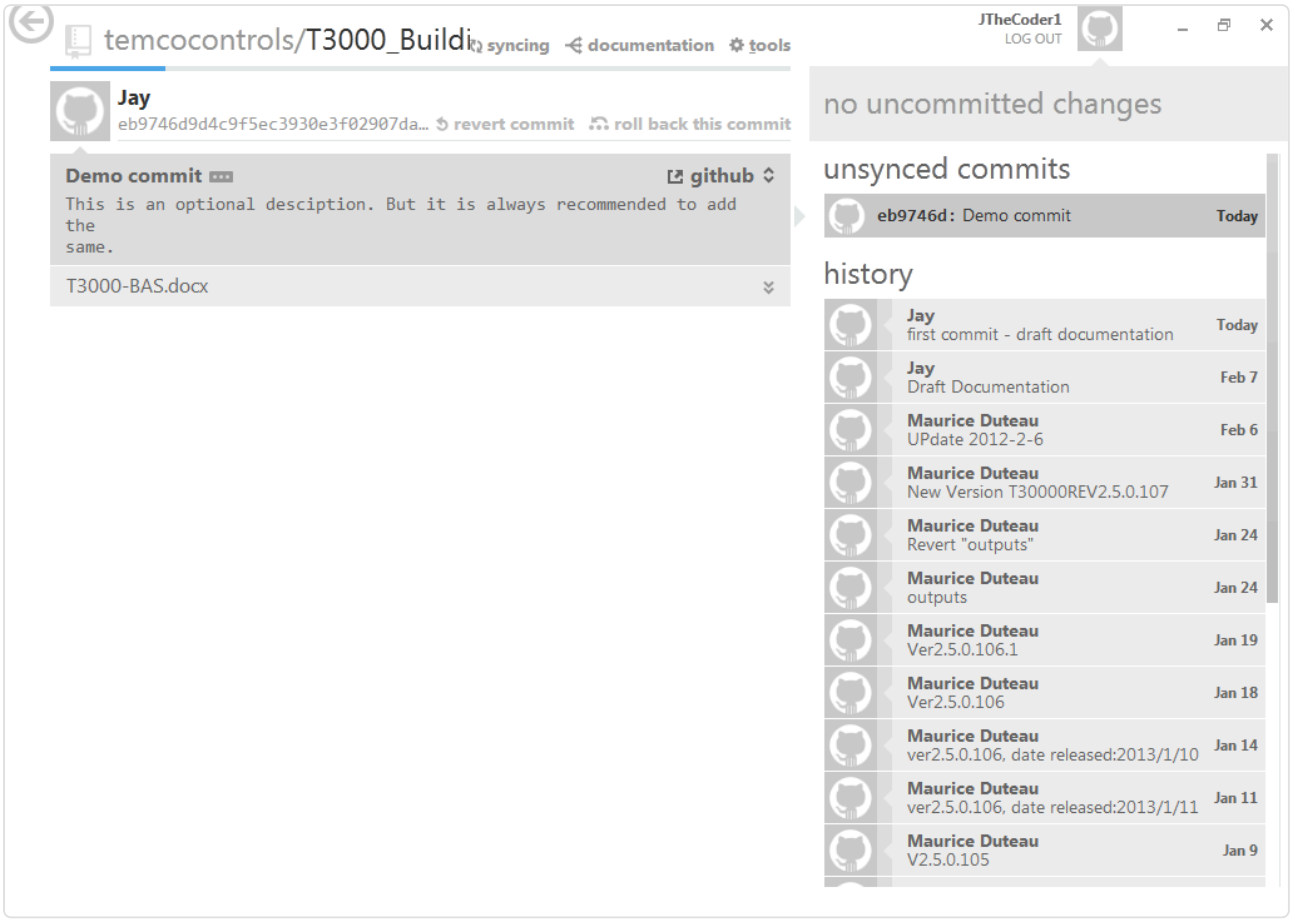
You can commit the change in the repository by adding a commit message and optionally an extended description. And then click on the COMMIT button given in the yellow area on right.



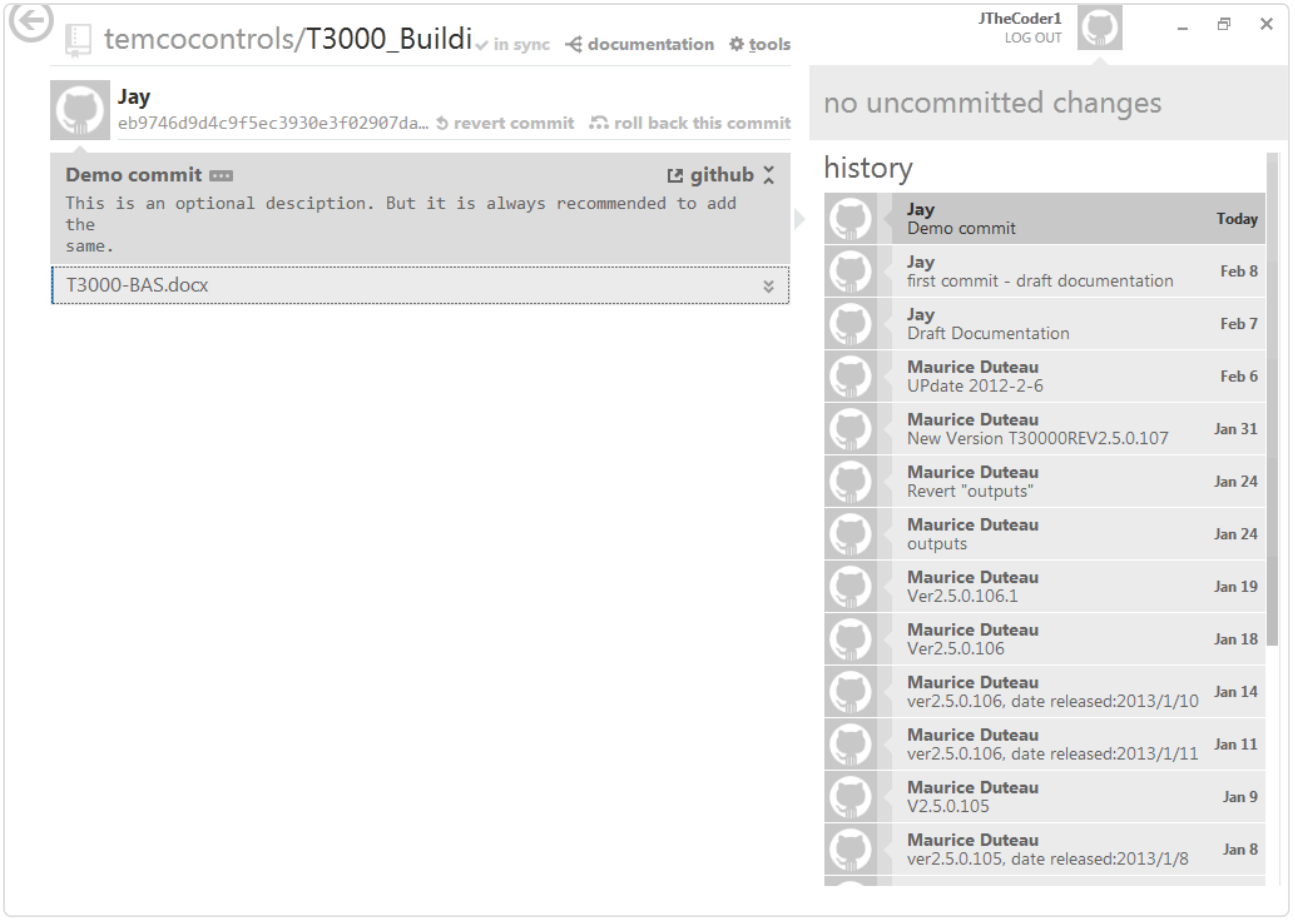
Once the COMMIT button is clicked, changes are committed to the repository. But they are not in sync with the repository on the github server. To push the changes to the github server, click on sync button as shown in blue in the following figure.



Following figure shows sync in progress.



Once in sync, the same is shown as follows.

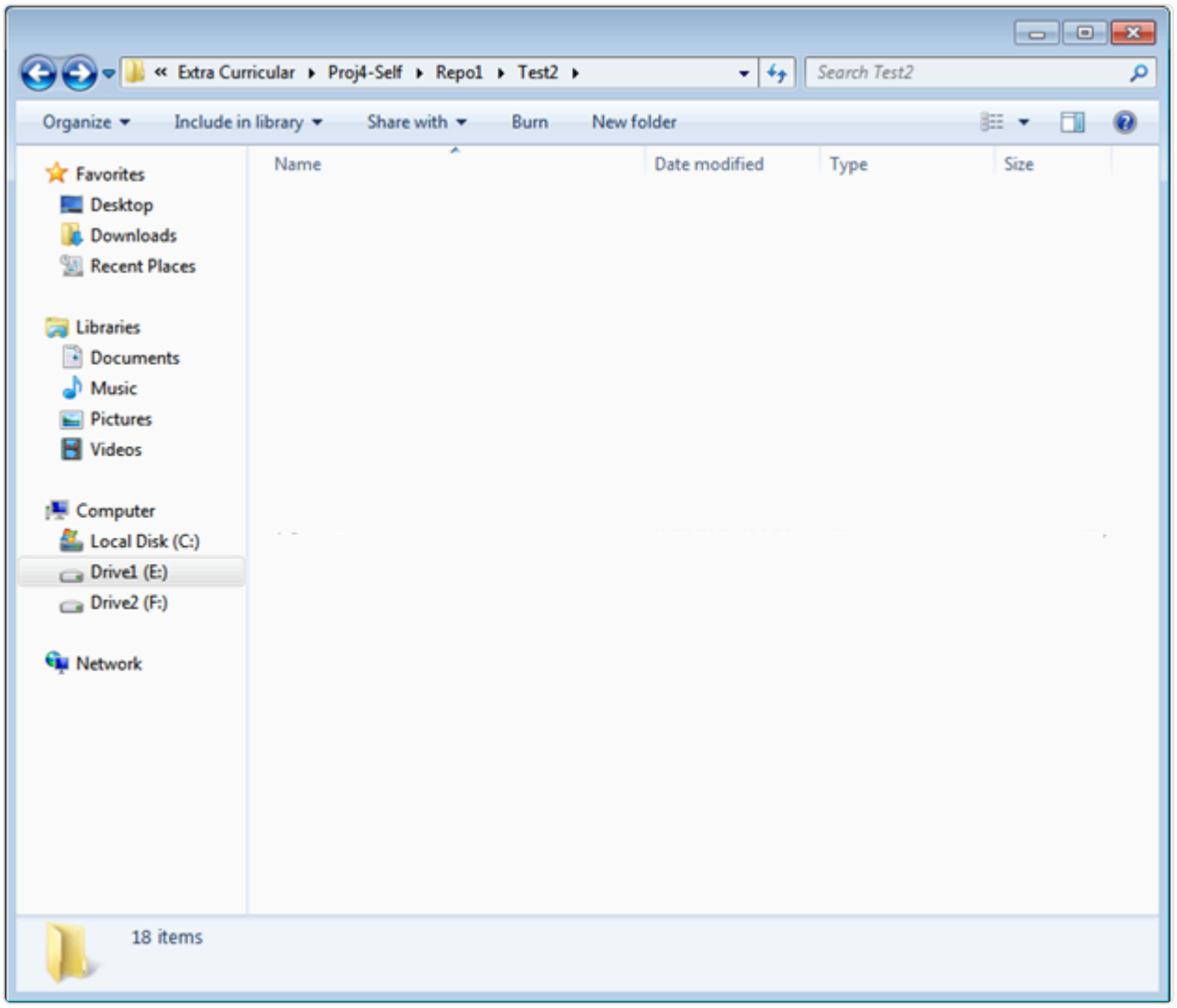


You can work with github and repository in this manner.

Procedure to Merge

First decide a folder on your PC where you want to create local copy of your git repository and git working directory.

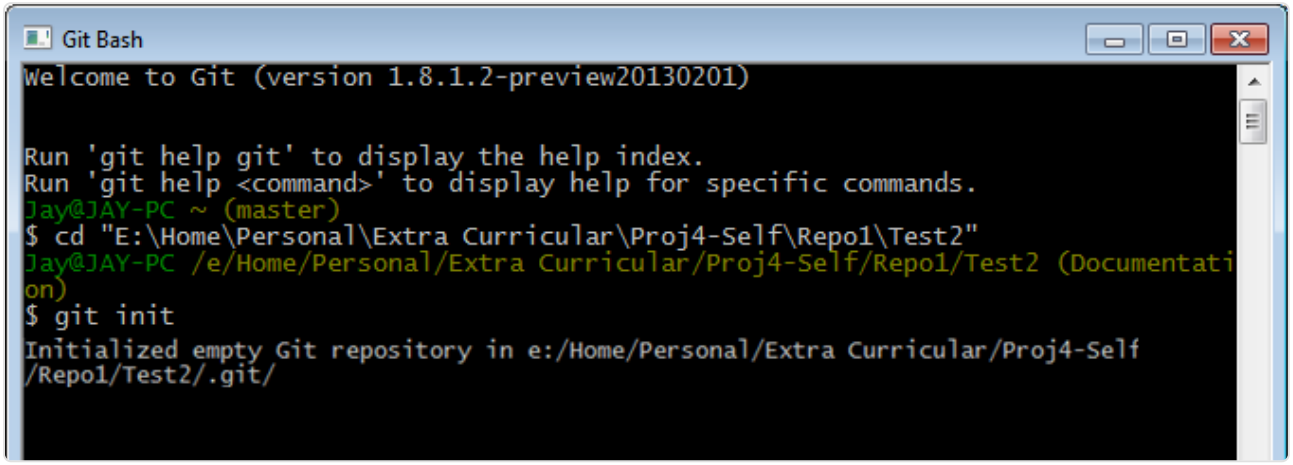
On my PC, this folder is “E:\Home\Personal\Extra Curricular\Proj4-Self\Repo1\Test2”.



Open Git Bash and type following commands.

$ cd <"path of folder">

$ git init



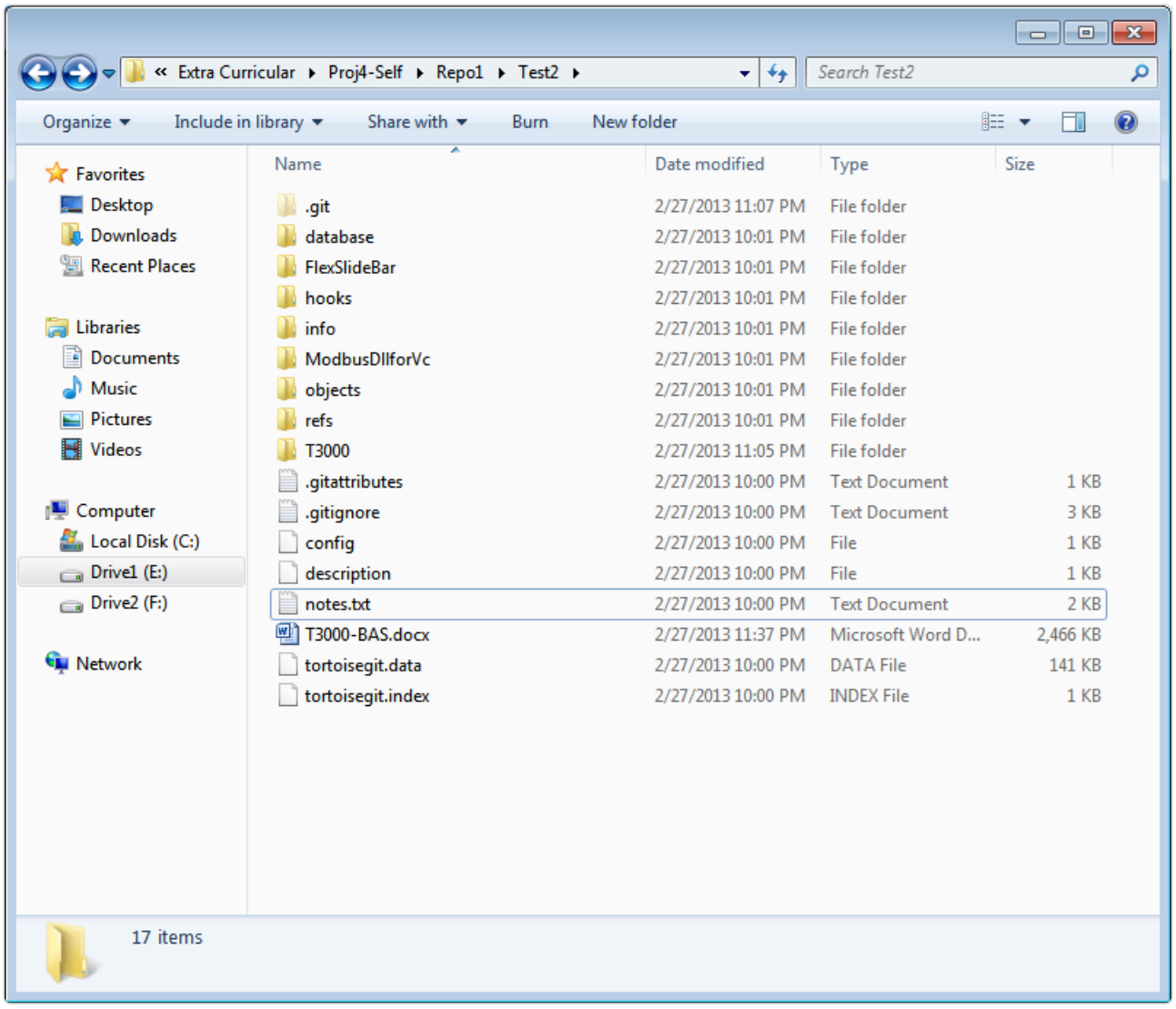
Next step is to clone repository of T3000_Building_Automation_System from github server. To achieve this, type following command.

$ git clone “https://github.com/temcocontrols/T3000_Building_Automation_System.git”

Upon executing this command, a message similar to the one shown in screen shot below appears. It might take 5 to 15 minutes depending on internet speed to clone complete cloning of repository.

```
$ git clone "https://github.com/temcocontrols/T3000_Building_Automation_System.
git"
Cloning into 'T3000_Building_Automation_System'...
remote: Counting objects: 623, done.
remote: Compressing objects: 100% (346/346), done.
remote: Total 623 (delta 289), reused 602 (delta 268)
Receiving objects: 100% (623/623), 61.83 MiB | 210 KiB/s, done.
Resolving deltas: 100% (289/289), done.
```

Once the cloning is complete, folder on PC looks like as shown below. There would be a hidden sub-folder named “.git” which is local git repository. Outside of it Is working directory.



Now, use following git commands to do tasks as desired. I am documenting normal steps that would be followed by development team (Alex) and documentation team (Jay).

Both documentation and development teams will have to clone git repository from github.com.

Run following commands to determine the branches present in the repository. Remember that once cloning is done, all these commands would work on local repository and not on repository on github server.

$ git branch

This command lists all the branches present in the local repository. In the list which appears following this command, there would be one branch with * marked. This is the current branch of the repository.

For basic concepts of git branch read following material:

http://git-scm.com/book/en/Git-Branching-What-a-Branch-Is

For complete tutorial on branching and merging refer to chapter 3 of Git Book.

http://git-scm.com/book/en/Git-Branching

Let's say there are two branches:

Master and Documentation.

Documentation team shall move to Documentation branch by executing following command.

$ git checkout Documentation

This command loads all files and folders of repository to working directory.

Documentation team shall add its comments to all the source code files. Once, a particular level of commenting is reached, changes shall be committed to git repository by executing following commands.

$ git add "."

$ git commit -m "commit message"

'git add' command adds files to the staging area and marks them for the commit (To understand this concept, read git theory from Git Book: http://git-scm.com/book/en/Getting-Started-Git-Basics). 'git commit' command actually commits staged files to repository.

Once the Documentation branch is updated, documentation team shall push it to github for the world to see it.

[I will update this procedure soon. I am still having some doubts on this, so trying to clarify from various sources of mine.]

Development team can fetch the contents from github server using following command.

$ git fetch

This command fetches all the new changes that have been pushed by documentation team to the local repository of development team.

Then development team may switch between master and documentation branches and go through the changes – that is, the commenting done by documentation team. It may decide to merge the changes of documentation branch to master branch using following command.

$ git checkout master

$ git merge documentation

These commands merge documentation to master branch. Now, the changes done by documentation team should have been reflected in master branch.

Once, this is done, development team shall push the changes to github and inform documentation team.

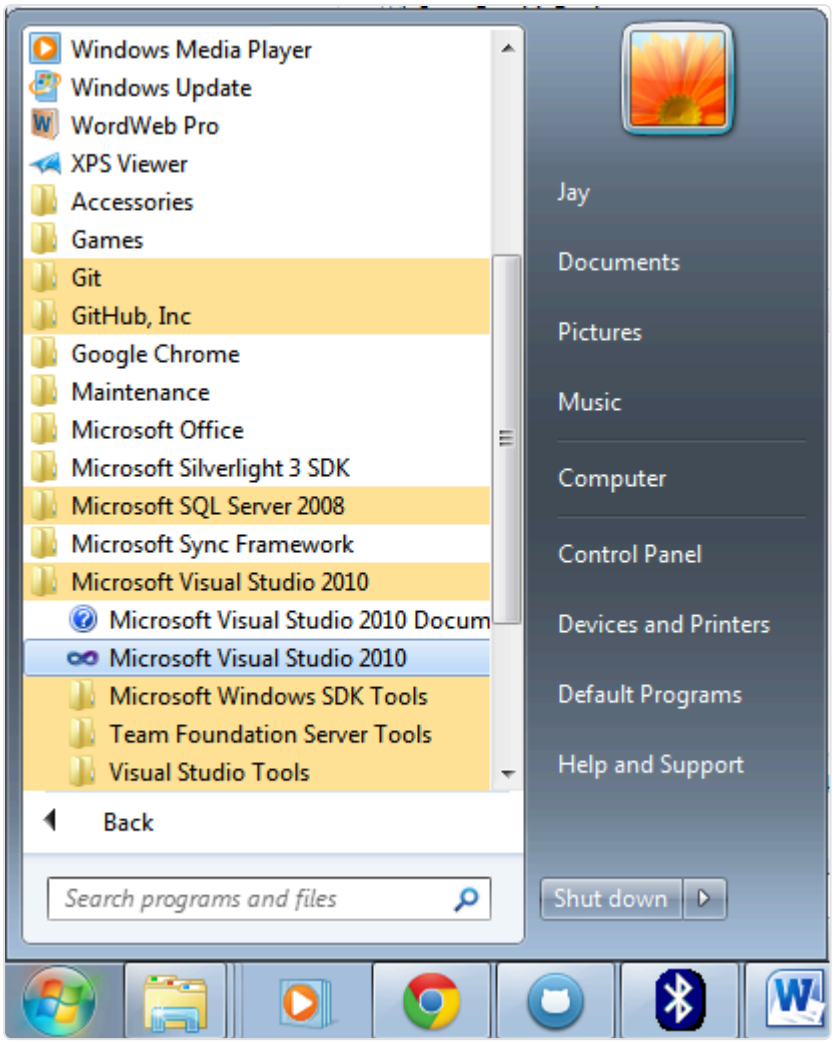
This way, both teams can collaborate.

[This procedure may change in near future as roughness would be smoothened out. Till then keeping this statement as it is.]

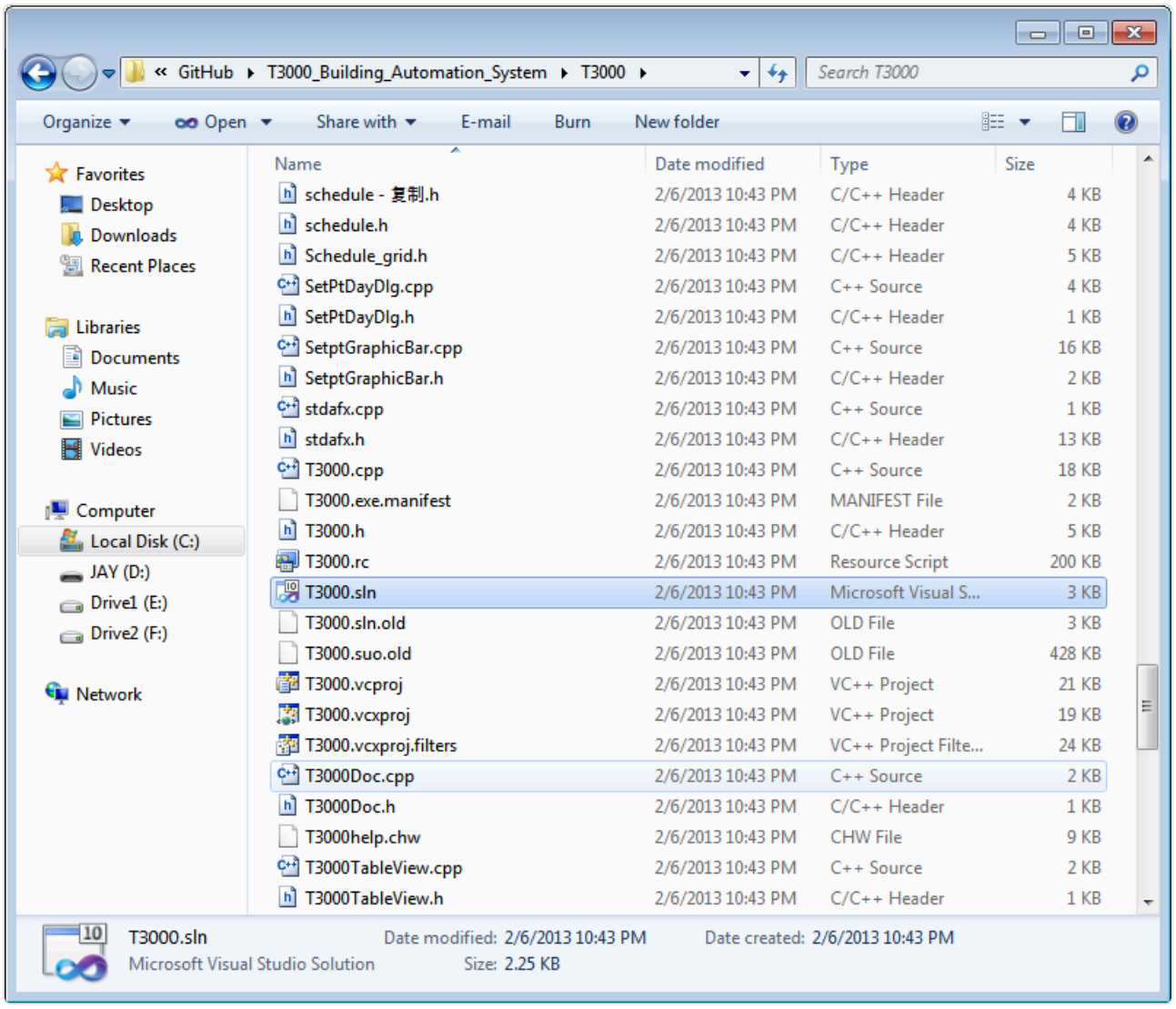
Compilation on Microsoft Visual Studio 2010

Install MS Visual Studio 2010 on your PC.

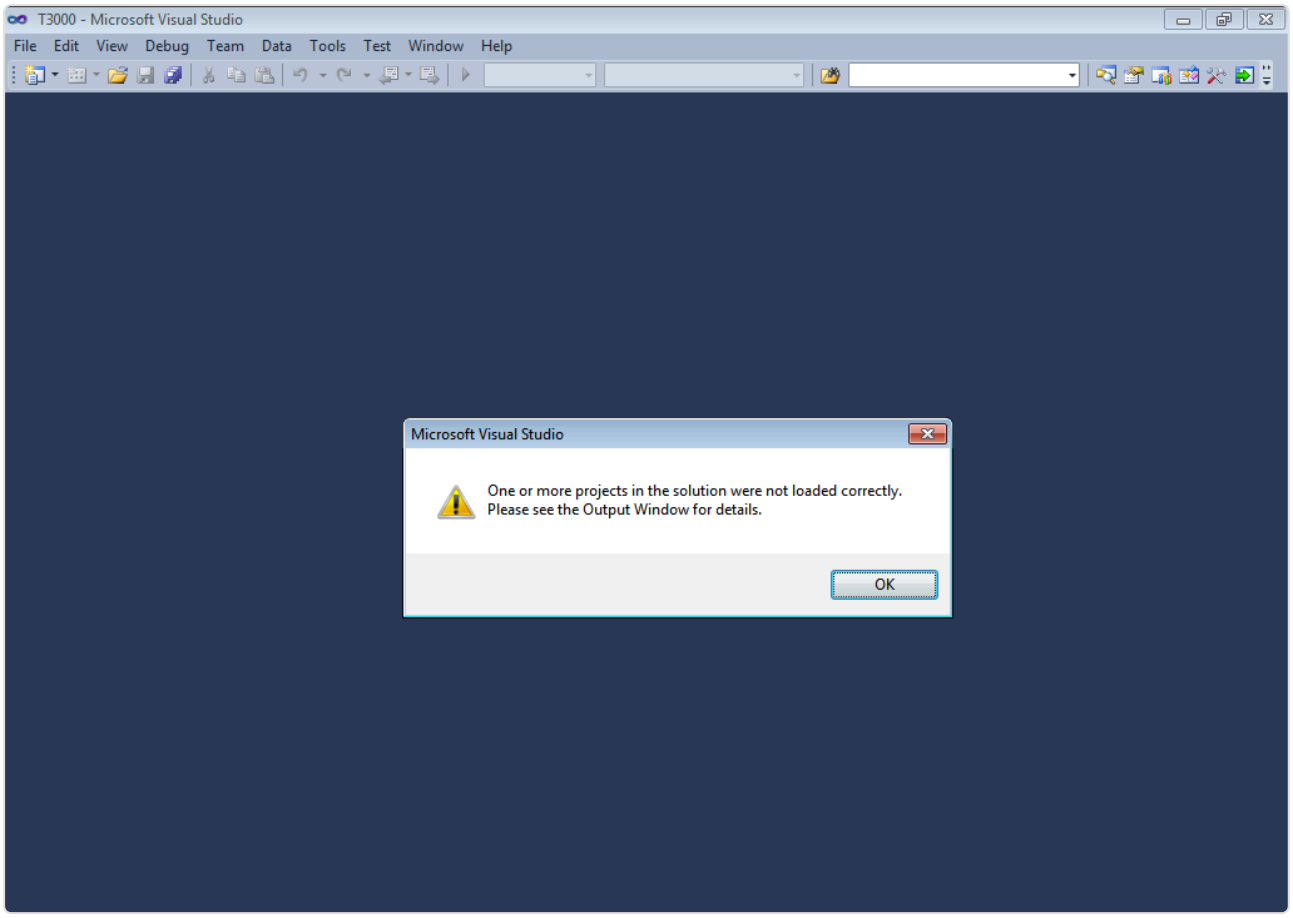
Start MS Visual Studio from the main start menu.



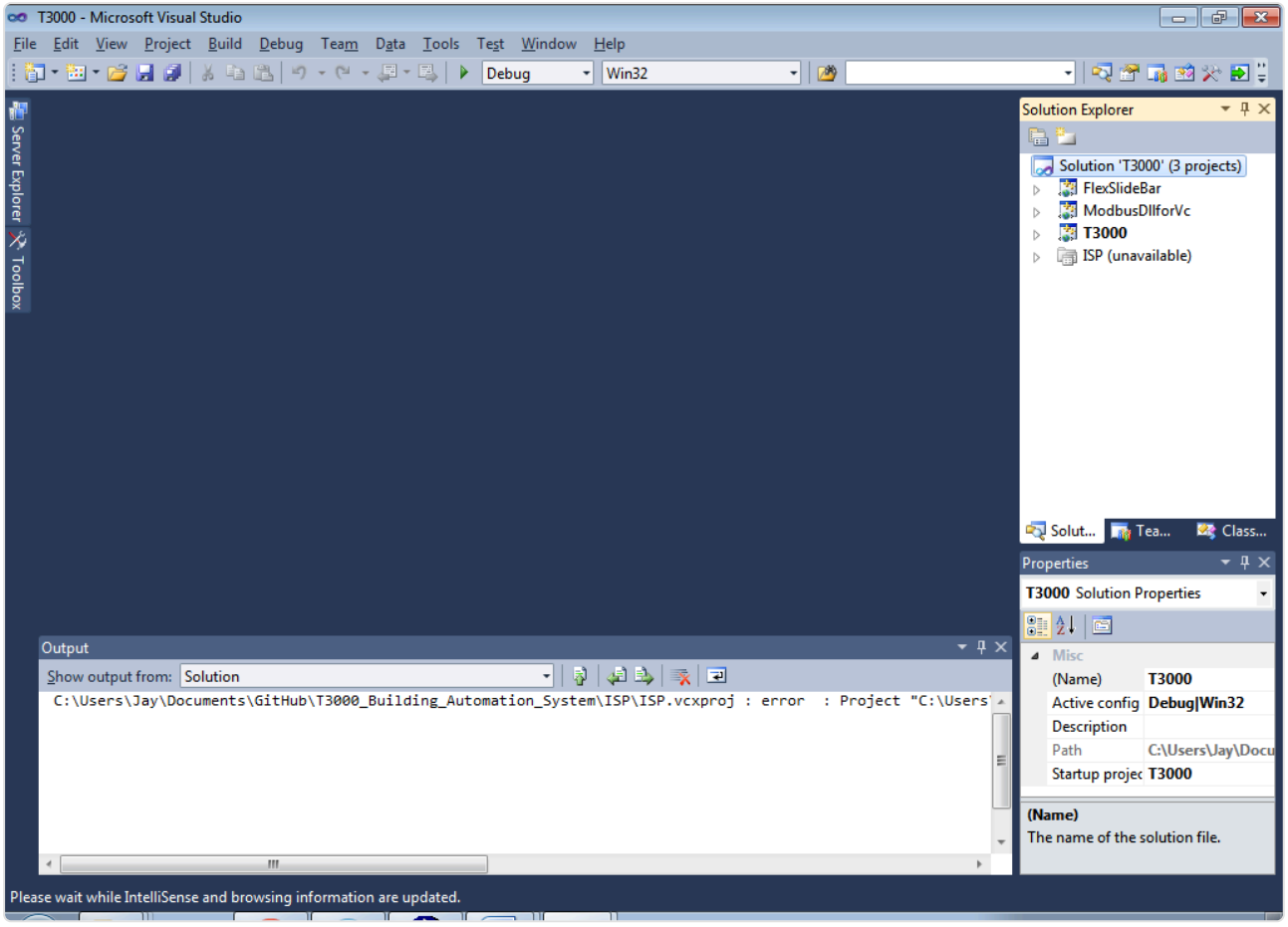
Go to the repository directory. Go to T3000 folder. Click on T3000.sln.



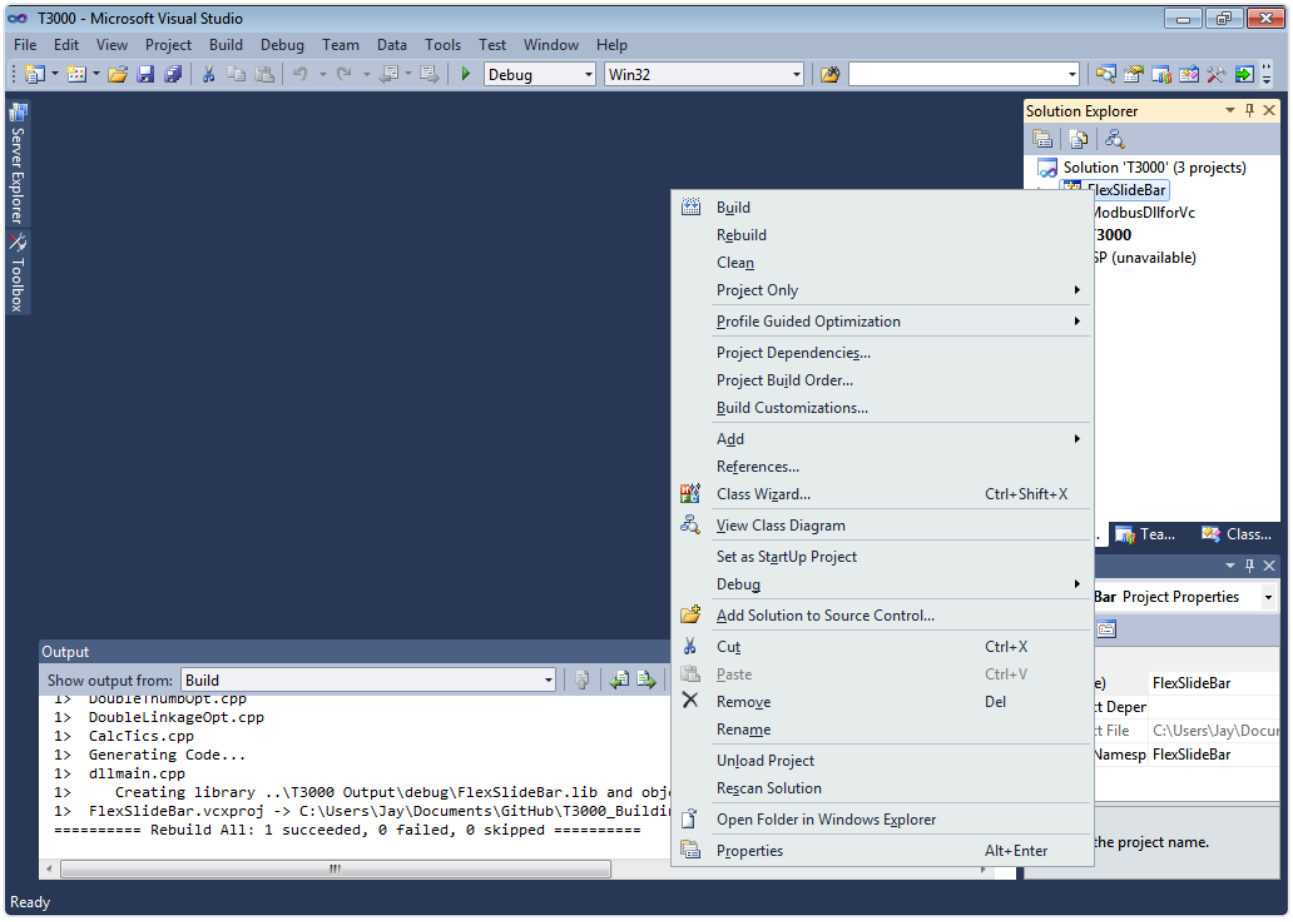
An error window appears as follows. [Right now, not sure why is this error coming? Anyway, capturing it in this doc as of now. We will remove this once the issue is solved.]



The following screen appears.



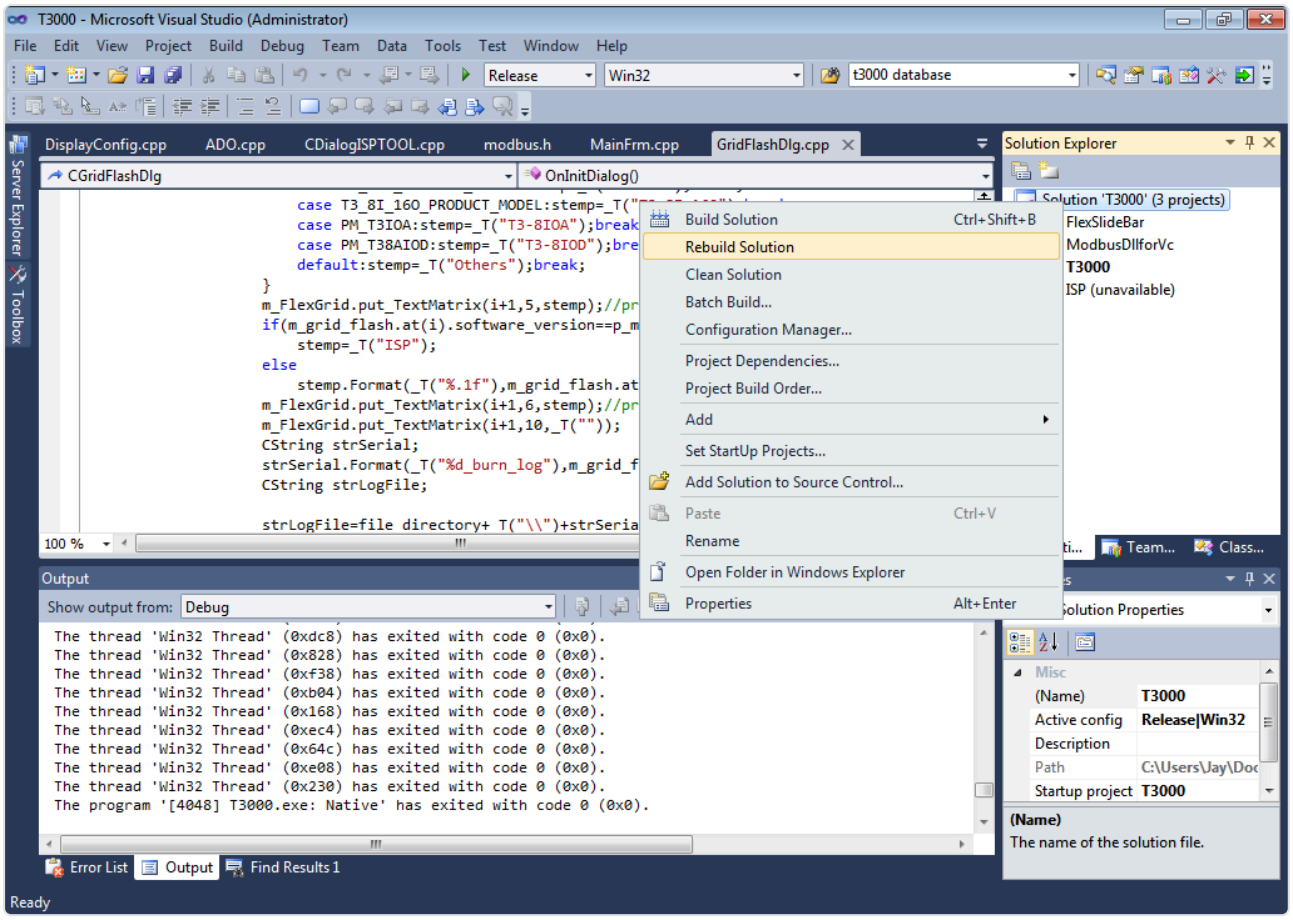
Right-click on FlexSlideBar project. And click Rebuild. This shall compile FlexSlideBar project.



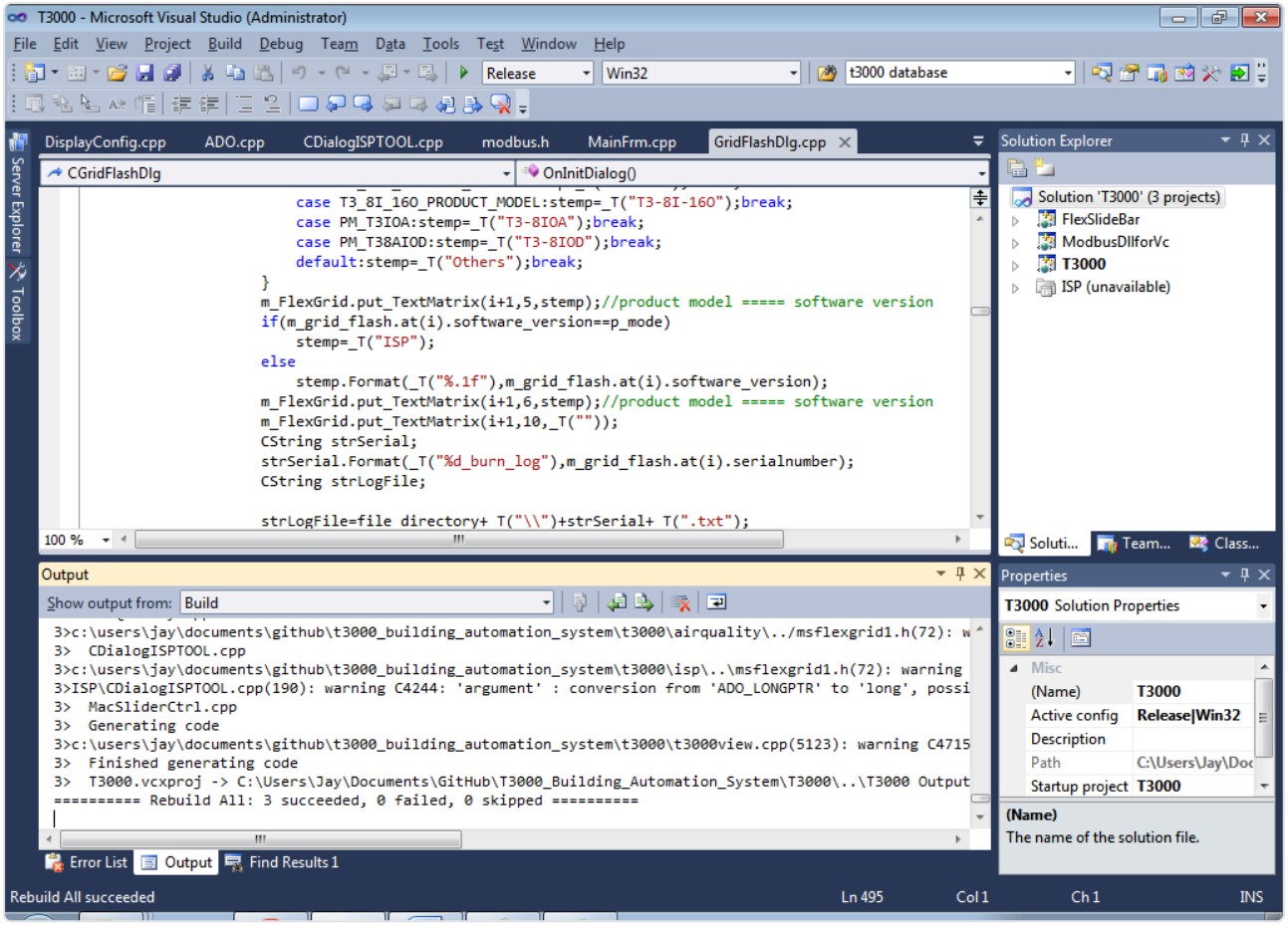
Similarly compile ModbusDllforVc project by right-clicking it and clicking Rebuild.

Similarly compile T3000 project by right-clicking it and clicking Rebuild.

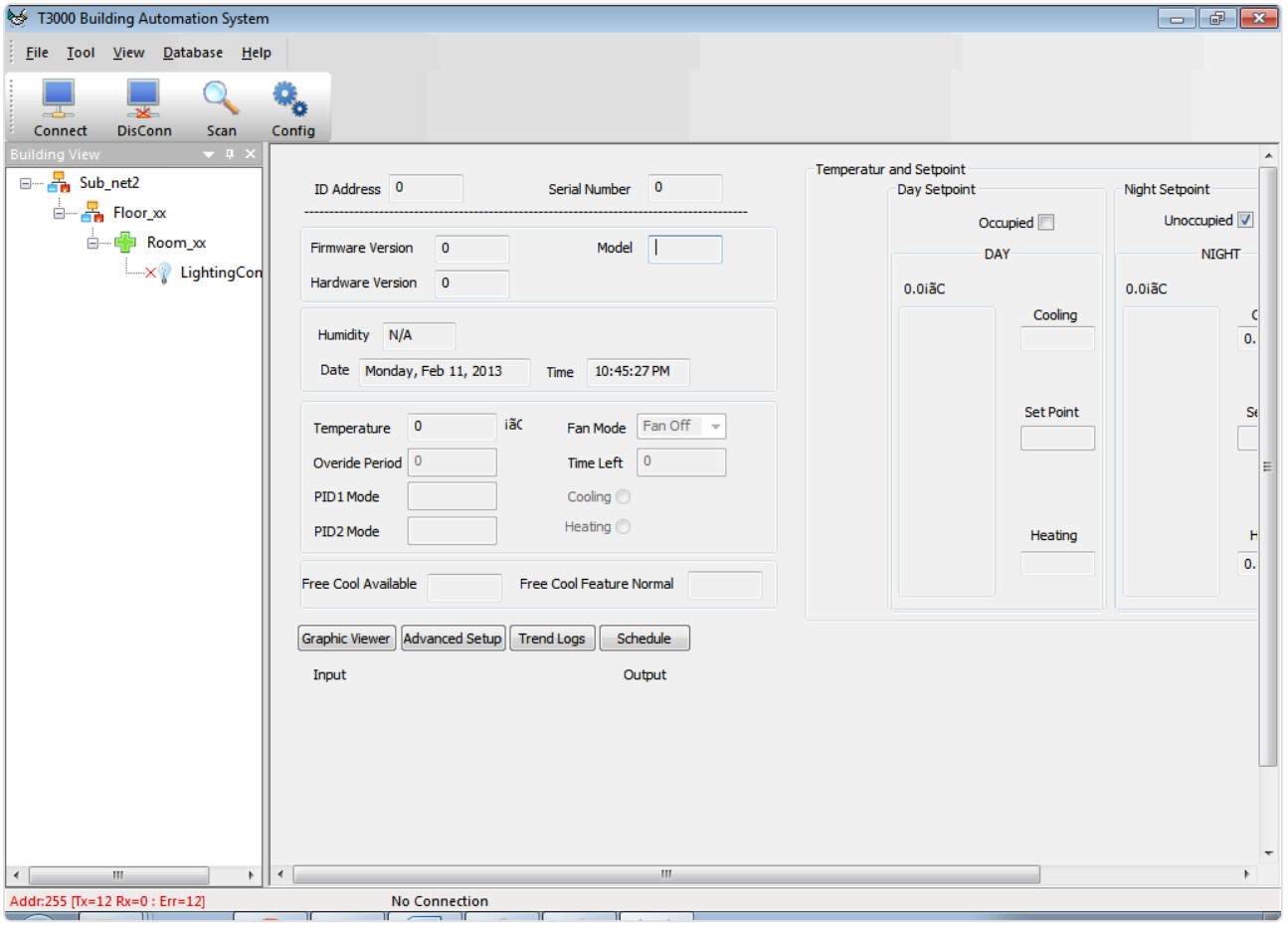
Another way to compile all the projects is to right-click on solution T3000. And click on Rebuild Solution.



Once the code is compiled successfully, Output window of IDE shall show some text as shown below.



Once the code is compiled and it is Run, following application runs.



Git commands summary

git config:

To set configuration parameters of git.

git help

To get help from man pages for git or any of its specific commands.

git clone

To clone a git repo from a remote repository.

git status

To get status of files that are to be staged / committed to the repository.

git add

To mark files to be added to the repository in the next commit.

git commit

To commit files to the repository.

git diff

To check differences between two versions.

git log

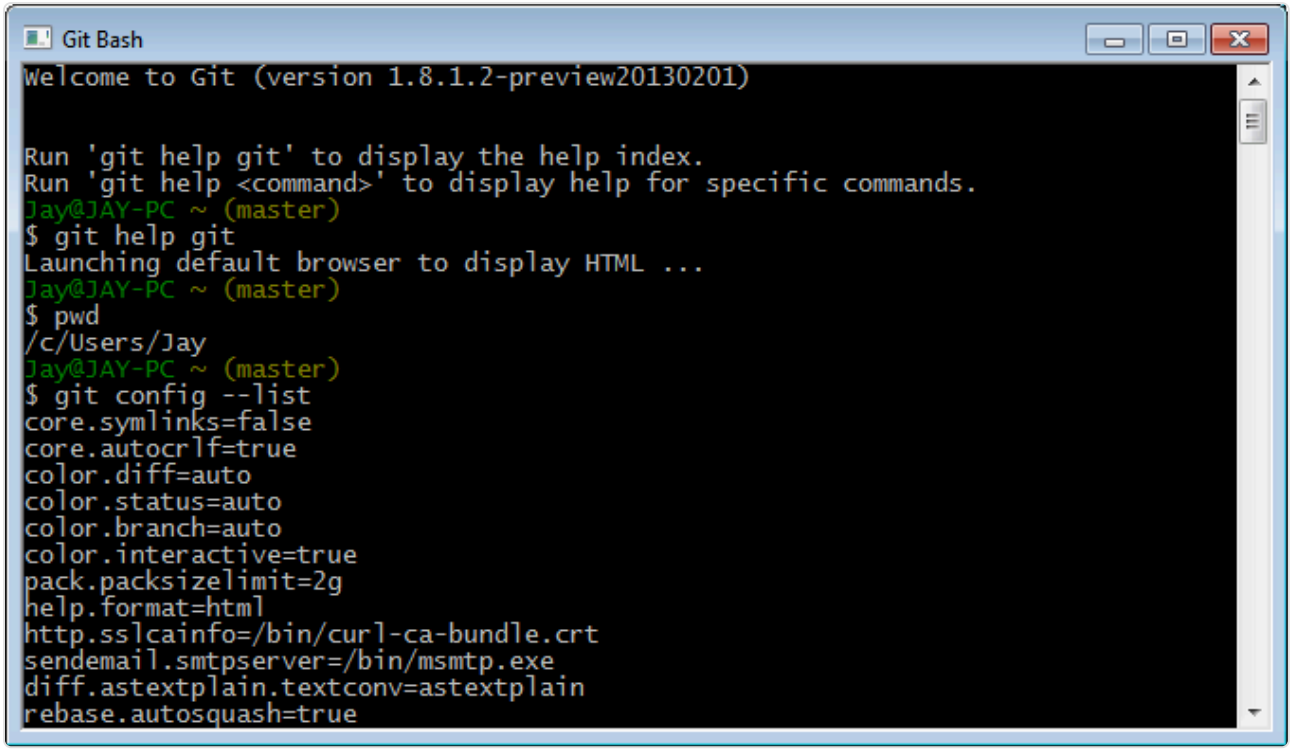
To view commit history.

gitk

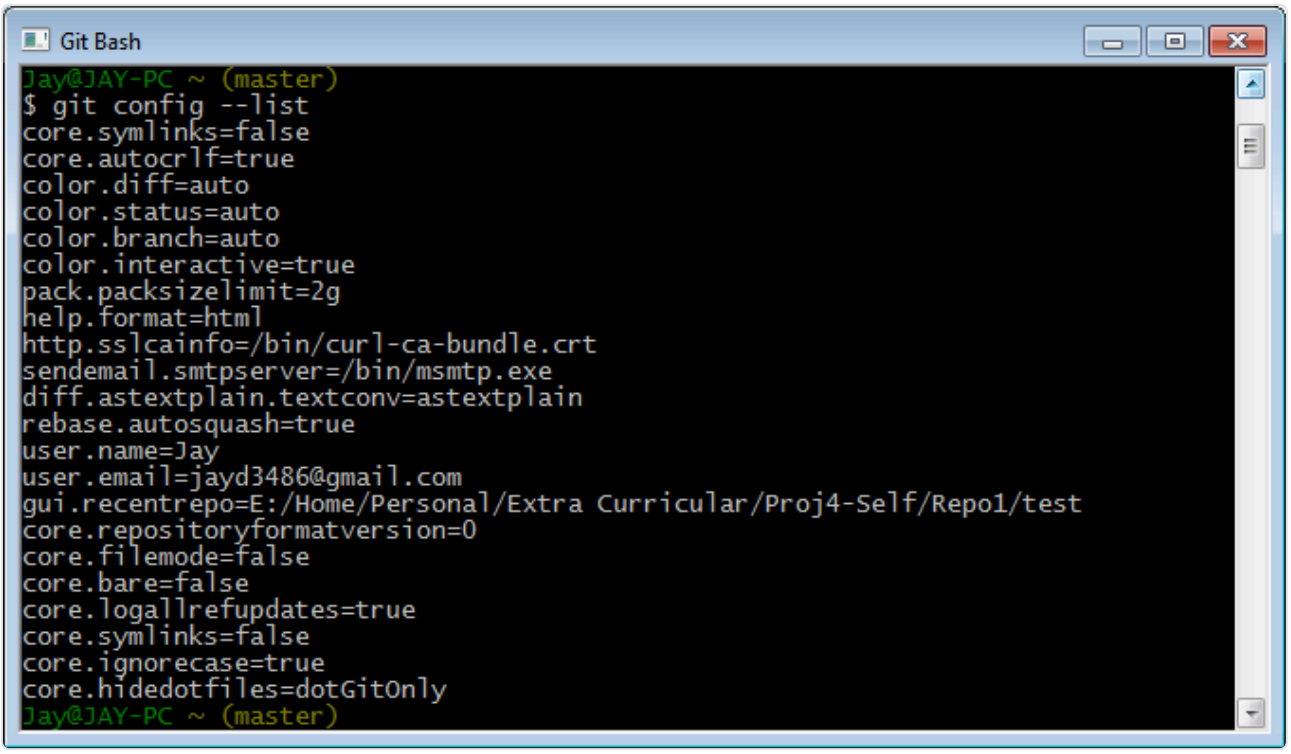
To invoke a GUI to visualize history.

Cloning a repo from github.com using Git Bash Shell

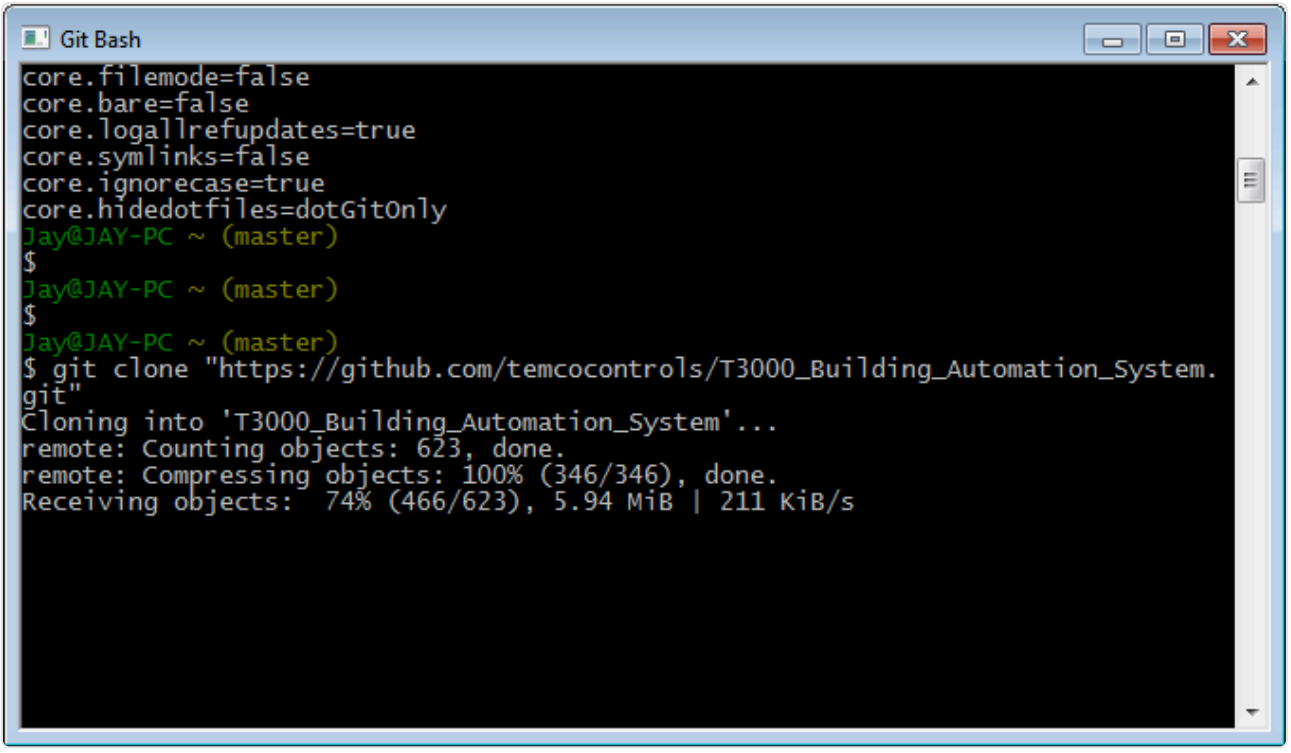
Use pwd (present working directory) command to know your working directory.



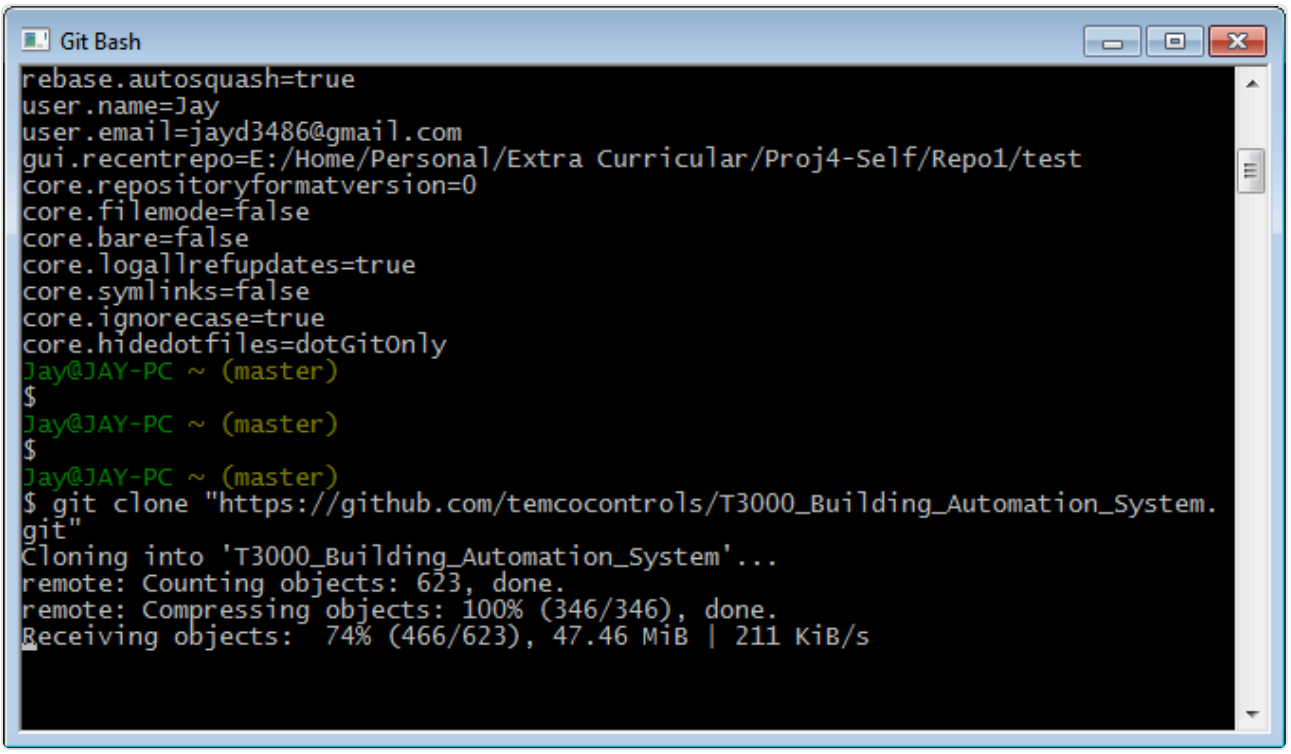
You can use git config –list to know present configuration settings of git.



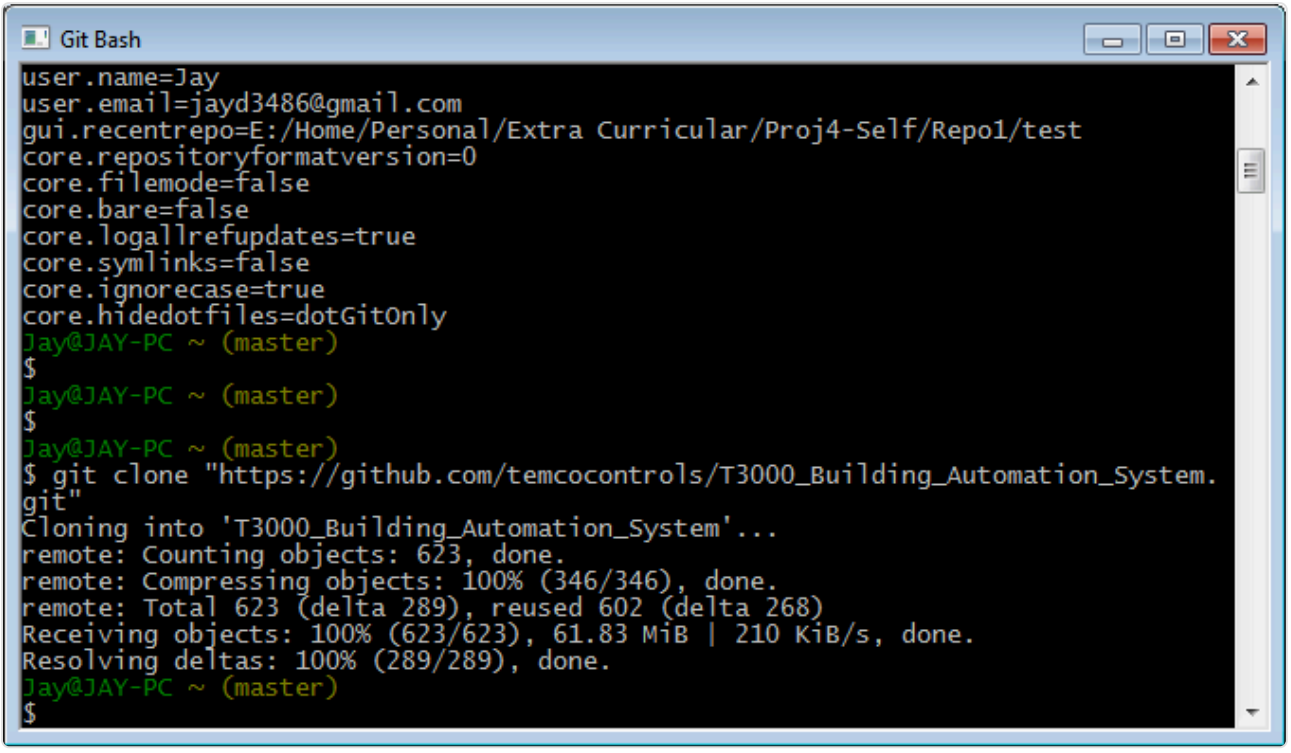
Use clone command as shown below to clone T3000 project from github website.



Continuation of the same.



Cloning done.



TortoiseGit

Setting up of TortoiseGit involves following steps:

Installation of msysgit, the windows version of Git

Installation of TortoiseGit

Sign up for a github account

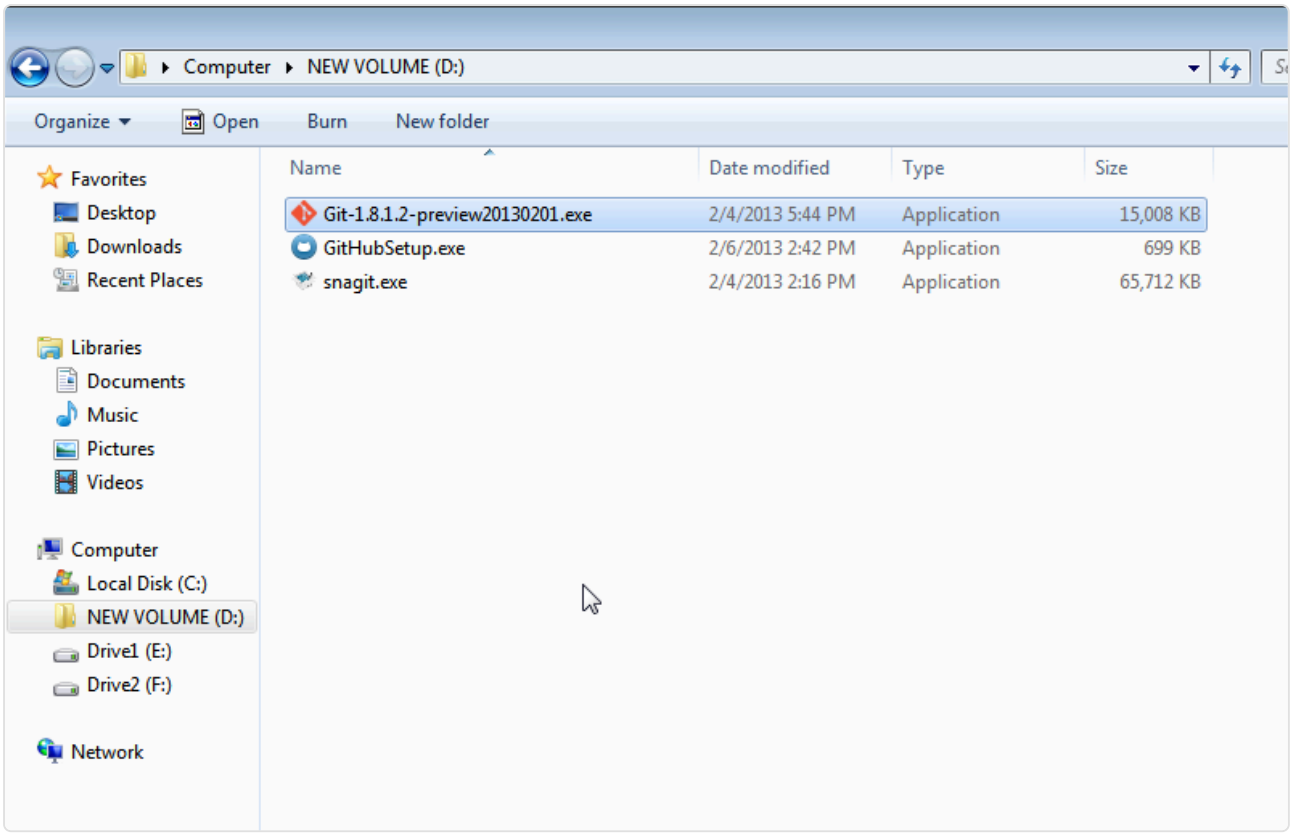
Setting up SSH key

Working with repository

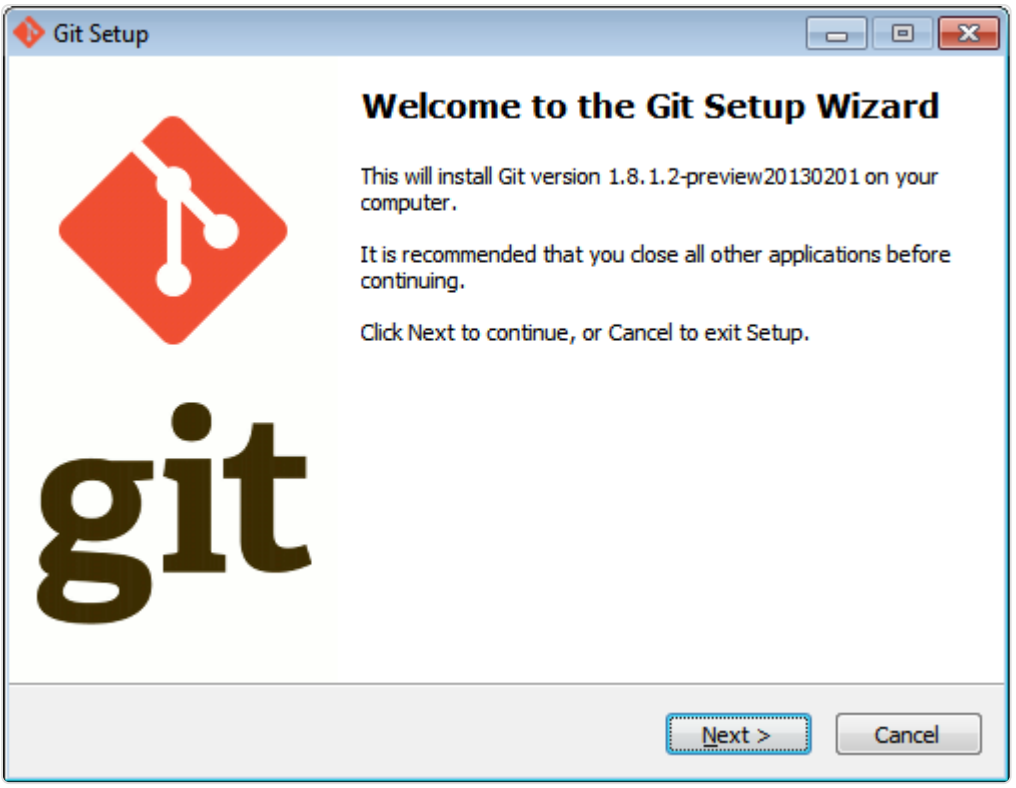
Installation of msysgit

Download git installer.

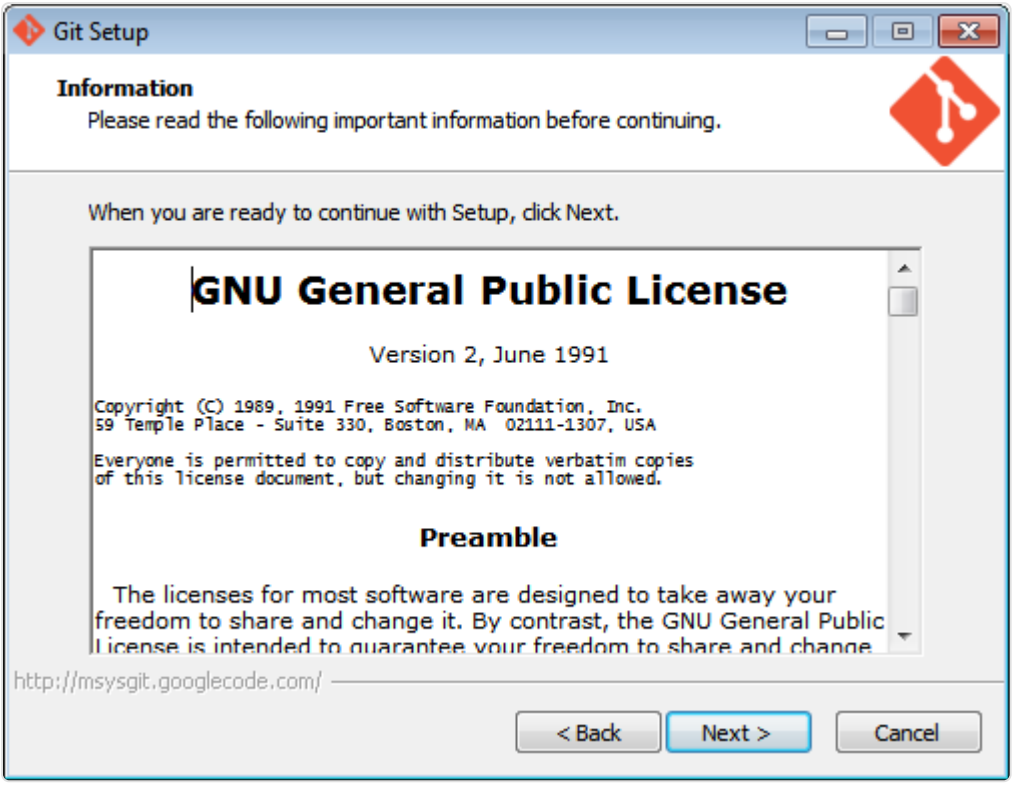
Double click on the setup file.



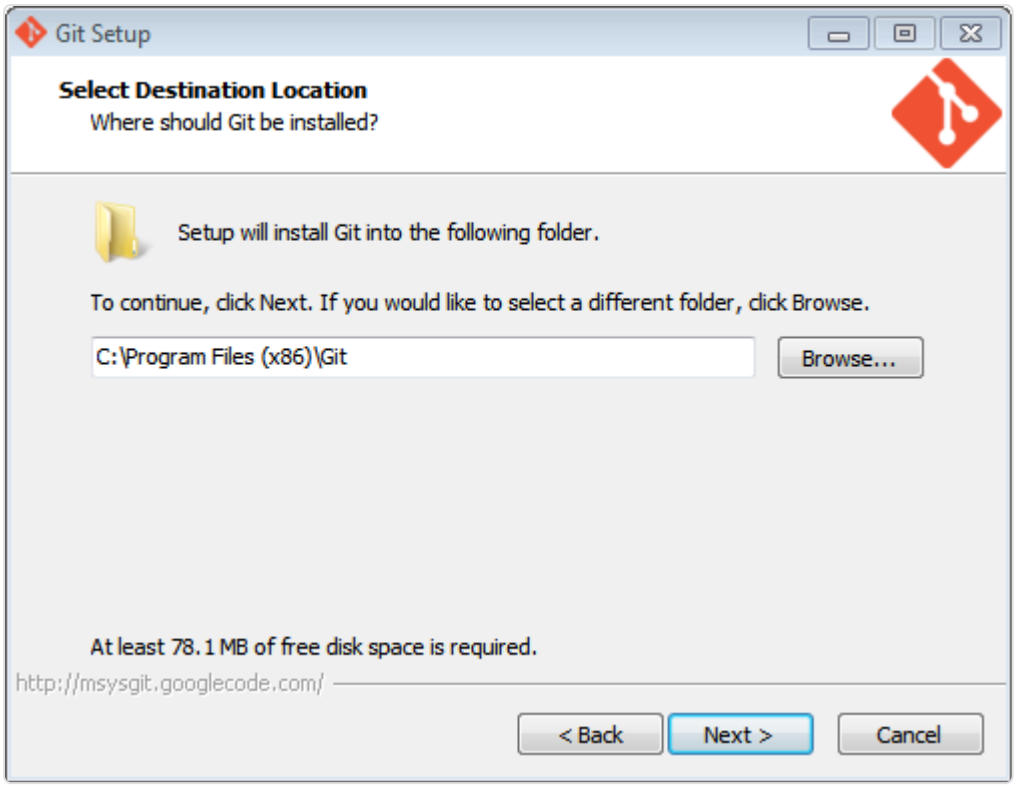
Following dialog box opens. Click Next to continue.



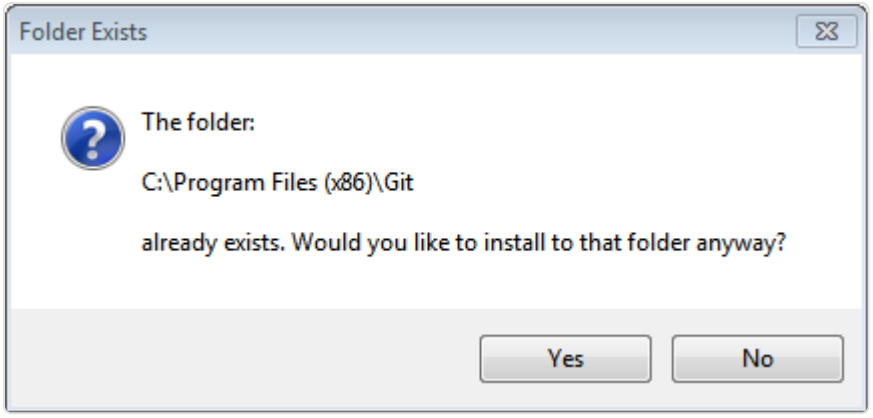
Read GNU General Public License. Click Next.



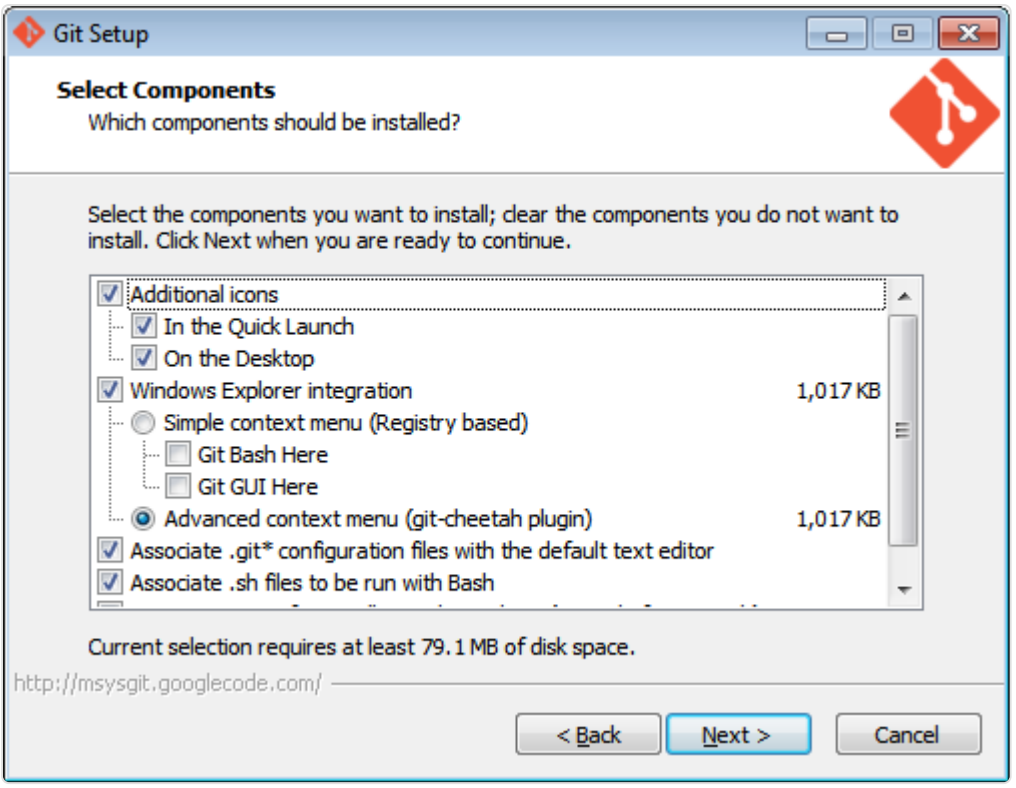
Select destination location. This is the directory where you want to install git. Normally, default directory can be kept as it is. If you wish, you can change the directory by clicking on Browse button. Here, we will install git in the default directory. (C:\Program Files (x86)\Git). Click Next.



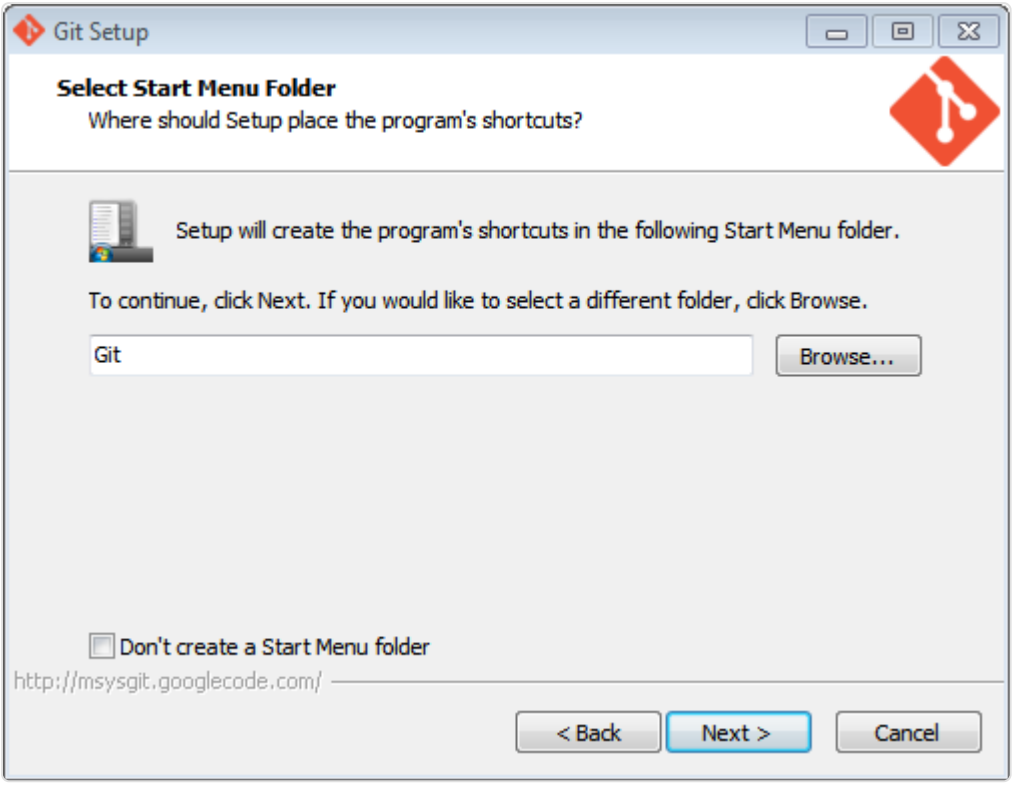
If following dialog box appears, click Yes.



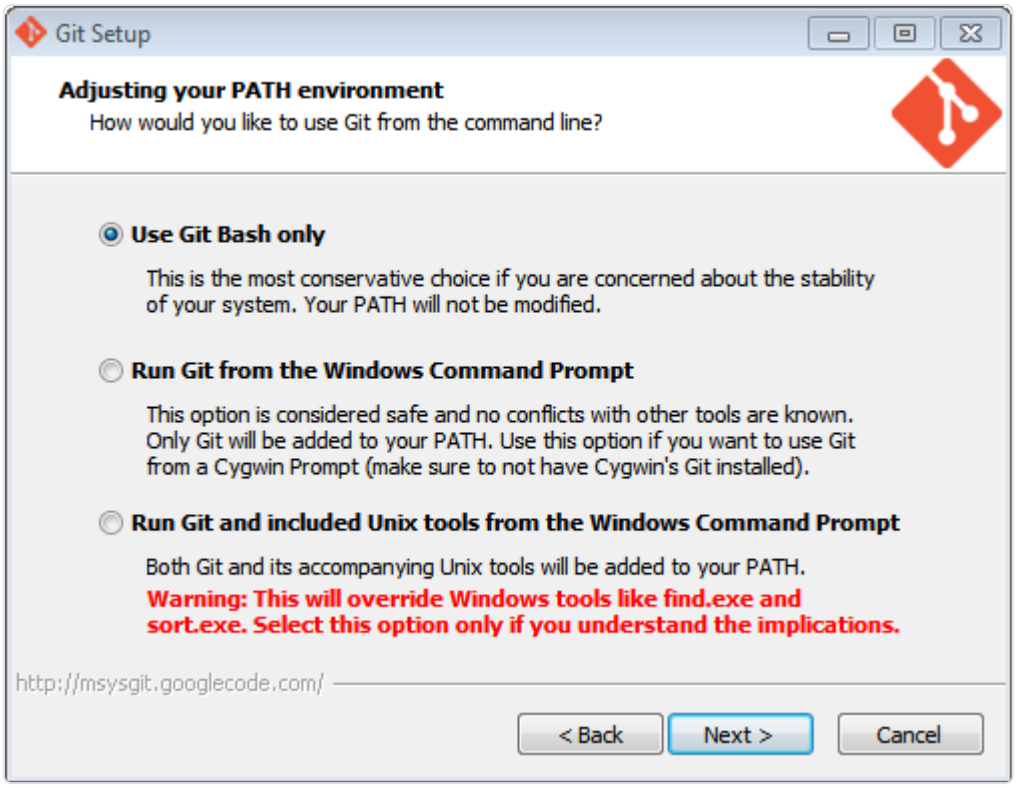
Following screen appears. Leave all the default options as is and click Next.



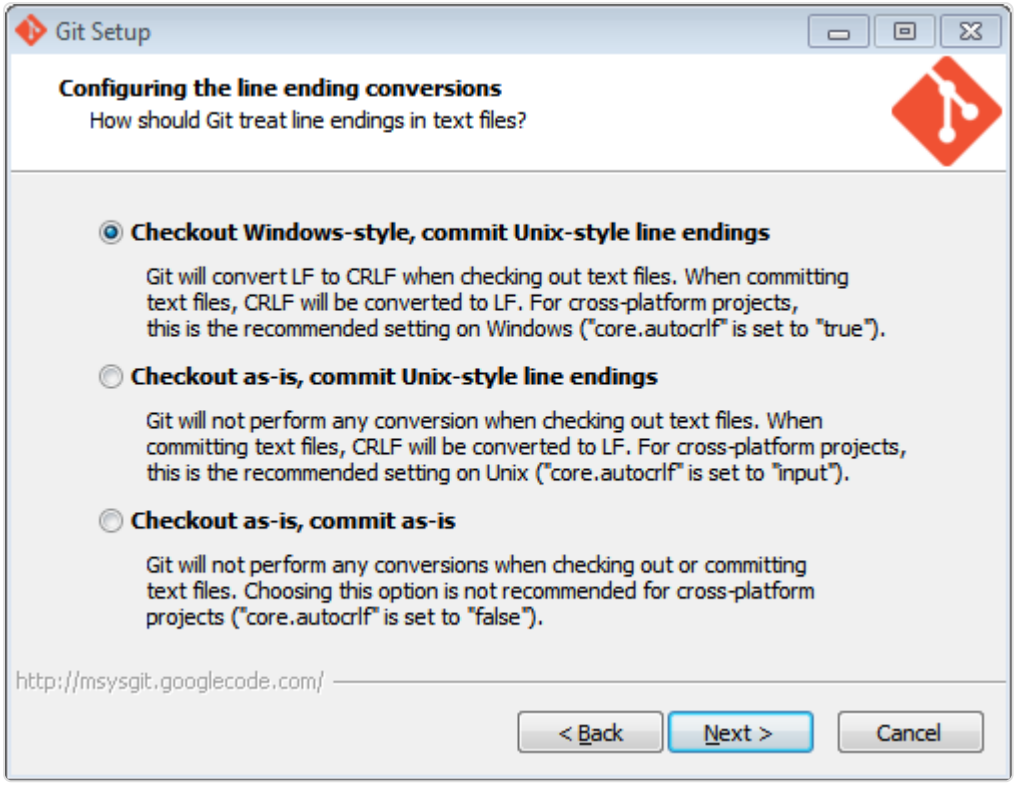
Following screen appears. Click Next.



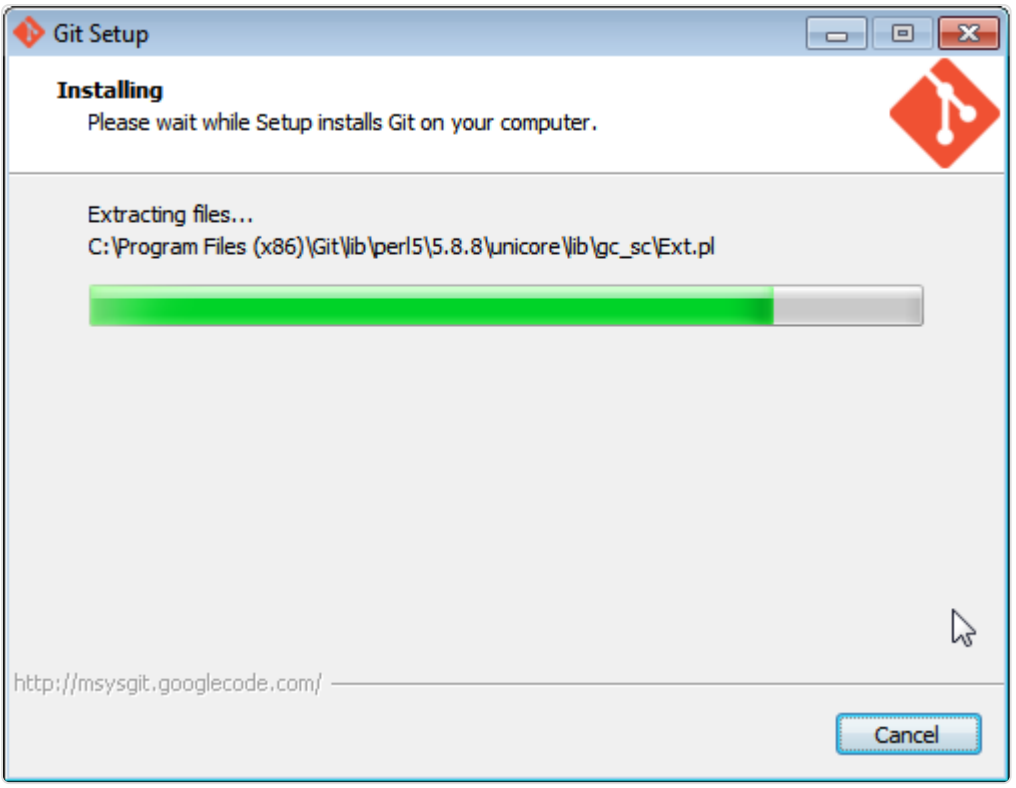
Following screen appears. Click Next to continue.



Following screen appears. Click Next to continue.



Setup will progress.



Click Finish to complete the setup.



Installation of TortoiseGit

TortoiseGit is a GUI wrapper around git, and provides one of the cleaner user experiences. You'll need to download and install msysgit before installing TortoiseGit.

The first screen you'll be presented with is a splash screen.

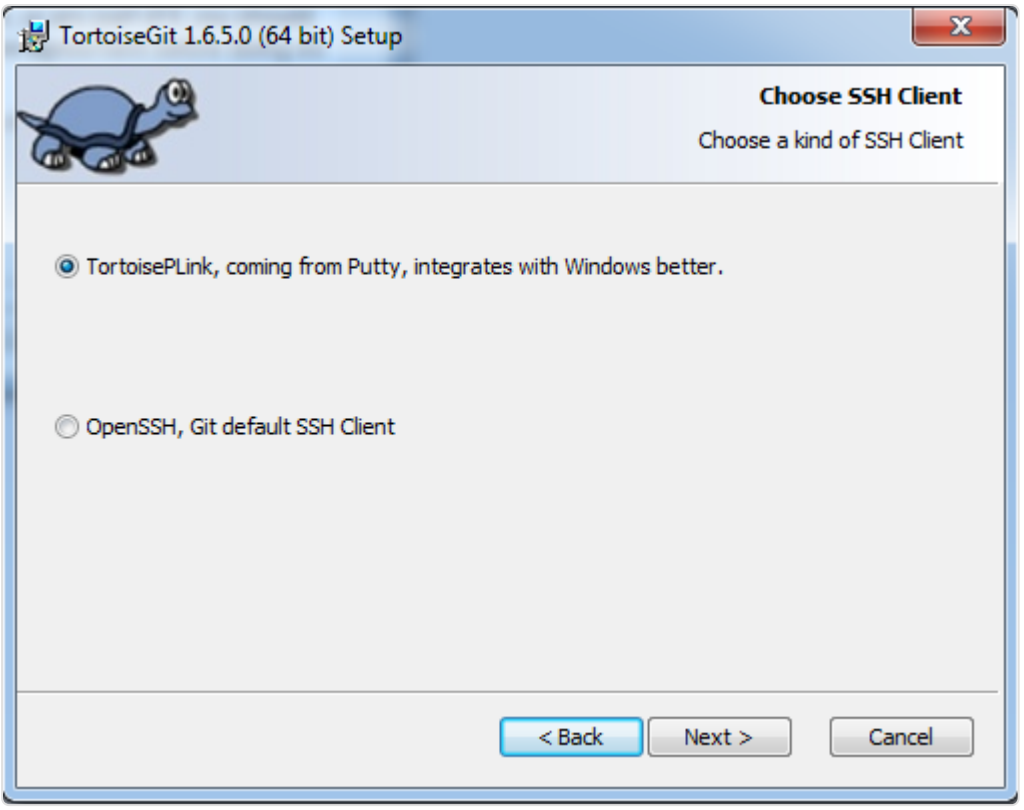


Click Next, and EULA will pop up.



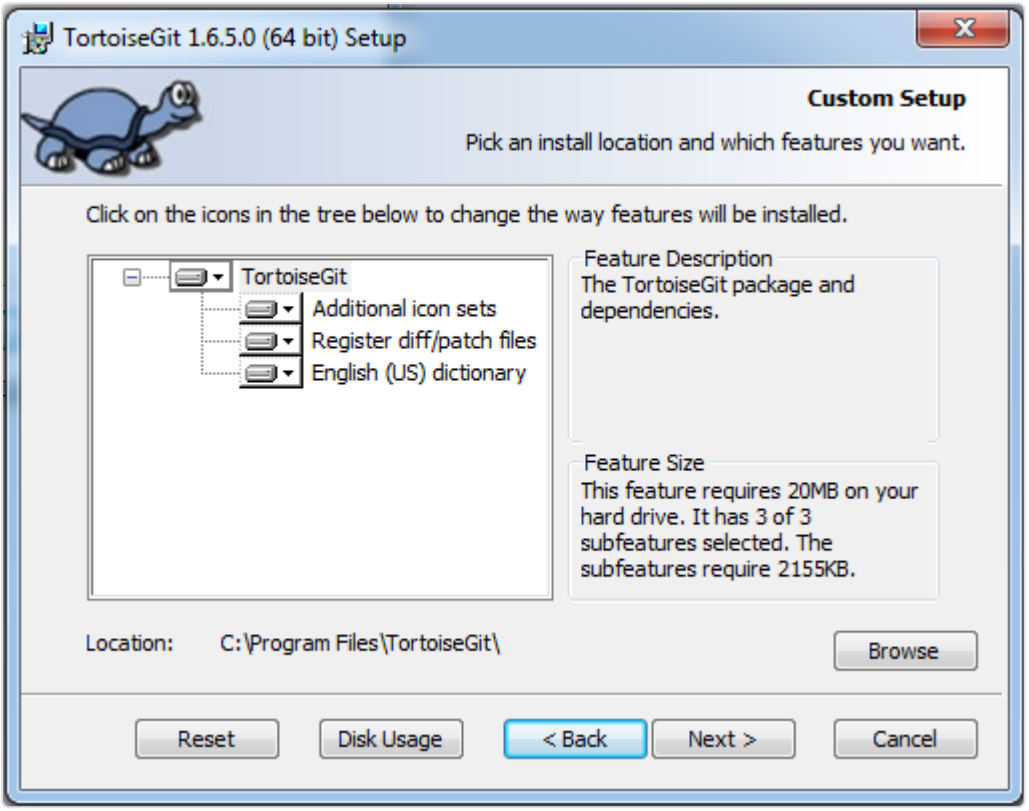
Select 'I accept...' and click Next.

Following screen appears.



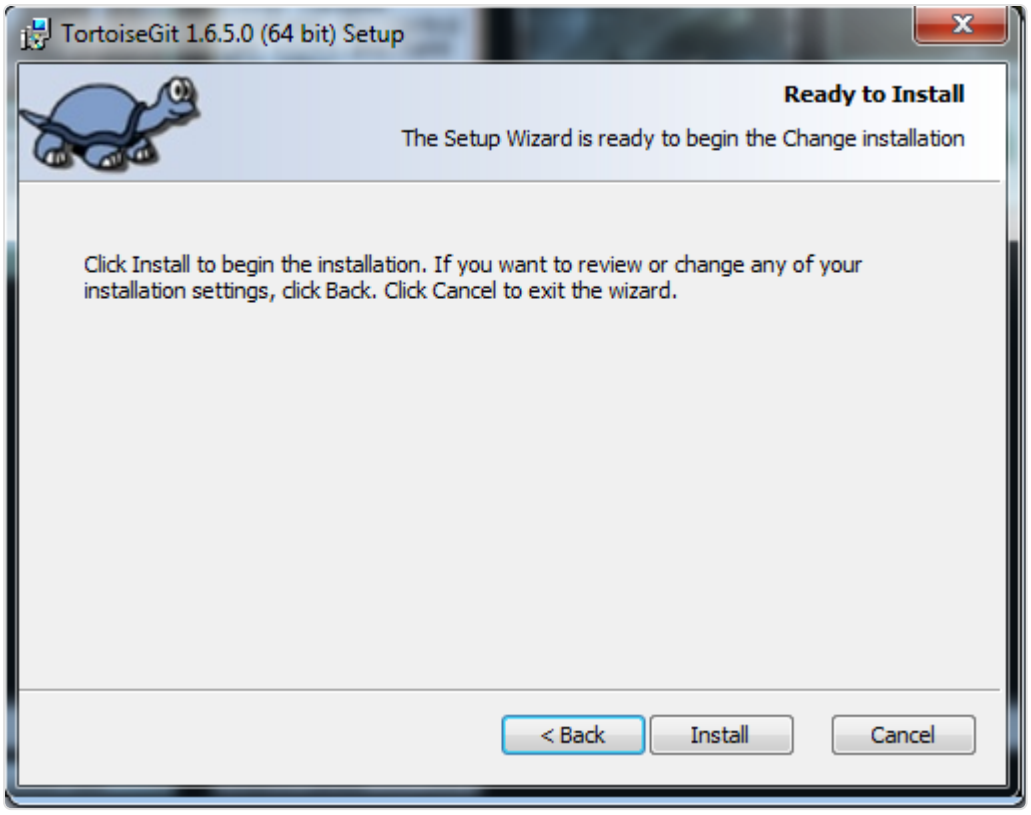
Make sure TortoisePLink is selected, and then press Next.

Screen for custom setup appears.

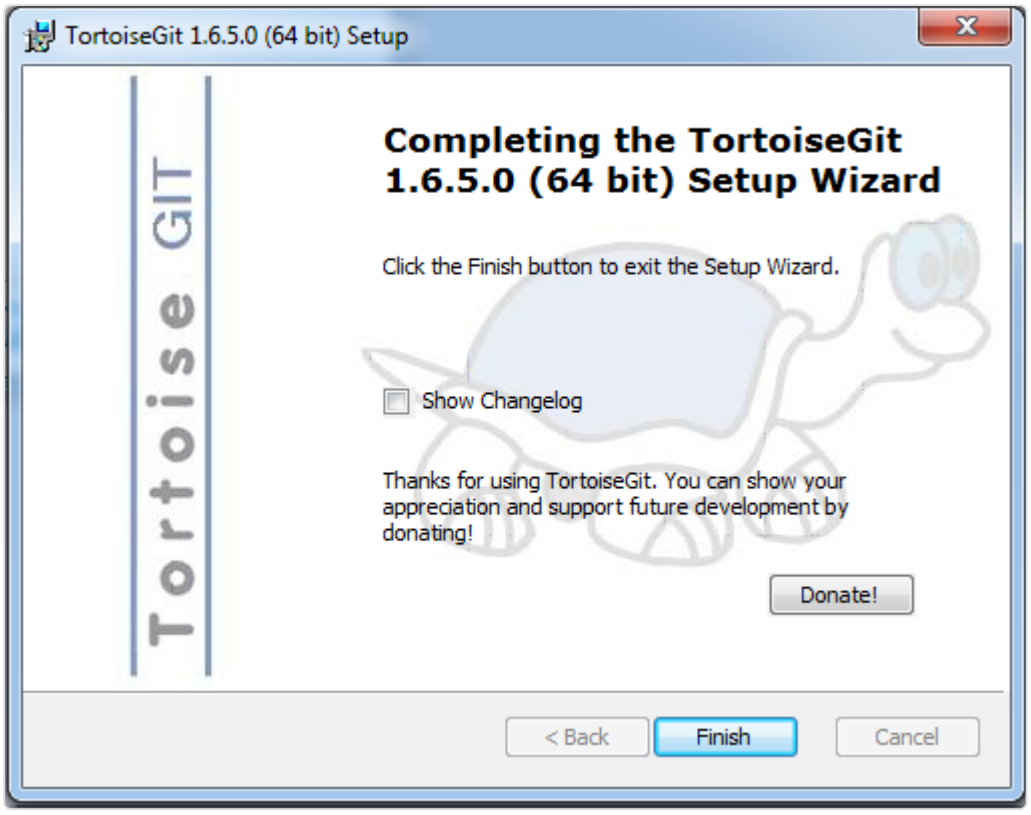


Click Next.

Following screen appears. Click Install button.



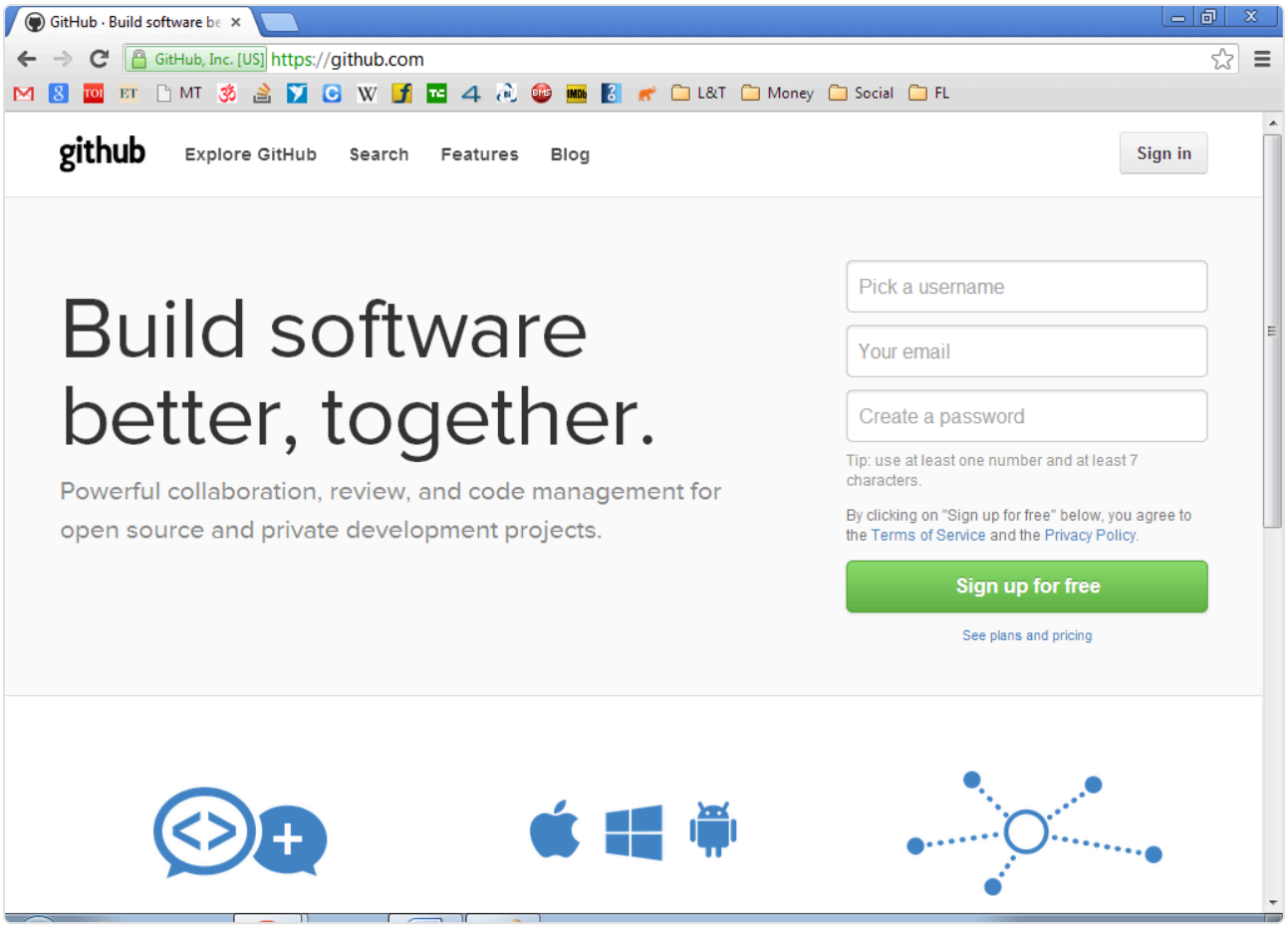
TortoiseGit gets installed and following screen appears. Click Finish.



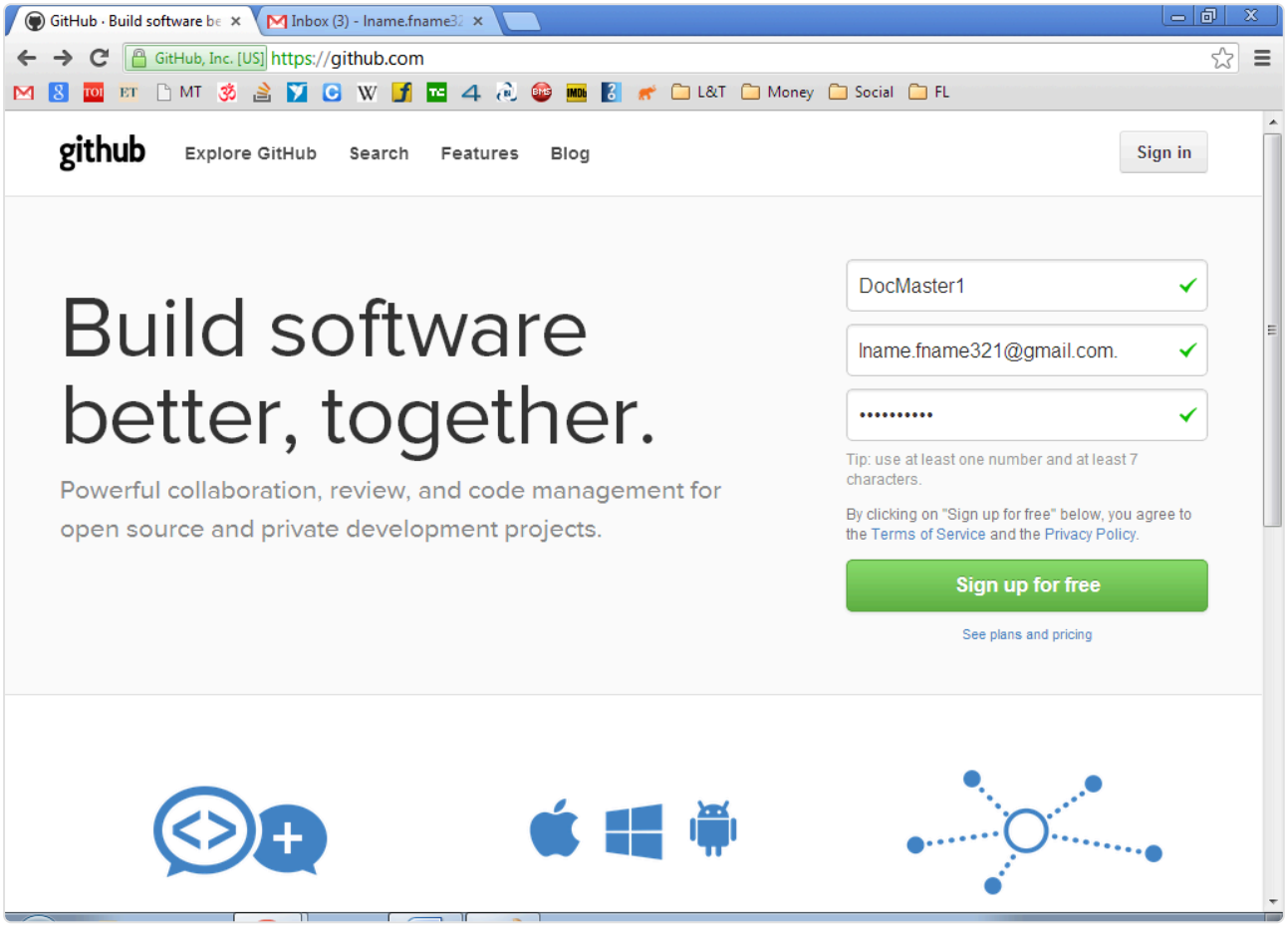
Your machine is now all set-up for playing with code on Github. You just need a Github account.

Sign up for github account

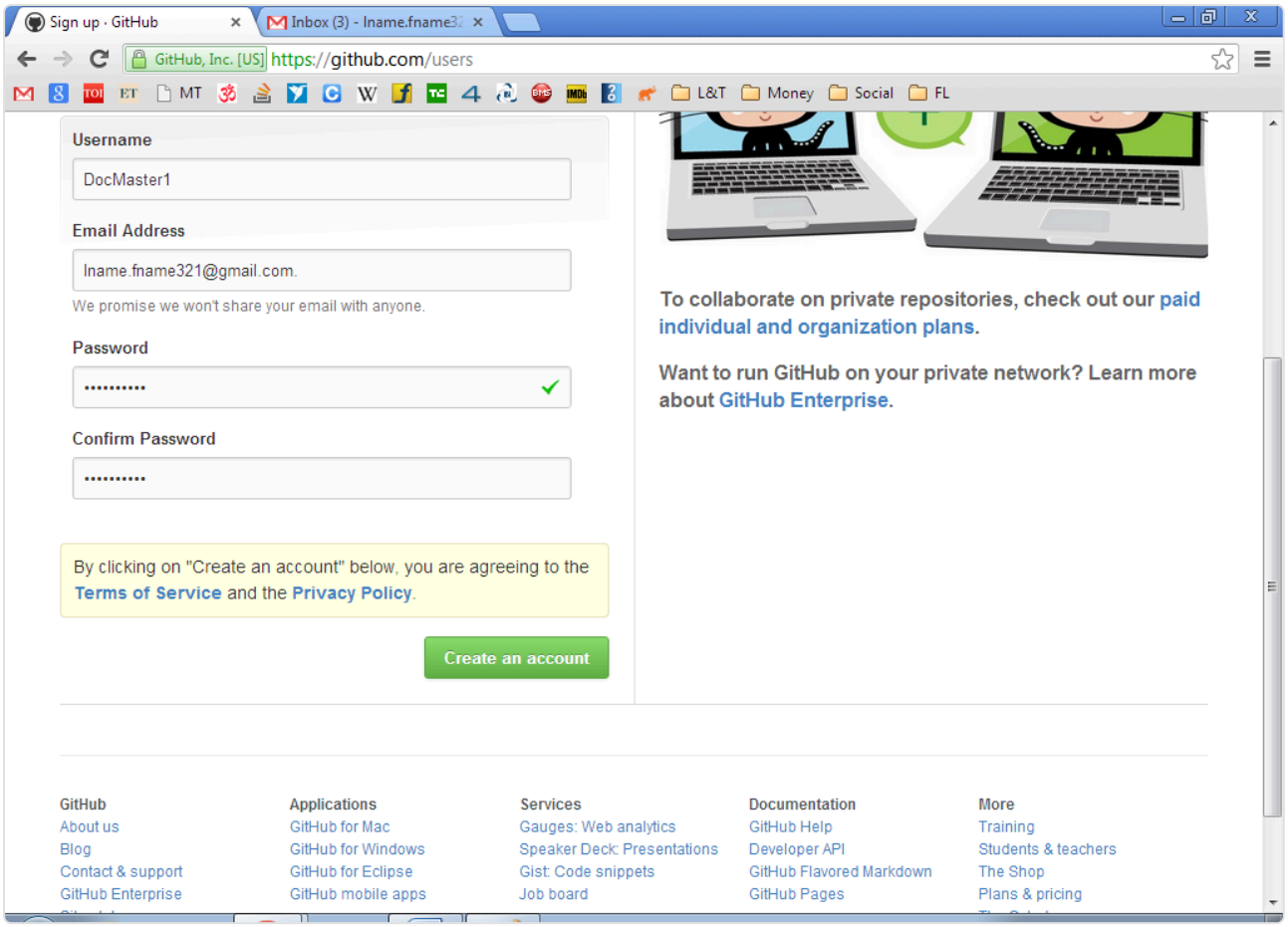
Go to github.com website.



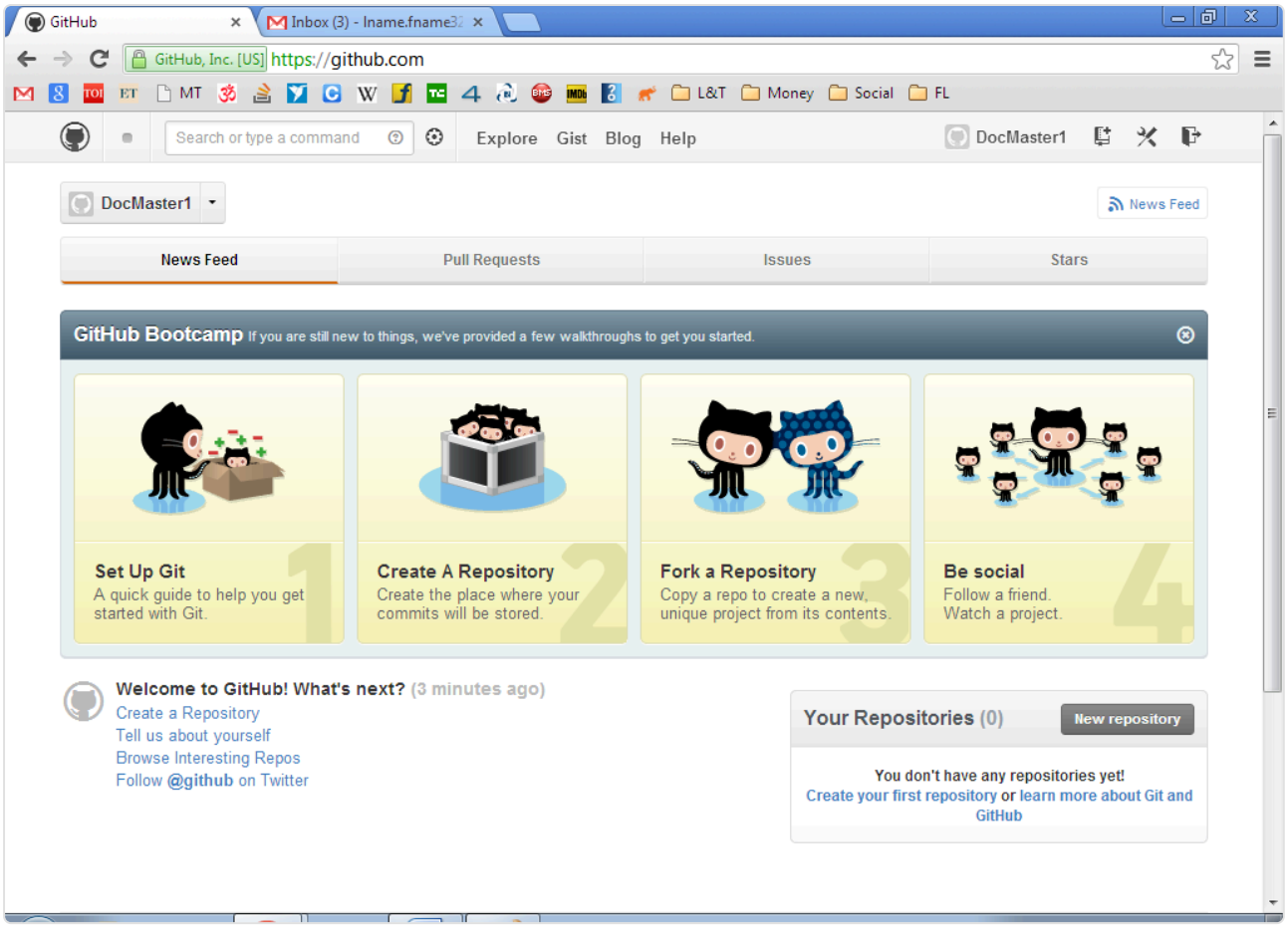
To setup github account, enter username, email id and password. Click 'Sign up for free'.



If there are any warnings, github shows them and takes you to the next page. Click on 'Create an account' button.



Now your github account is ready and you are good to go.



Setting up SSH key

To use your Github account on a Windows machine, you'll need to generate a new RSA key. You'll also need the following tools installed:

msysgit

TortoiseGit

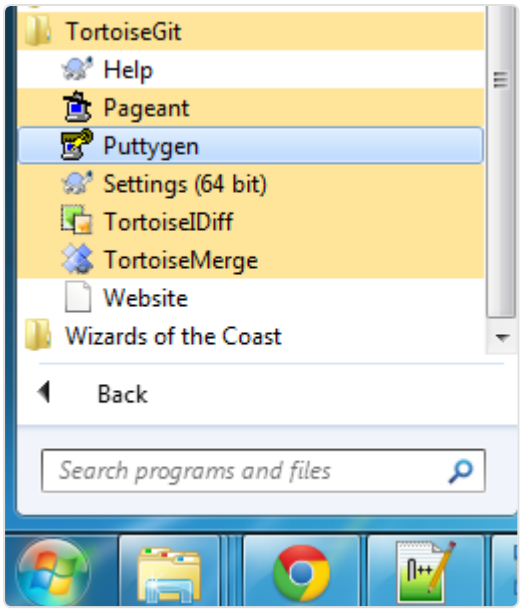
Generate a new RSA key

RSA is an asymmetric encryption algorithm used to authenticate users without using passwords. It's used instead of passwords simply because it provides a mechanism for verifying who you are without giving the other end of the connection information they can use to impersonate you.

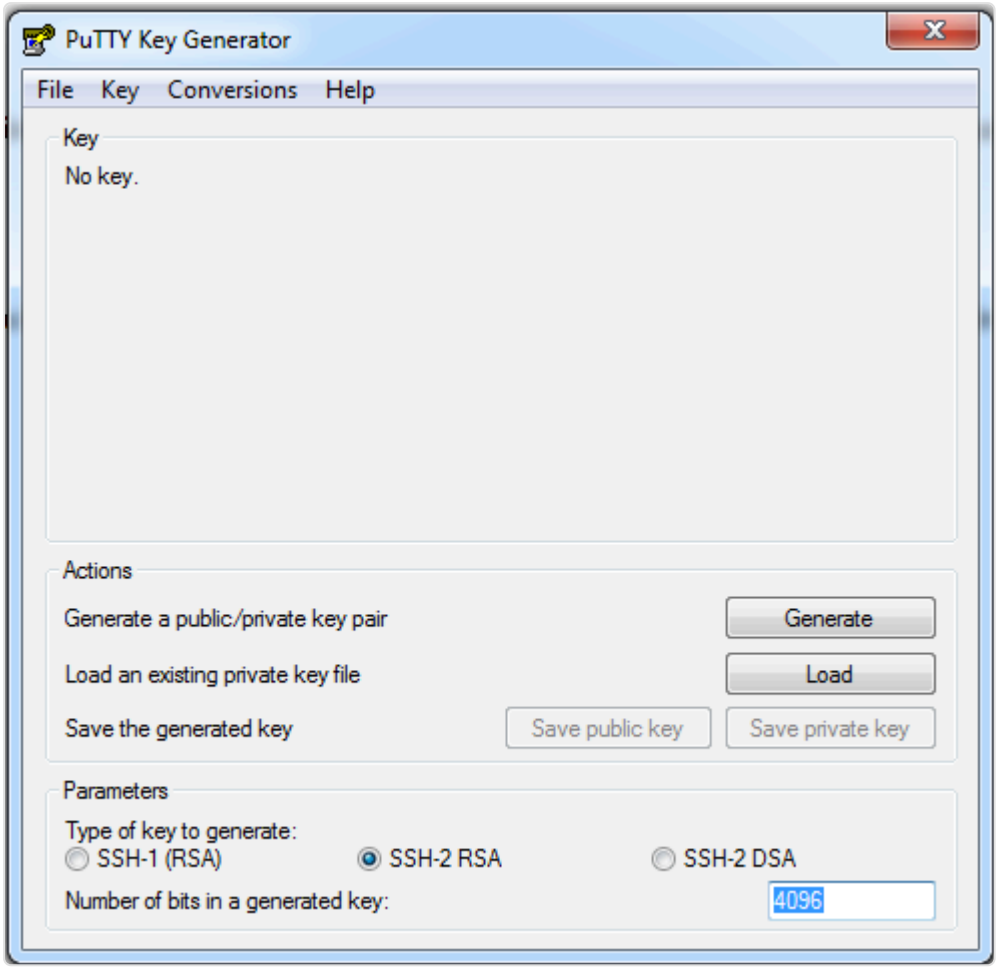
An RSA key consists of two halves: a public key and private key. You give your public key to other sites, which allows them to verify (using a lot of math) that you are the owner of your private key. You must keep your private key private.

Since a private key can theoretically be stolen, it's encrypted with a password (in RSA terms, a "passphrase"). So you need to make sure you both keep the private key in a safe place and remember the passphrase it's encrypted with.

To generate a new RSA key, you'll need to use a tool called Puttygen. It's installed along with TortoiseGit, and you may be able to find it in your start menu:

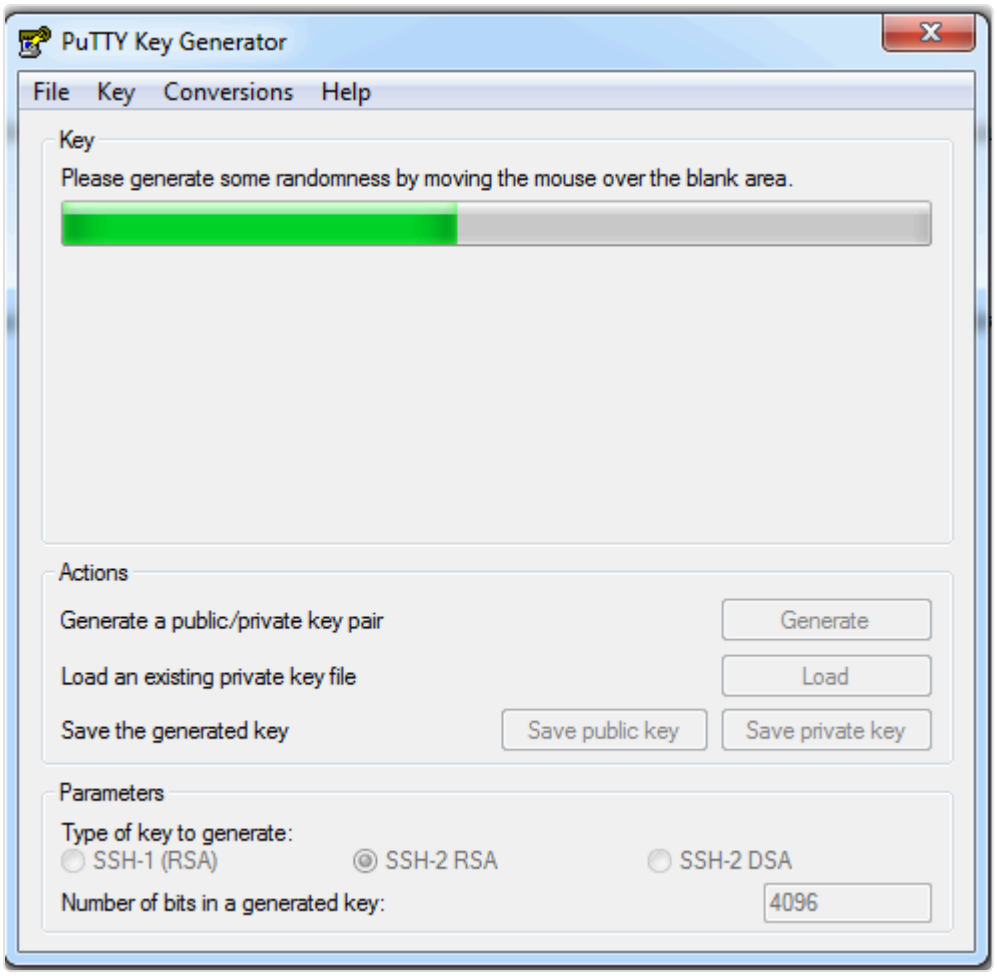


Once it is running, you will be presented with following screen:



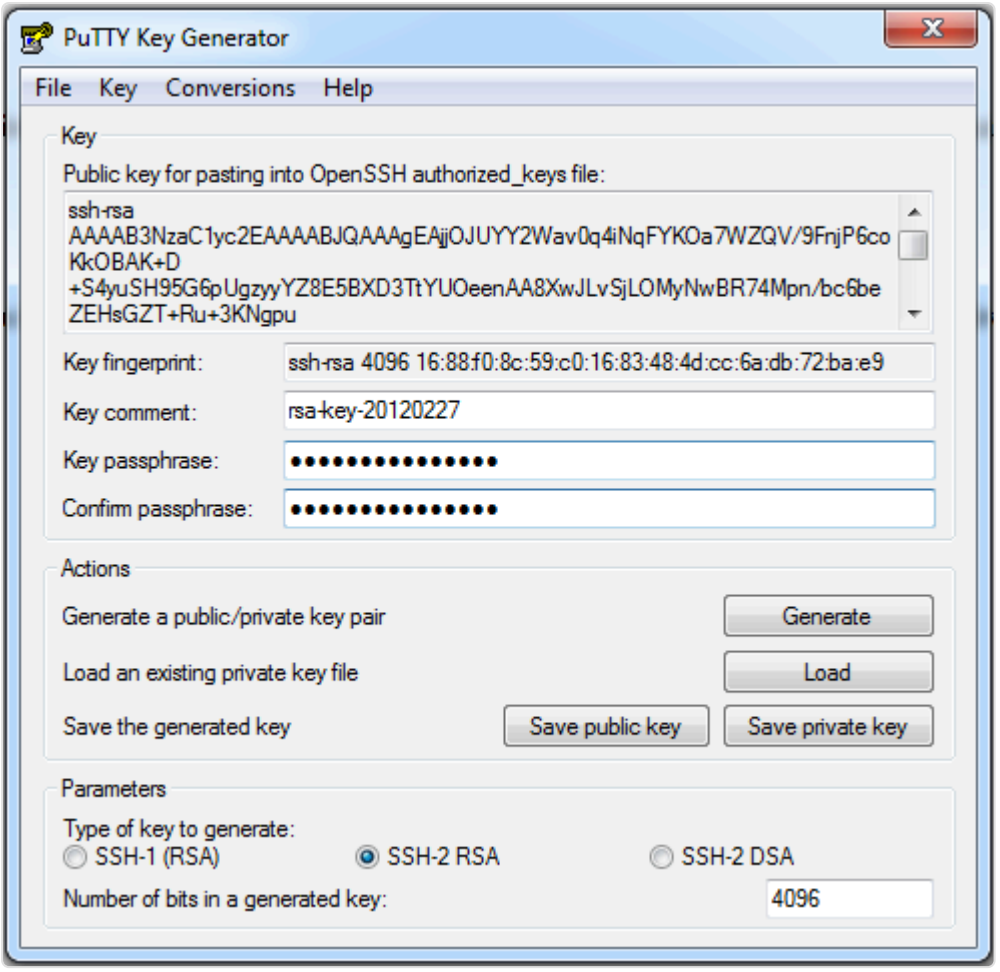
Change the number in the lower-right hand corner, labeled with Number of bits in a generated key, to 4096. This is the current acceptable length for RSA keys. Once this is done, hit the Generate button.

Following screen appears. As indicated in the screen, you are supposed to move mouse over blank area of the screen till progress bar reaches its 100 percent value.



The Puttygen program does not use Window's built-in cryptographically secure random number source. So, this is the fallback option to obtain entropy.

Following screen appears.



Do not close this window until the end.

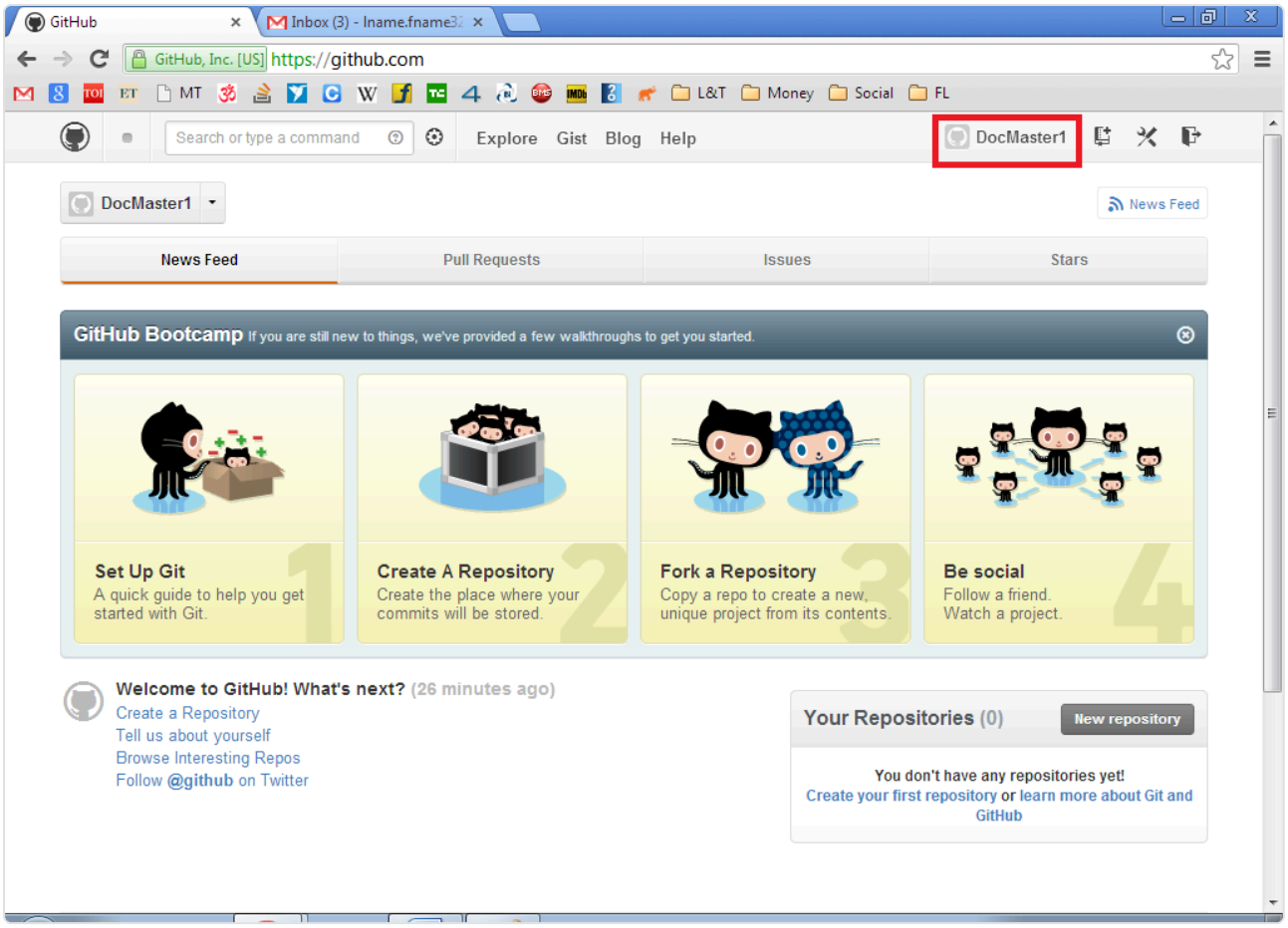
Enter your chosen passphrase into both the ‘Key passphrase’ and ‘Confirm passphrase’ box.

Then, click Save private key and save it somewhere safe.

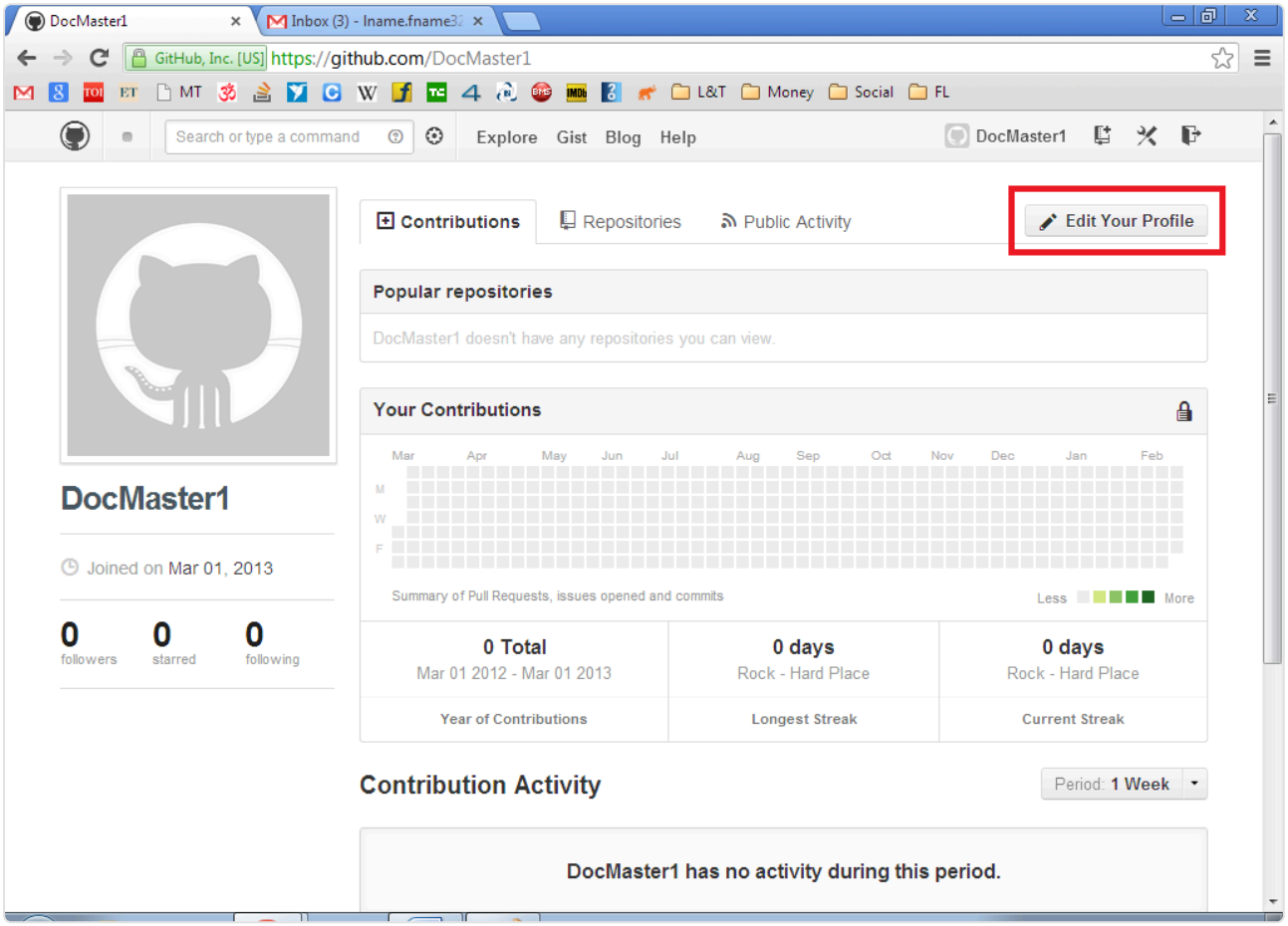
Don't close the window yet. You'll need all of the text in the Public key for pasting... box for the next step.

Give your public key to github

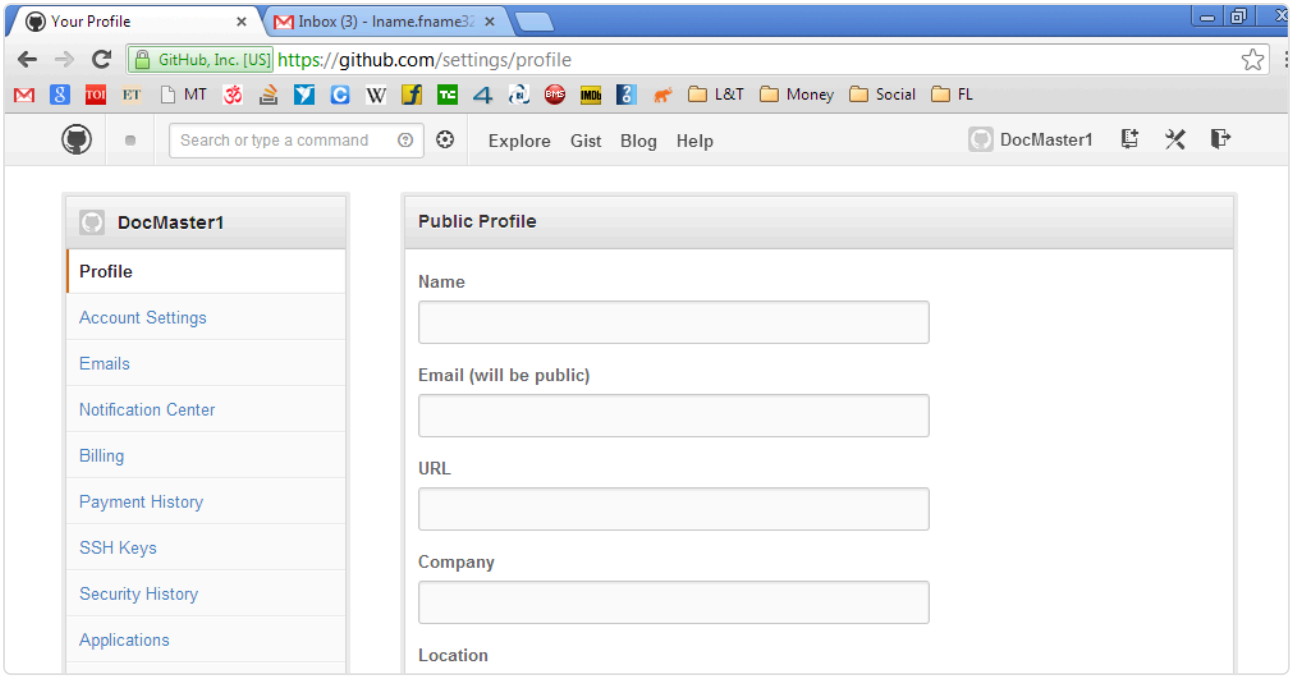
Open web browser. Log into your github account. In the top right corner click your username.



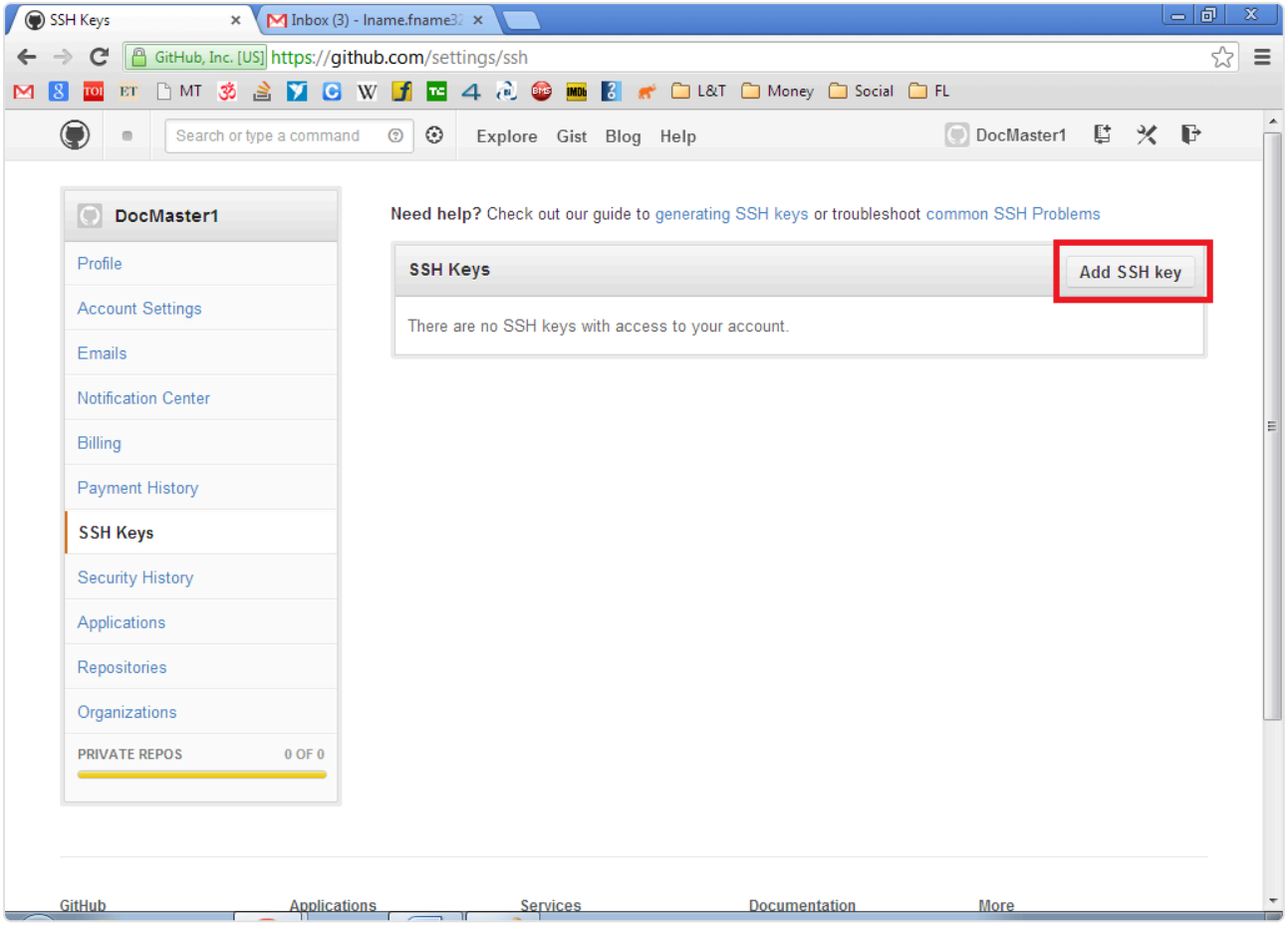
From the screen that appears, click on 'Edit your profile'.



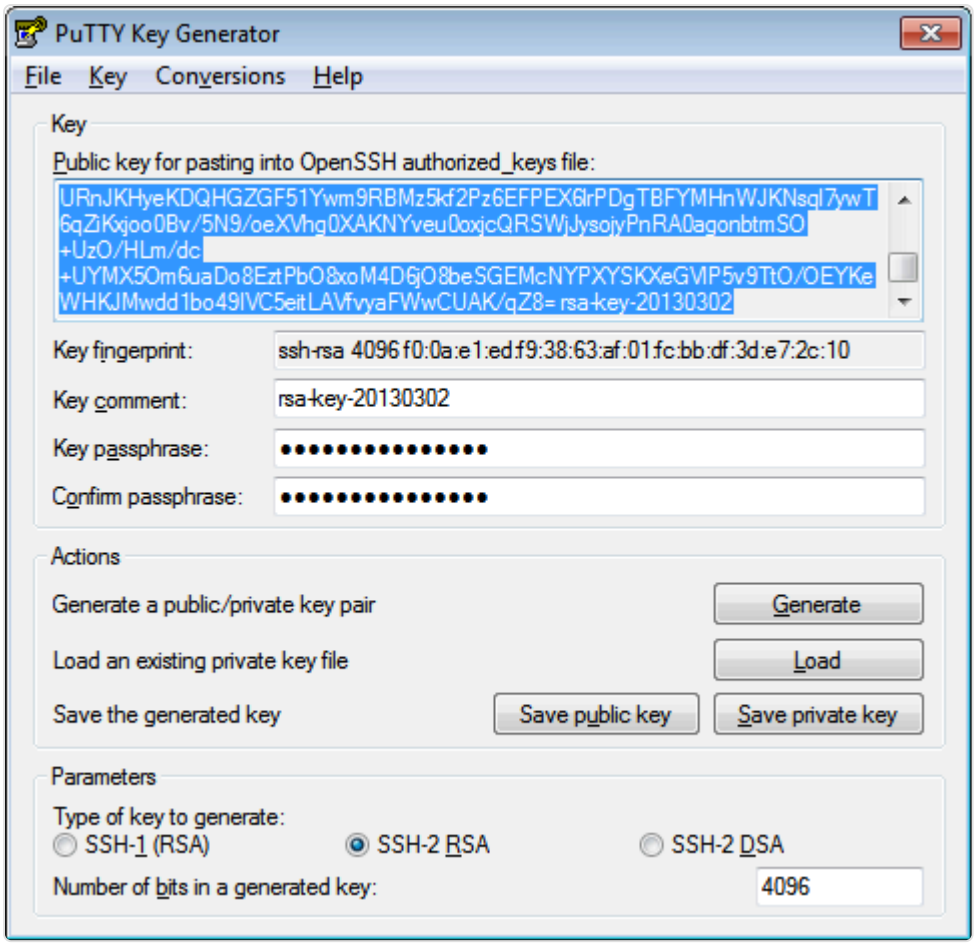
Then from the menu on left, click 'SSH Keys'.



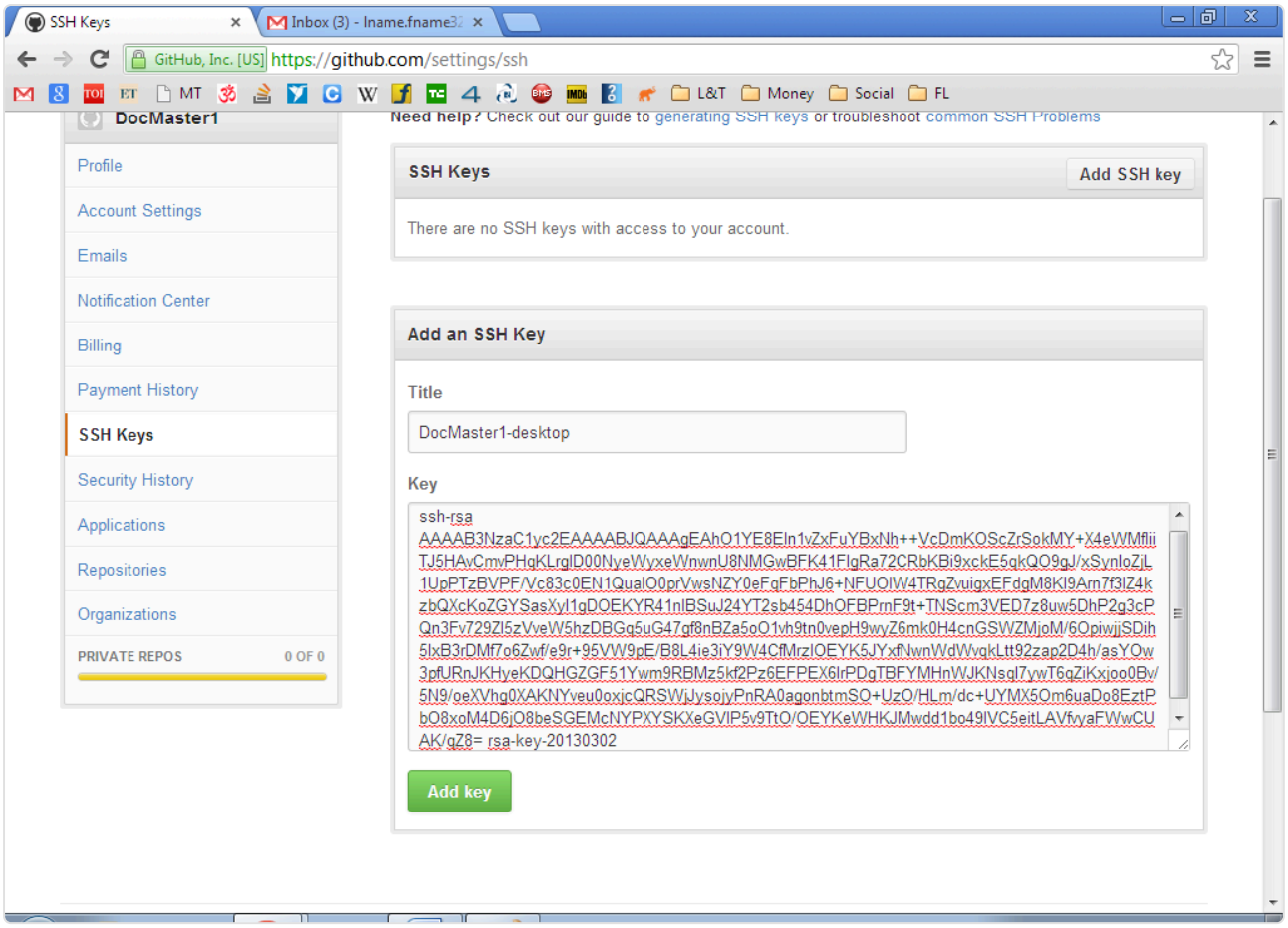
Click on the button 'Add SSH Key'.



Go back to Puttygen window and copy text as shown below. Right click -> Select All would be a good option.



Paste copied text in the textbox labeled Key. Also, enter a Title.



Finally click ‘Add Key’.

That’s it. Make sure to keep your private key somewhere safe. And do not forget your passphrase.

Github Workflow

Before starting to learn the workflow, make sure you have following things:

Installed msysgit, the windows version of Git (section 6.1)

Installed TortoiseGit, a friendly GUI wrapper around msysgit (section 6.2)

A github account setup with public half of your private key (section 6.3 and 6.4)

Private key and passphrase (section 6.4)

A word about git and github:

Git is a decentralized version control system. The "decentralized" bit sets it apart from other version control systems like CVS, SVNor Perforce. "Decentralized" means that there is no central repository -- each developer has a repository (or a few copies) that contains the entire history of everything. This has some advantages:

You can make commits without being network-connected

You can commit without worrying about build-breakage

In a disaster scenario, you're fine unless every copy of the repo is wiped out

Unfortunately, these come at the cost of additional complexity.

To simplify things a bit, in most real-world scenarios, one repository is designated as blessed [In our case, this blessed repo would be: temcocontrols / T3000_Building_Automation_System]. This blessed repository serves as a central, true copy. All the other repositories (or, "clones") still contain the same version history; the blessed repository simply serves as synchronization primitive to make the workflow cleaner.

You can do a variety of things with a repository. These include --

Add new commits to a local repository (on your machine)

Send commits to another repository (on your machine or the network)

Get commits from another repository (on your machine or the network)

In the setup we're using, each developer interacts with three repositories:

The repository on your local machine

Your "fork" on Github

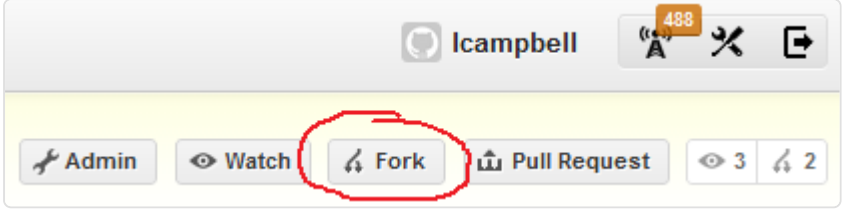
The "one true repo", which is the central, blessed repository

You're free to do whatever you want with the repository on your machine and your fork on Github. Typically, you'll make commits on your local repository and send them to your Github fork immediately (or, as soon as you're network-connected). Commits going into the blessed repository must go through a review process (using Github's wonderful interface) before they're accepted and merged into the codebase. This process is performed using a pull request on Github, which is detailed further down.

Forking:

Right now, we've only got the blessed repository. We need to create both your fork and your local copy set up. First, log into Github, navigate to our Github page and select a repository you'd like to work on. I'm going to assume for this case you want to work on T3000_Building_Automation_System.

Click the Fork button in the right-hand corner:



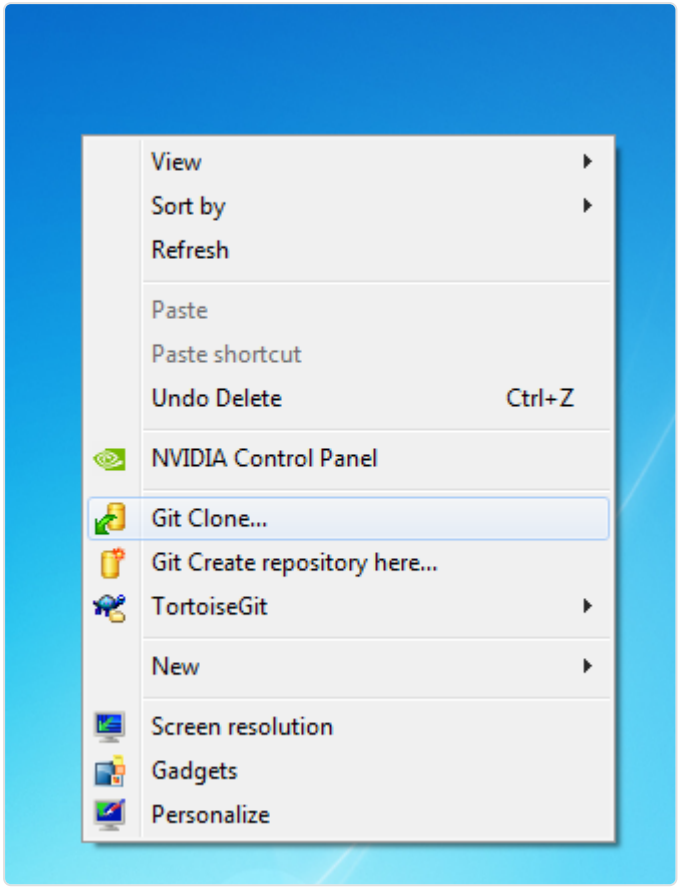
After some hardcore forking action, you'll be greeted with your Github fork. This is your personal copy of the blessed repository.

Assuming your Github username is Jay and the name of the upstream repository is T3000_Building_Automation_System, your fork will be located at https://github.com/Jay/T3000_Building_Automation_System.

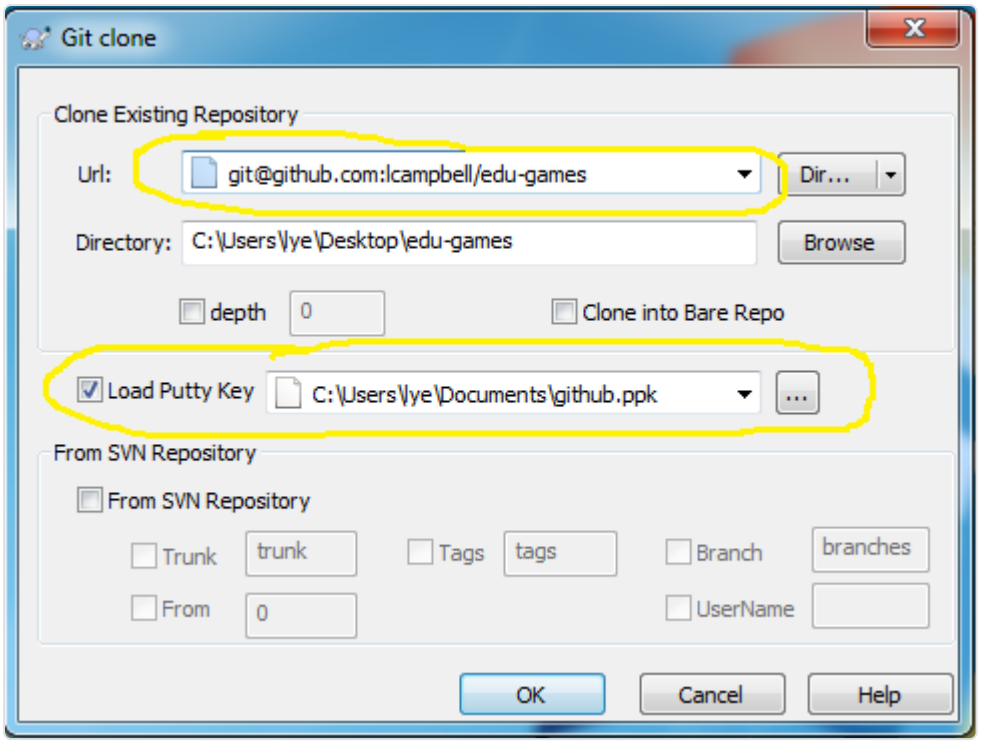
[Remember, these names are just for example. In reality, username Jay has already been taken up by someone else. Here, I am mentioning the same because it would be easier to understand. Also, the usernames and repo names shown in diagrams are different ones. Do not get confused by that. Just have a look at diagram for position of the buttons and conceptual understanding, etc.]

Cloning your fork into a local repository

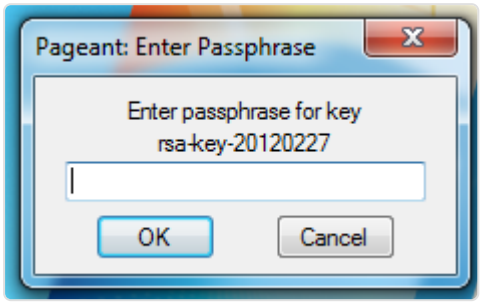
Git doesn't allow you to commit directly to a repository on the network -- you need to make a local copy before you can start doing work. Making a copy, in Git terminology, is called "cloning", and we're going to do it with TortoiseGit. Start by right-clicking on the Desktop (or other appropriate place) to bring up a context menu.



From this menu, select Git Clone... and you'll be greeted with the following dialog:

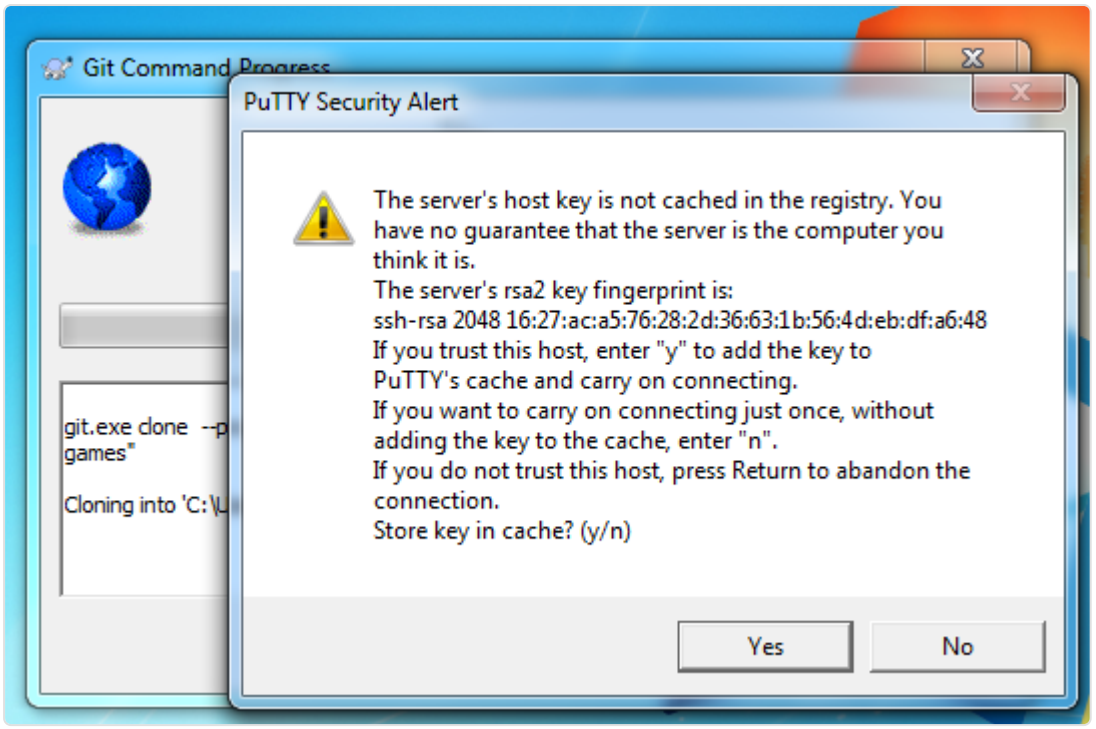


Here, you'll need to enter the URL for your Github fork, and the location of your private key. Do not enter the URL of the blessed, upstream repository. Once you've got the appropriate information inputted, click OK, and it will prompt you for your key's passphrase.



Toss in your passphrase and hit OK. TortoiseGit will save your passphrase for an indeterminate amount of time.

If this is your first time connecting to Github's servers, you'll get the following message:

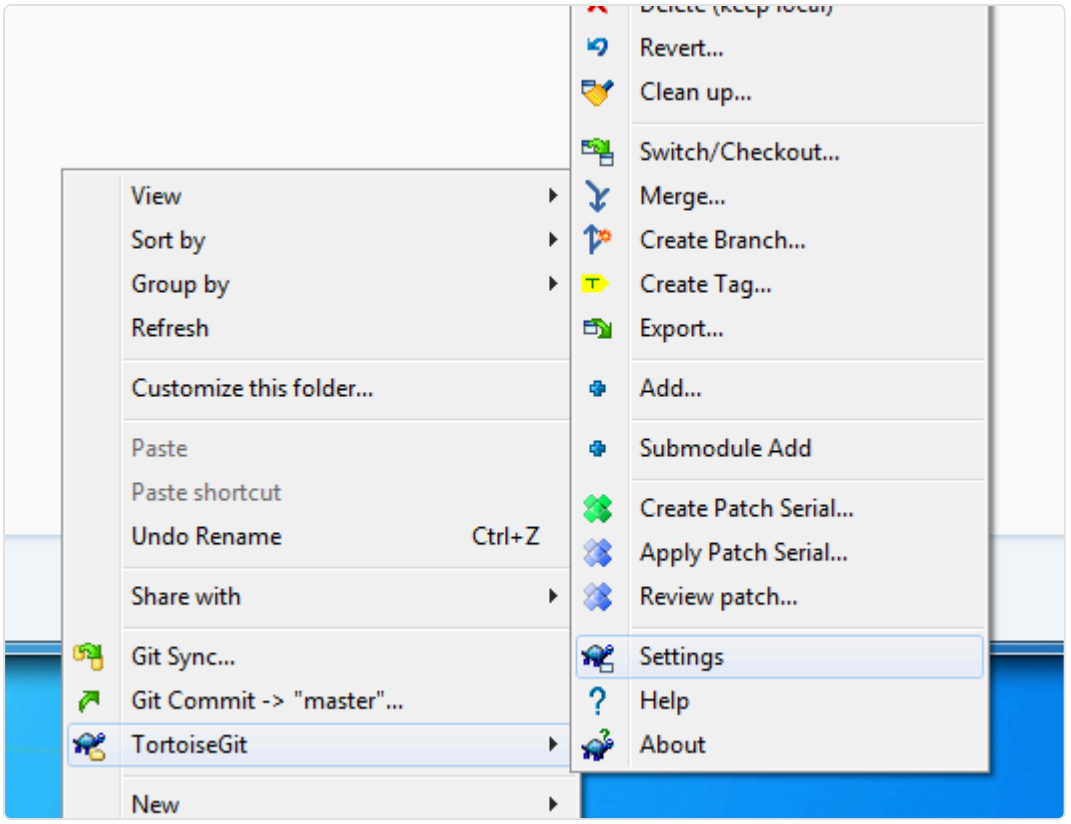


Check to make sure that the rsa2 key fingerprint the dialog tells you is the same as in the screenshot; if it's not, then something is terribly wrong (it's not Github you're communicating with). The reason you get this dialog is due to the nature of public key cryptography. You don't yet have Github's public key. Once you connect once, their public key is saved on your machine and you'll never be prompted again. If their public key changes for some reason, you'll know immediately and Git will cease communication before any damage is done.

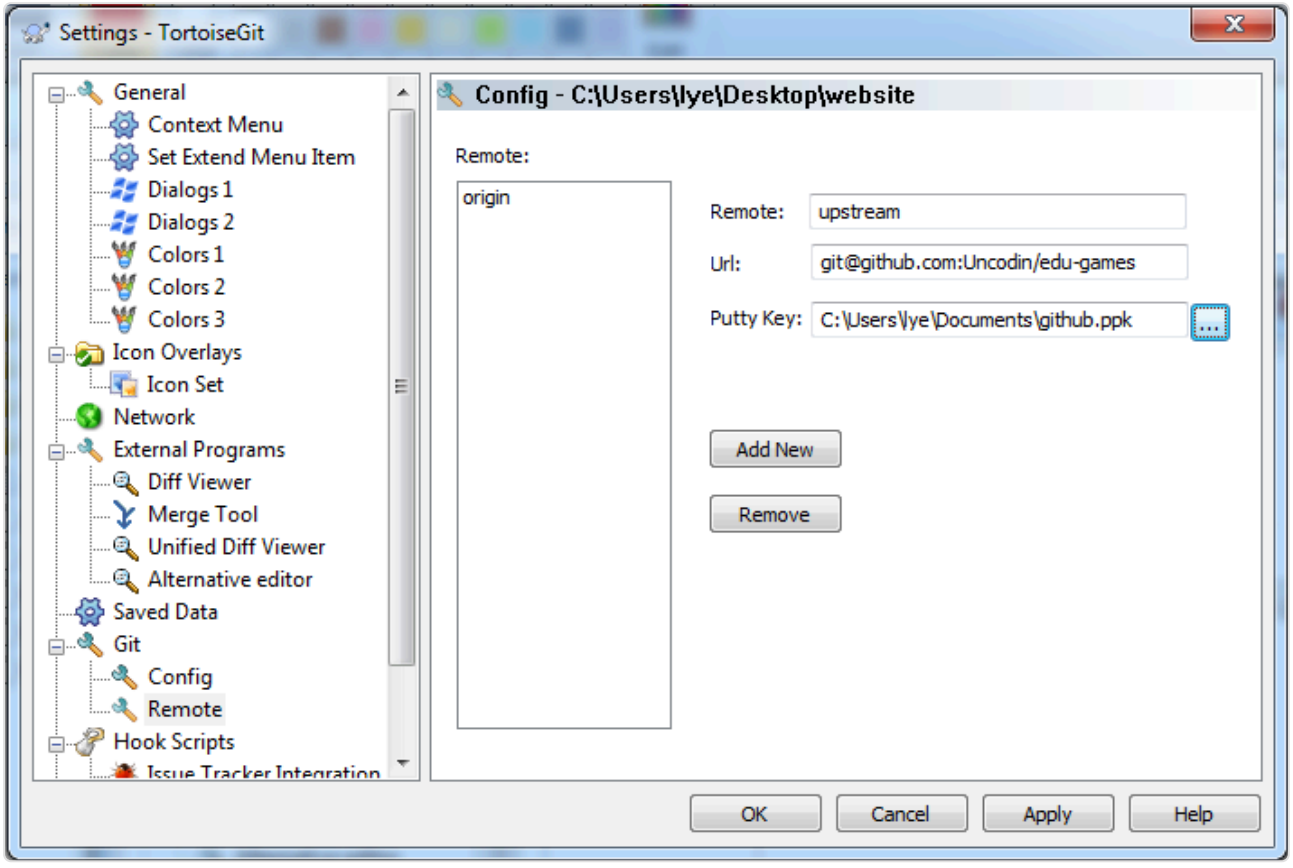
If the key fingerprint is correct, click Yes.

A window should pop up with a progress bar as the repository is copied to your machine (forked repo). Since it contains the entire history it can take a bit of time on a slow network connection.

Once it's finished downloading, we need to take one last step -- we need to tell your local repository about the blessed upstream repository (temcocontrols/T3000_Building_Automation_System). To start, open up the TortoiseGit settings by right-clicking in your repository and selecting Settings



Select Remote from the left menu. You'll get this dialog, albiet without the details filled in:



In the fields, put in

Remote: upstream

Url: The URL to the upstream repository:

git@github.com:temcocontrols/ T3000_Building_Automation_System

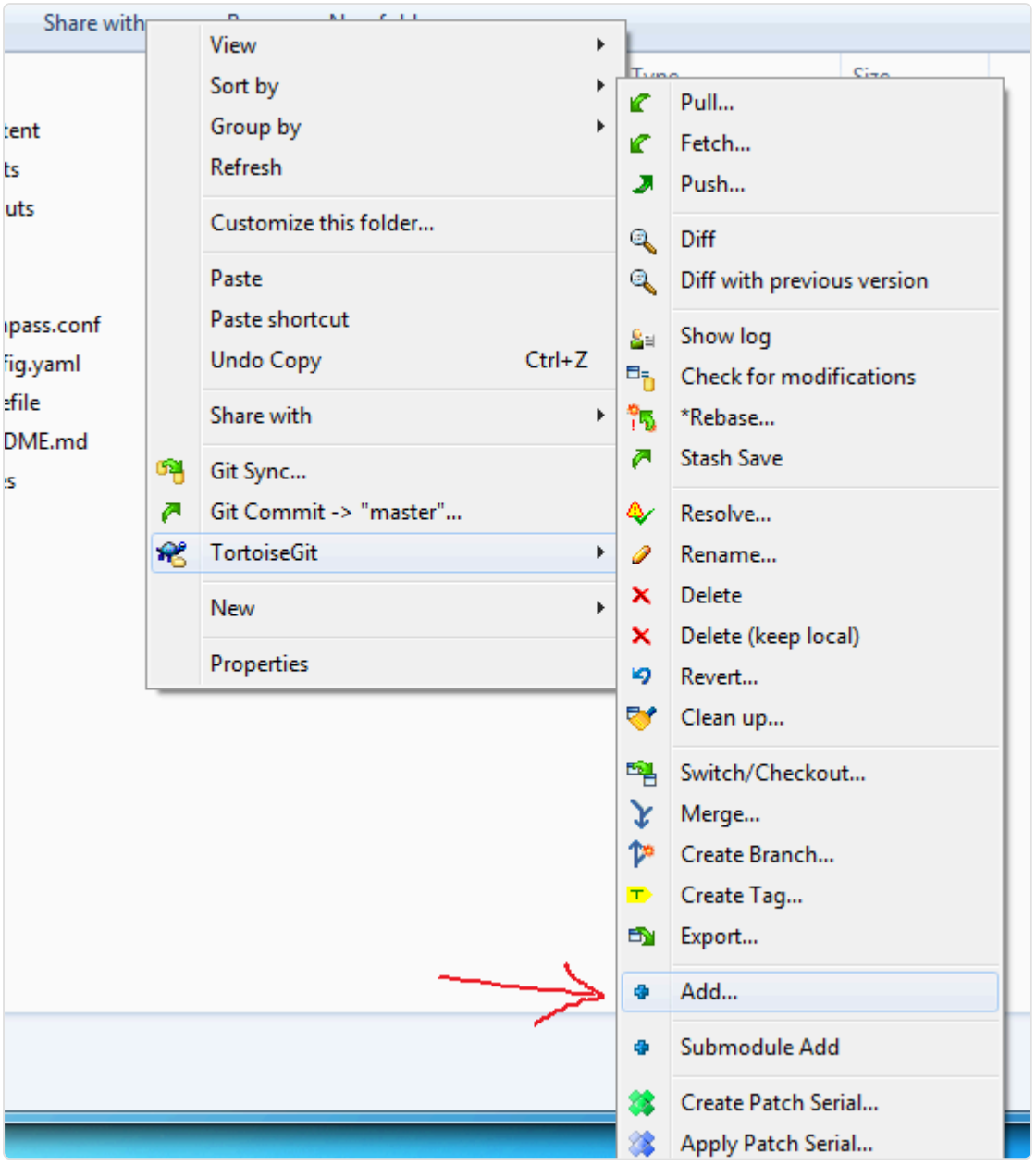
Putty Key: Your private key

Hit Add New, then OK. You're all set!

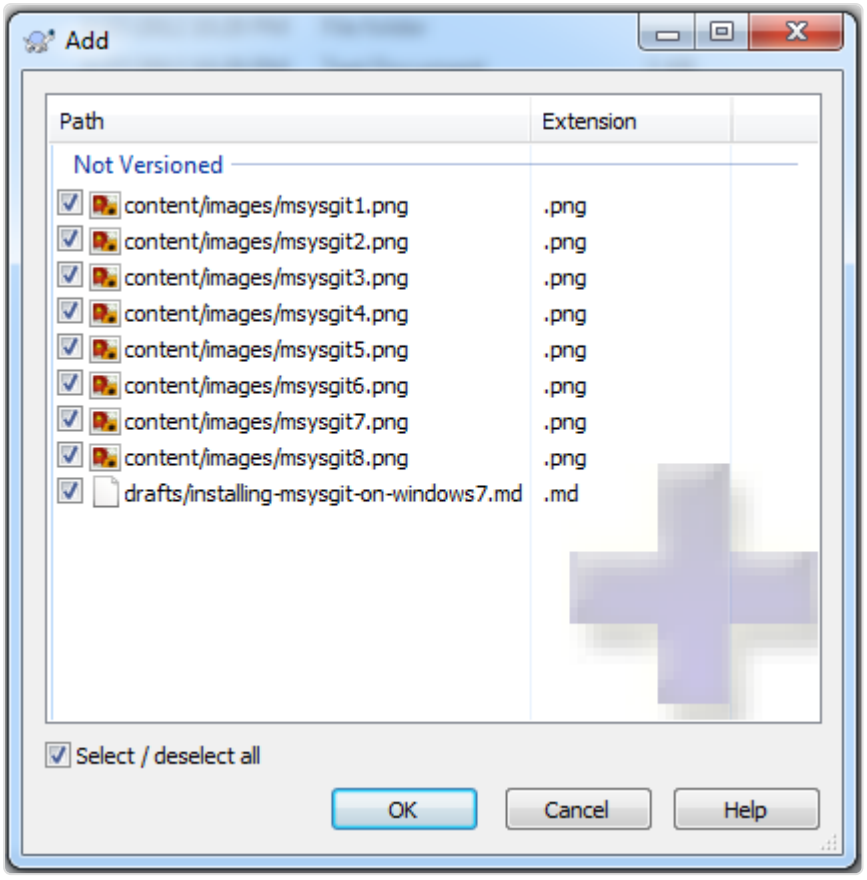
Making changes, adding commits

With your fresh clone, look around, make some changes. Add a file or something. Or get some actual work done.

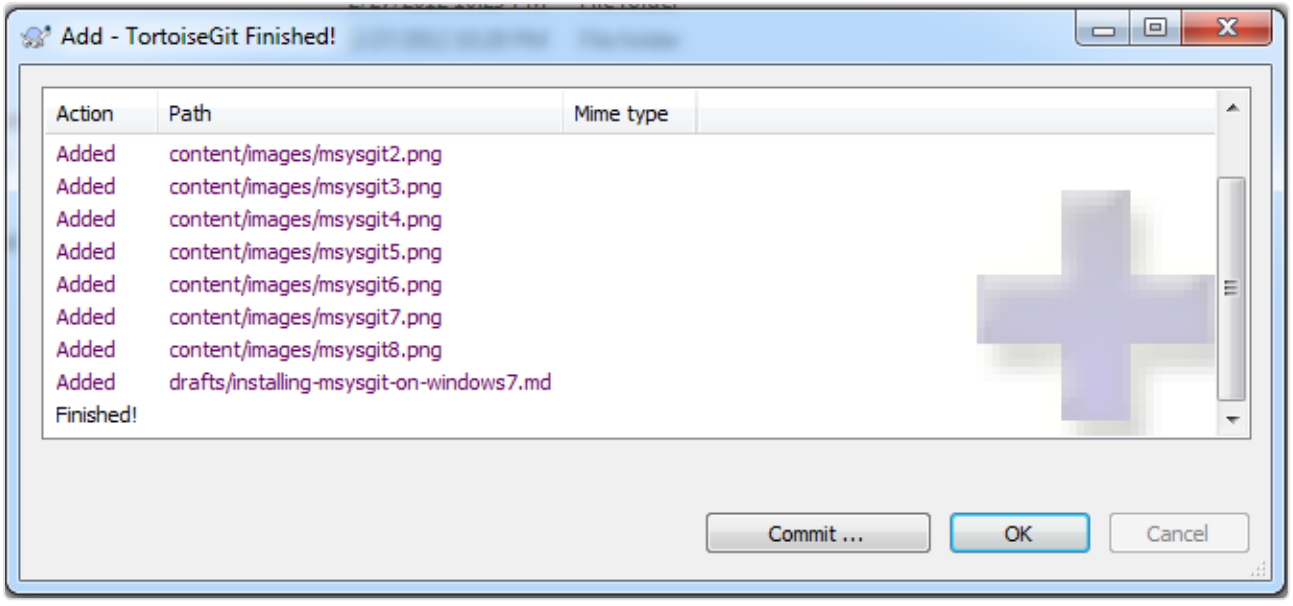
When you've changed something, go to the main folder of your repository and open up the right-click context menu. From that menu, select Add...



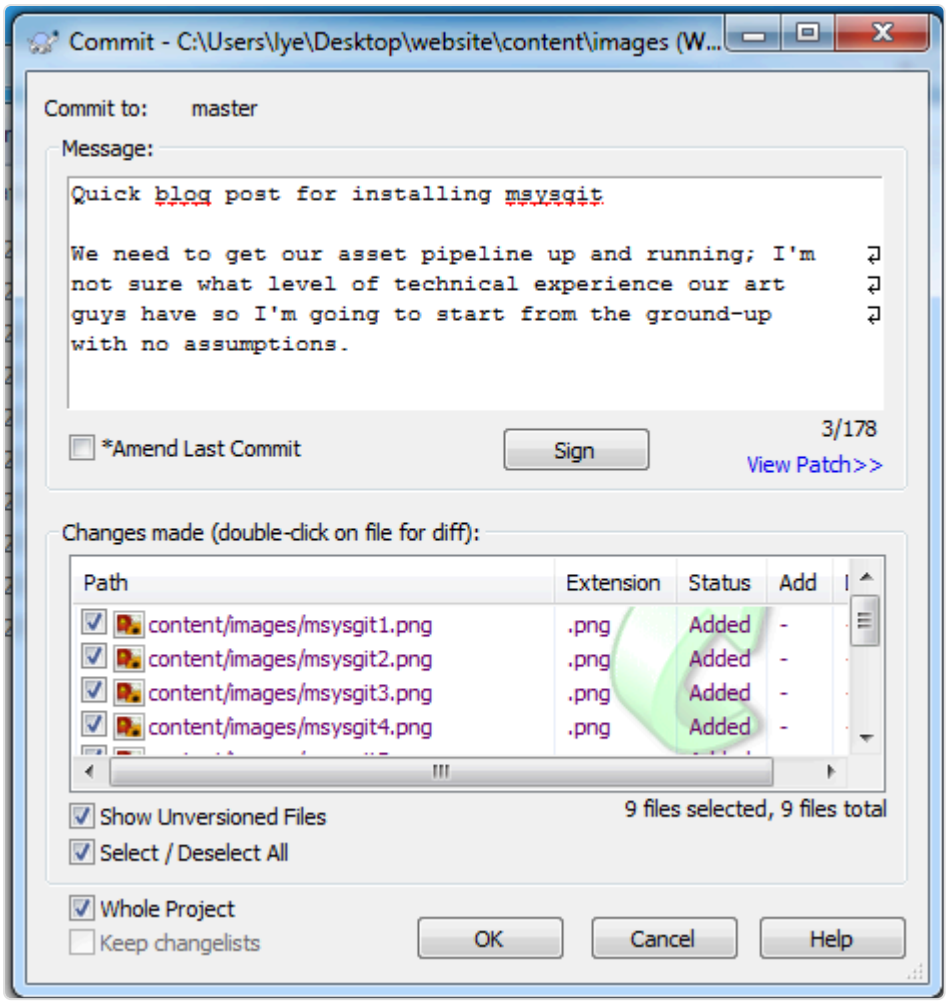
You'll get the dialog telling you which files have been added, removed or changed. Make sure they're all checked and hit OK. Do note that adding files does not commit them. You need to do that in a separate step.



Once you've clicked OK, you'll get this:



To hit the previous point home, this dialog gives you a helpful Commit... button to commit the files you've added/changed/removed. If you don't hit commit, you can select Commit from the right-click context menu at a later point in time. For now though, hit Commit... to arrive at this dialog:

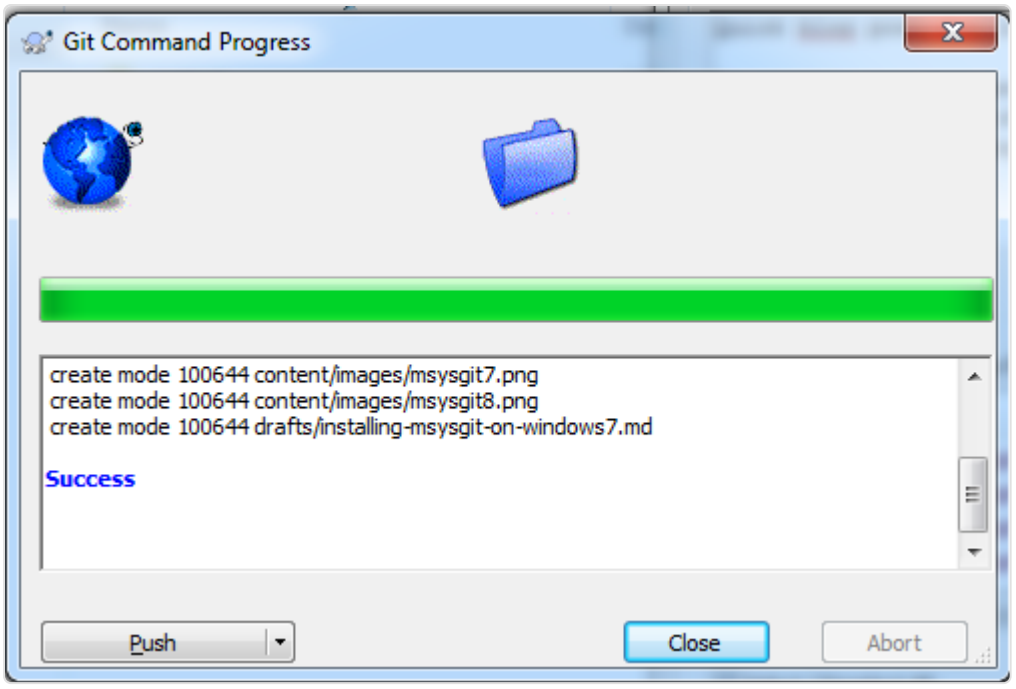


At this point, write a quick blurb about the changes you've made. The first line of your message should be no longer than 80 characters (basically, fit it in the one line of that text box). Each paragraph should be separated by exactly one blank line.

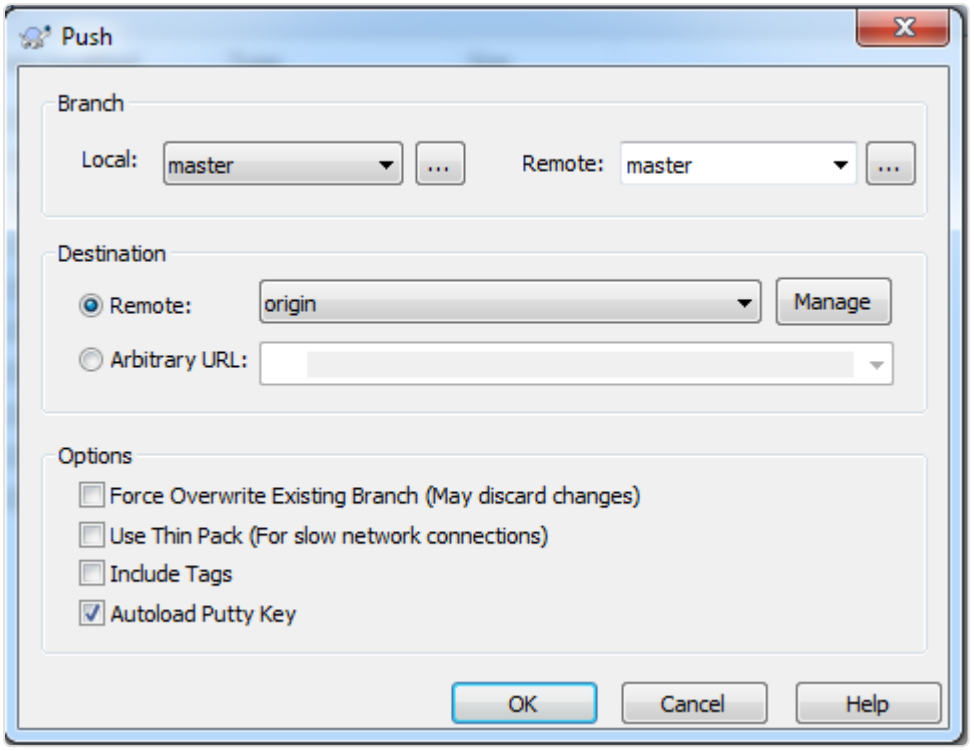
The defaults are usually what you want.

Hit OK to finalize your commit.

You should now be at this dialog:



At this point, we've made a commit on our local repository. We want to send that commit up to our fork [Jay/ T3000_Building_Automation_System]; this action is called "Pushing". TortoiseGit gives us a helpful button to push our new commit to our fork. Click Push.

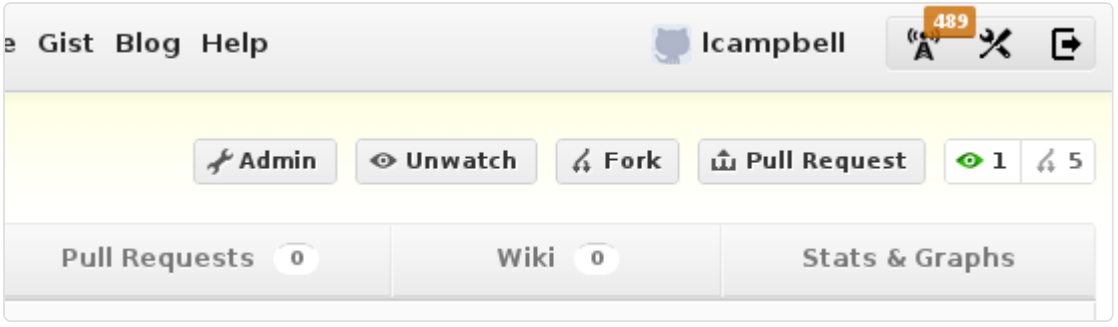


As usual, the defaults are probably what you want here. Make sure that Remote is set to origin, not upstream, then click OK.

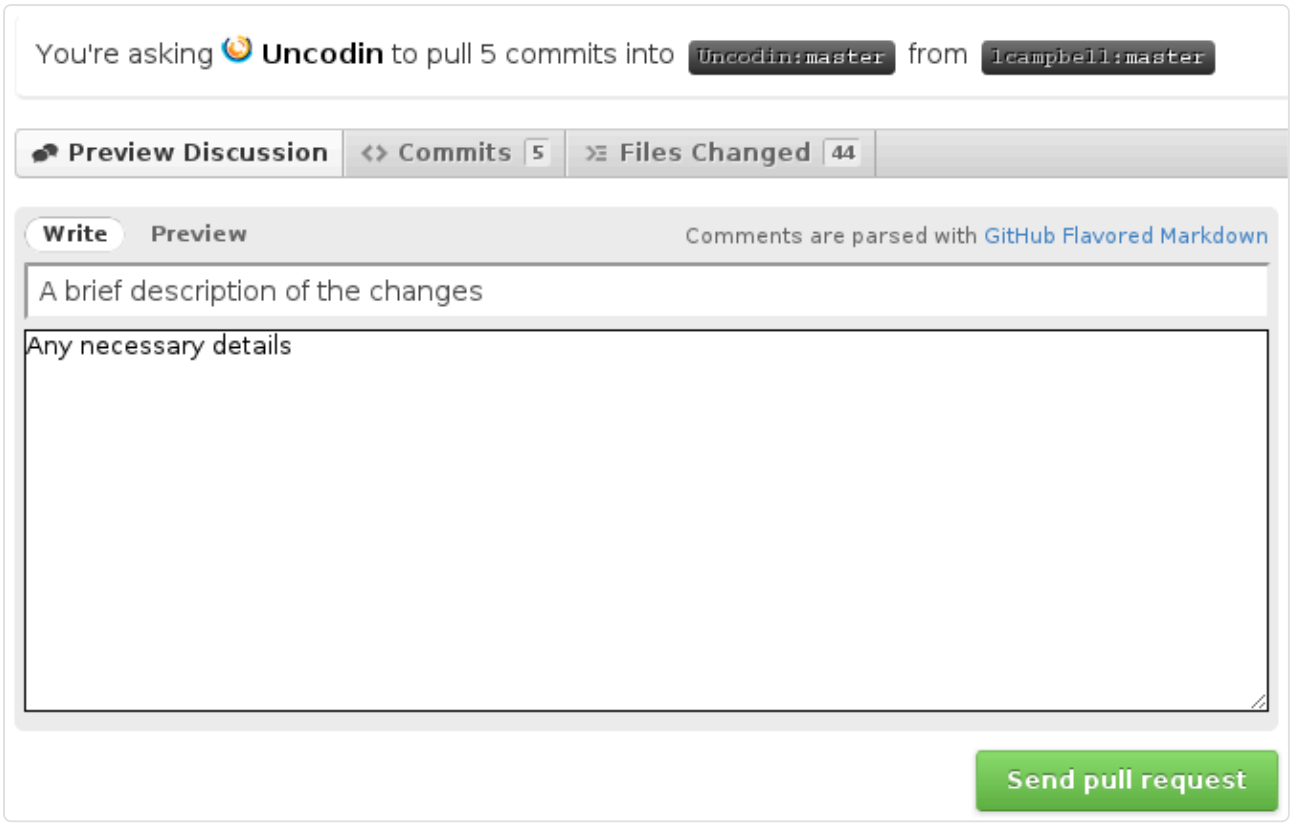
Once your commit is pushed to your Github fork, you can peek at your Github page with your browser to see the changes you've made. Note that they haven't been submitted for review for inclusion into the upstream repository. They're just on your fork.

Getting changes into the upstream repository

You've made changes to your Github fork [Jay/T3000_Building_Automation_System], but you need to get them into the upstream repository [temcocontrols/T3000_Building_Automation_System] for everyone else to have access to them. This is done by sending a pull request on the Github web interface. Log into your account and navigate to your fork's page. In the upper right-hand corner, there's a Pull Request button.



Click the Pull Request button and you'll be taken to the pull request page.



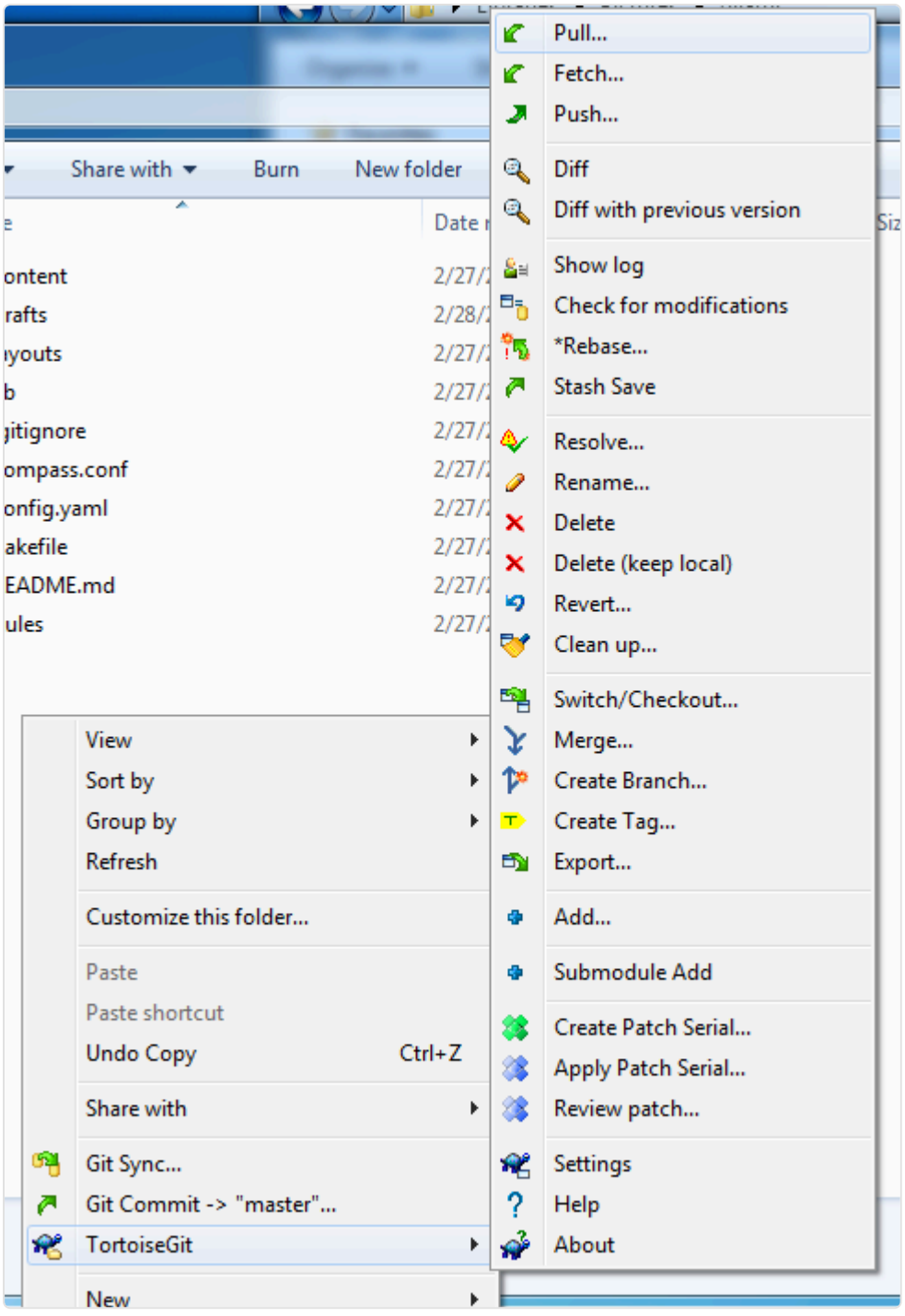
Throw in a title and a description for the pull request, and then hit Submit. We will be sent emails and will have a chance to add comments to your changes.

An important note is that any commits you send to your fork before the pull request is accepted will be merged into that pull request. This lets you make any changes in response to comments, but can be a bit of a surprise if you don't know about it ahead of time.

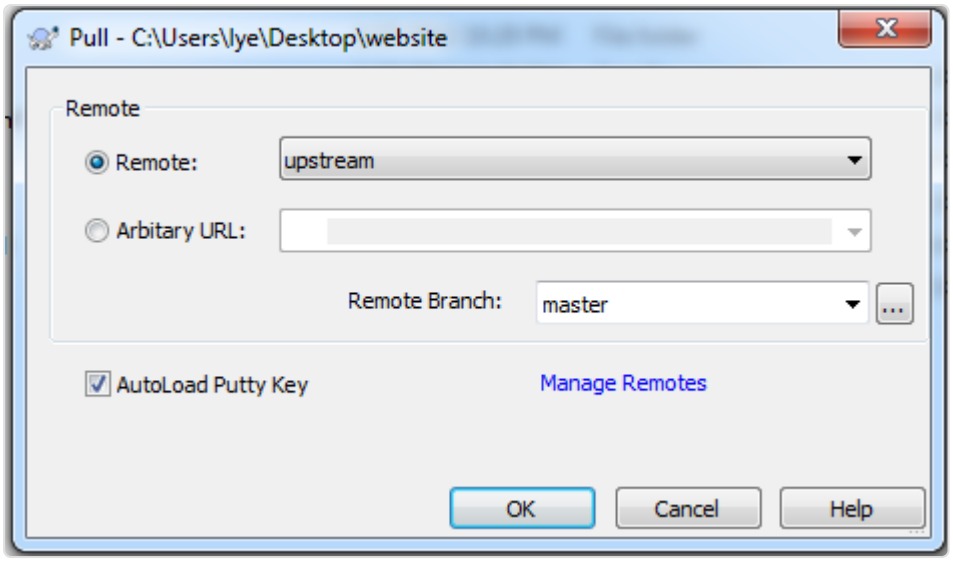
When the pull request is accepted, you'll be notified by email.

Getting upstream changes back into your copy

As all of this is going on, we'll be making changes and sending pull requests. To migrate those changes into your codebase, you'll need to use a Pull command. From the right-click context menu, select Pull.



In the dialog that pops up, select upstream as the remote.



Then just hit OK and any changes made by other people will be pulled into your local repository. They'll be sent to your Github fork when you next push to it.

Conclusion

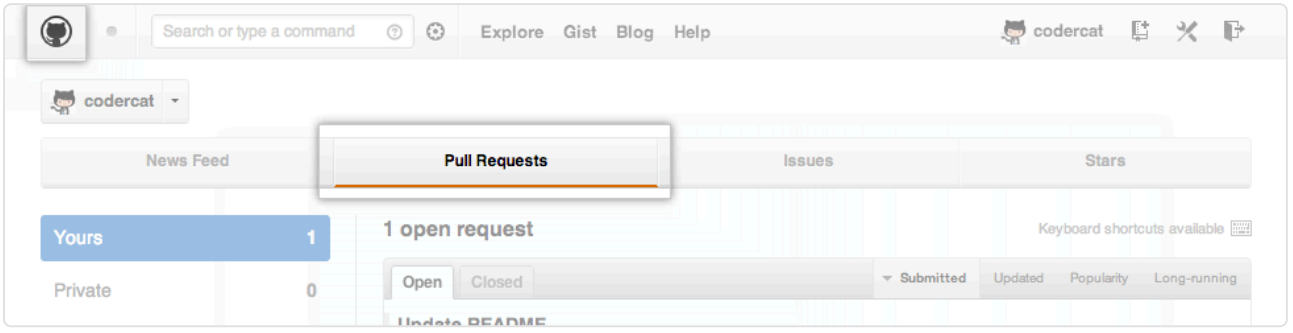
In this way, TortoiseGit and Github are connected. There are built-in tools to establish a proper code-review workflow.

Accepting Pull Requests and Merging changes to upstream repo

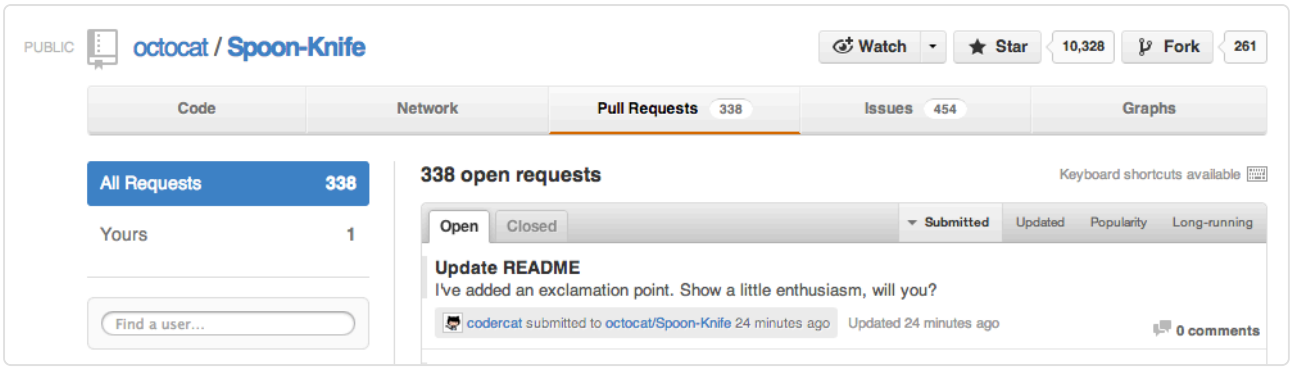
By [temcocontrols/T3000_Building_Automation_System]

Managing Pull Requests

All pull requests sent or received by you are browsable through the pull request dashboard.



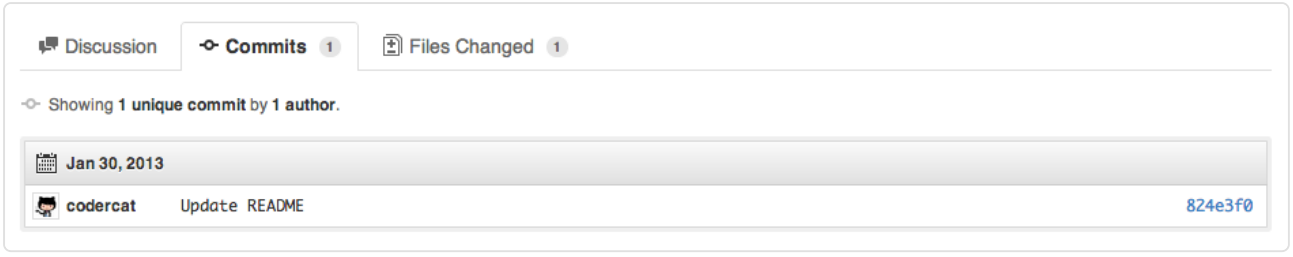
Pull requests for a specific repository are also browsable by anyone with access by visiting the Pull Requests page.



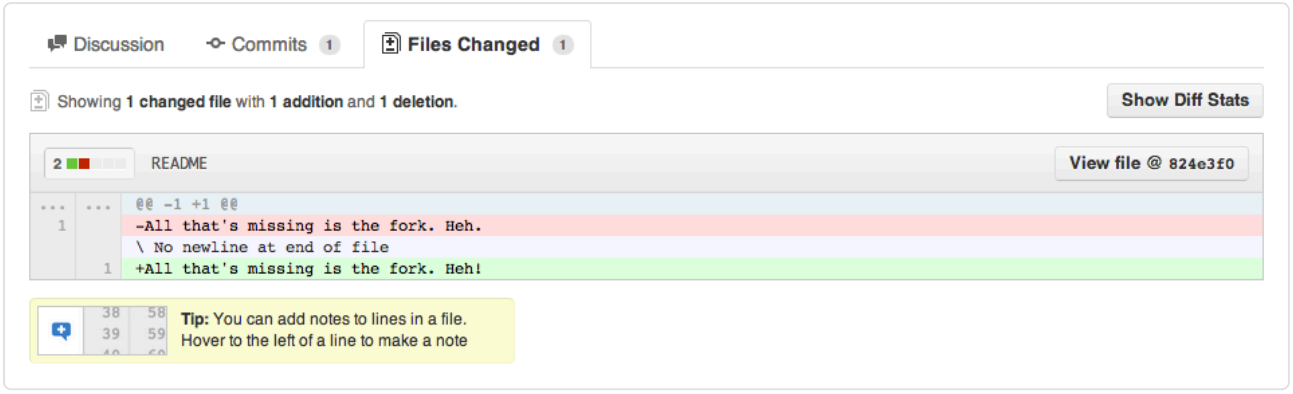
The pull request dashboard and the repository pull request list support a wide range of filtering and sorting controls. Use them to narrow down the list to the pull requests you're interested in.

Reviewing Proposed Changes

When you receive a pull request, the first thing to do is review the set of proposed changes. Pull requests are tightly integrated with the underlying git repository, so you can see exactly what commits would be merged should the request be accepted:

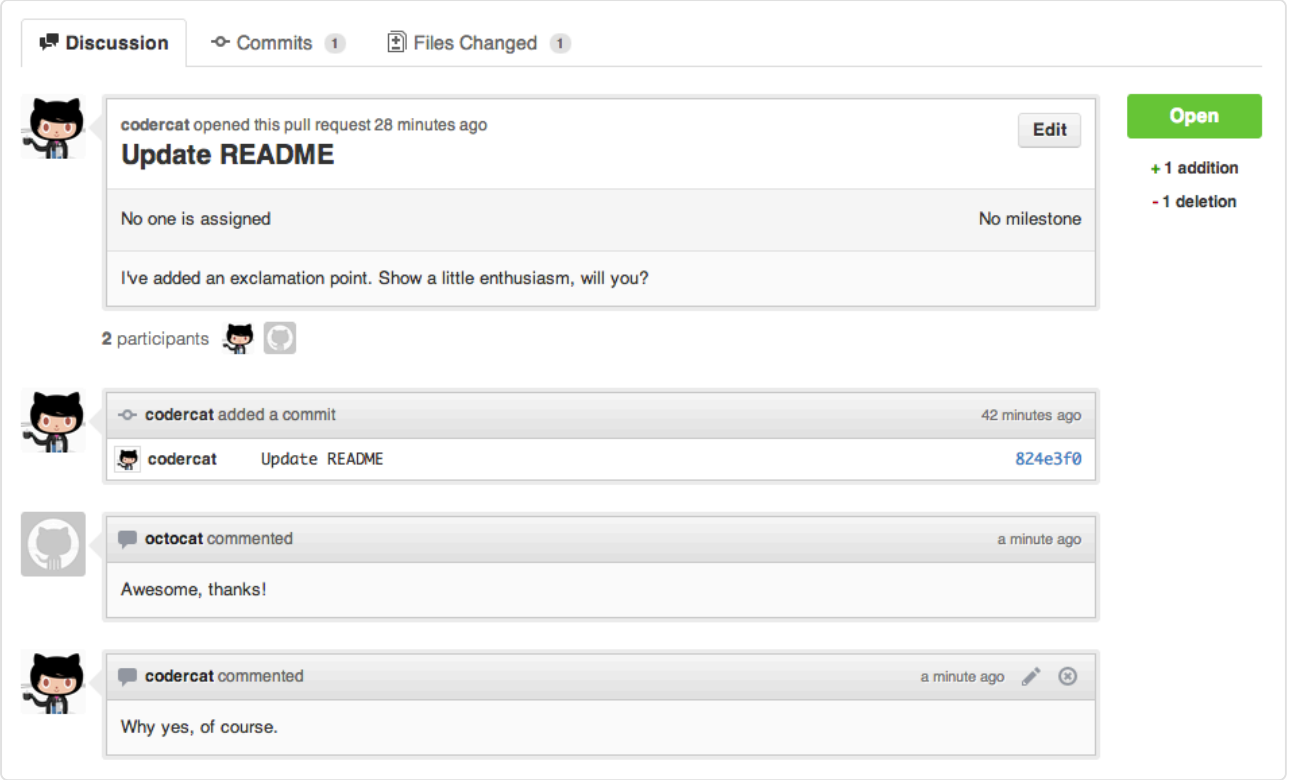


You can also review the cumulative diff of all file changes across all commits.



Pull Request Discussion

After reviewing the basic description, commits, and cumulative diff, the person tasked with applying the changes may have questions or comments. Perhaps the coding style doesn't match project guideline, or the change is missing unit tests, or maybe everything looks great and some props are in order. The discussion view is designed to encourage and capture this type of discussion.



The discussion view starts with the pull request's original title and description and then captures additional activity to display chronologically from there. Any of the following types of activity are captured as they happen:

Comments left on the pull request itself.

Additional commits pushed to the pull request's branch.

File and line notes left on any of the commits included in the pull request's range.

Pull request comments are Markdown compatible, so you can embed images, use preformatted text blocks, and other formatting supported by Markdown.

Merging a Pull Request

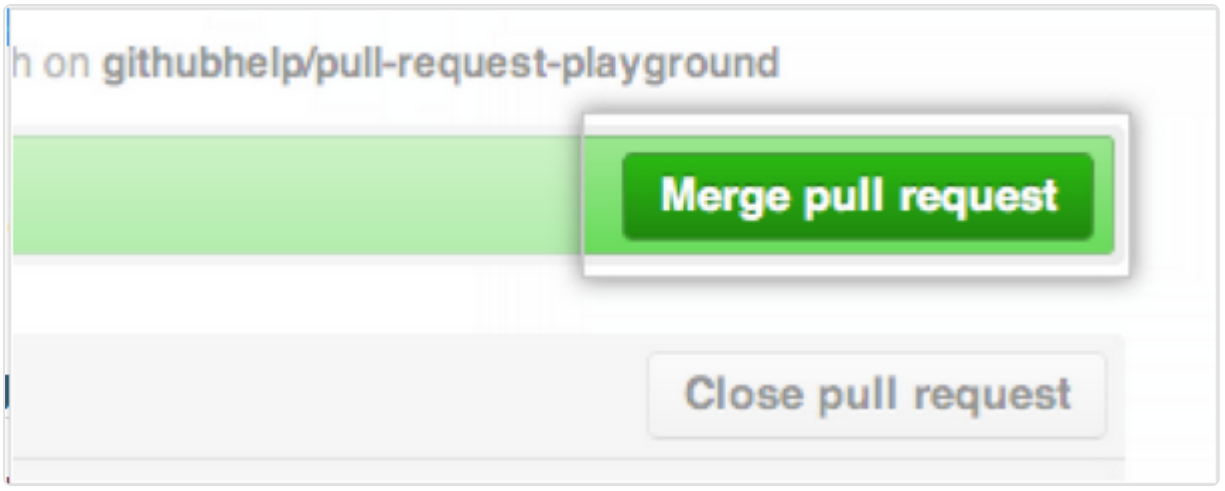
Once a pull request is deemed satisfactory, anyone with push access to the destination repository can perform the merge. There are a variety of ways to accomplish this. Two popular methods are described below.

Merge directly on Github

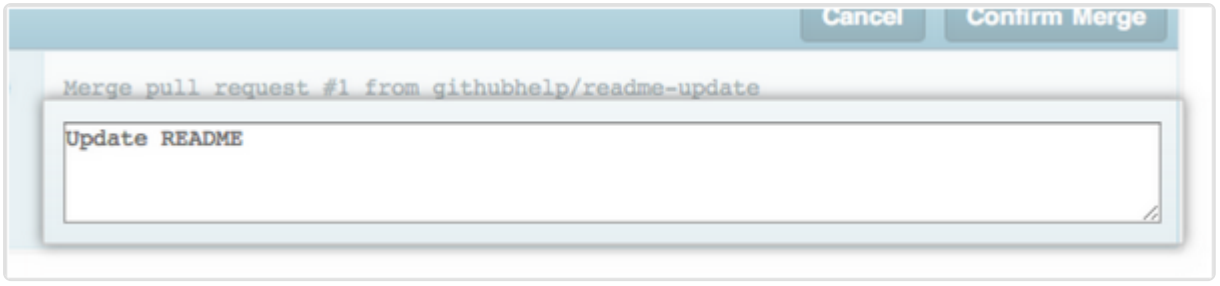
If the merge will not have any conflicts, you can merge the pull request online without ever typing a single git command.

Open the pull request's page

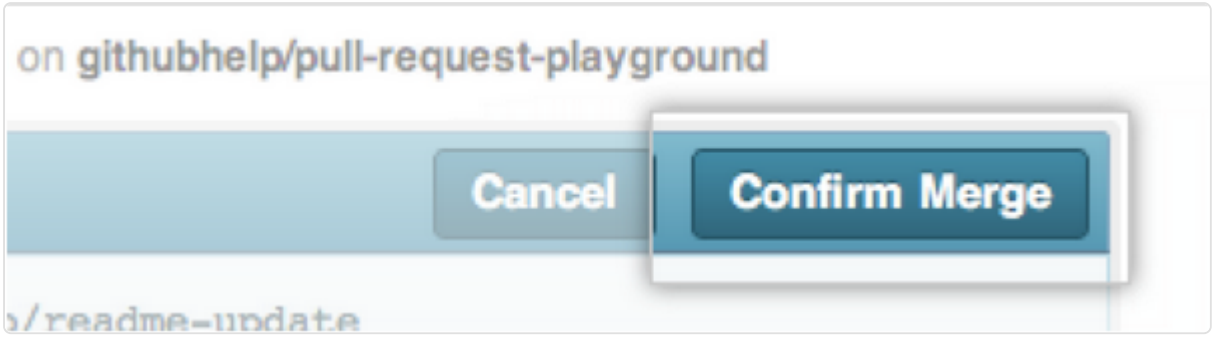
Click the "Merge pull request" button



Enter a commit message



Click “Confirm Merge”



TortoiseGit and Github workflow for T3000_Building_Automation_System project

Before reading this chapter, make sure you have read and understood chapter 6. This chapter only builds on the concepts explained in chapter 6 and makes use of those concepts w.r.t. T3000_Building_Automation_System project. This chapter would be mostly containing screen shots of T3000 project and related steps.

Github Terms:

Base repo: temcocontrols/T3000_Building_Automation_System. Base repo is the main repository of the code. This shall correspond to the finalized work of the team.

Fork repo: Each fork of the main repo corresponds to a team member’s work. In my case, this repo is: jay-github/ T3000_Building_Automation_System

Branch: Each branch within the fork and/or in the main repo can correspond to several kinds of things depending on how you want to work. Usually, each branch refers to a version of the project but can also correspond to different channels of development, like hotfixes or experimental work.

Pull Request: The pull request (in the Github ecosystem) corresponds to the task. Every time I want to contribute an isolated finished task to the main repo, I create a pull request corresponding to the commits made in that task. These commits are pulled from my fork to the main repo.

The ecosystem for Temco Controls software development is proposed as follows:

temcocontrols/T3000_Building_Automation_System remains the base repo controlled from a single point.

Each developer will fork his or her own repo. With this logic, my repository would be jay-github/T3000_Building_Automation_System.

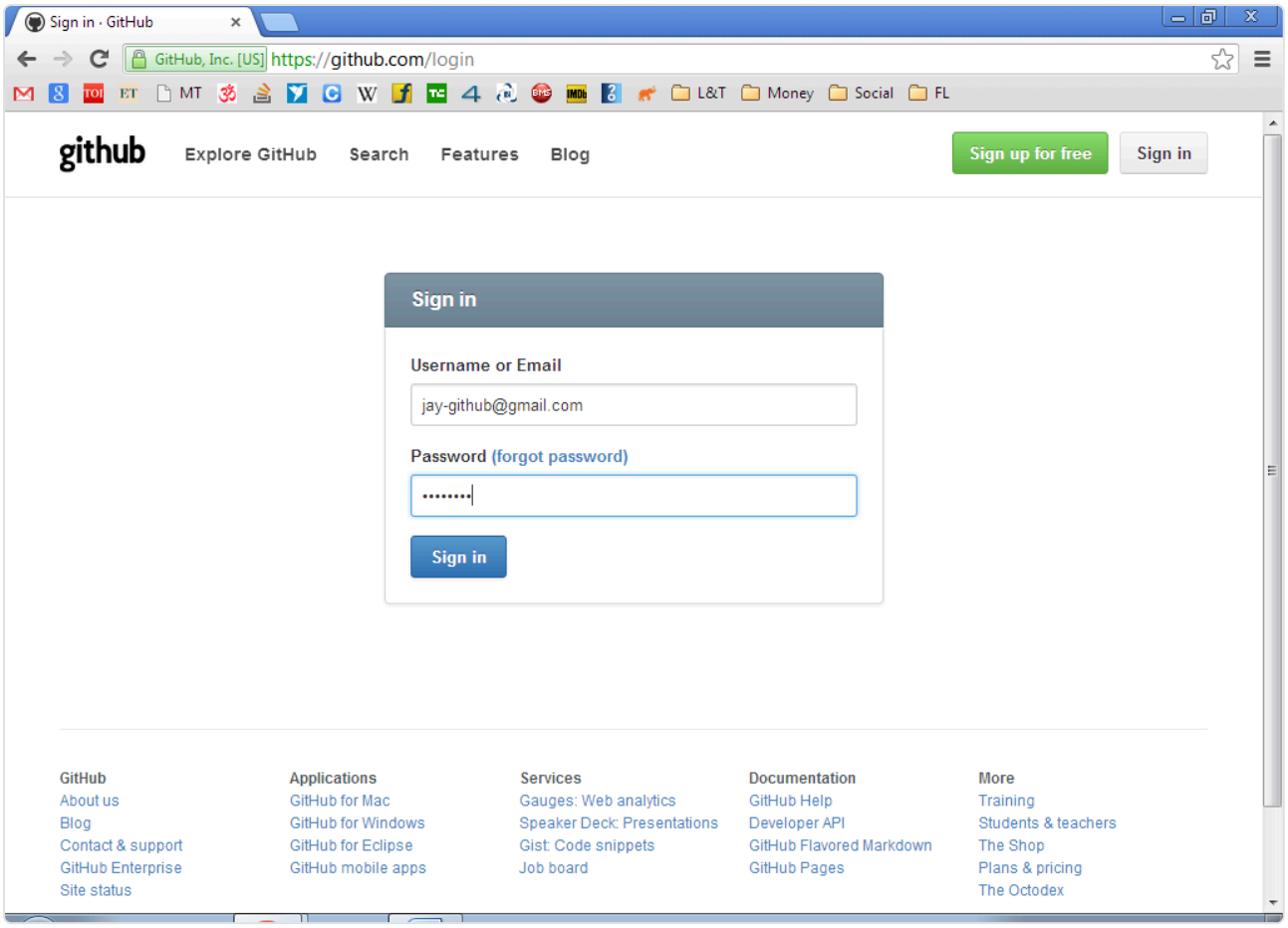
After forking the repo, each developer shall clone that repo in his own local PC. Work there and Push the changes back to his fork.

Once the changes are pushed to fork, developer shall send a Pull Request to temcocontrols on github.

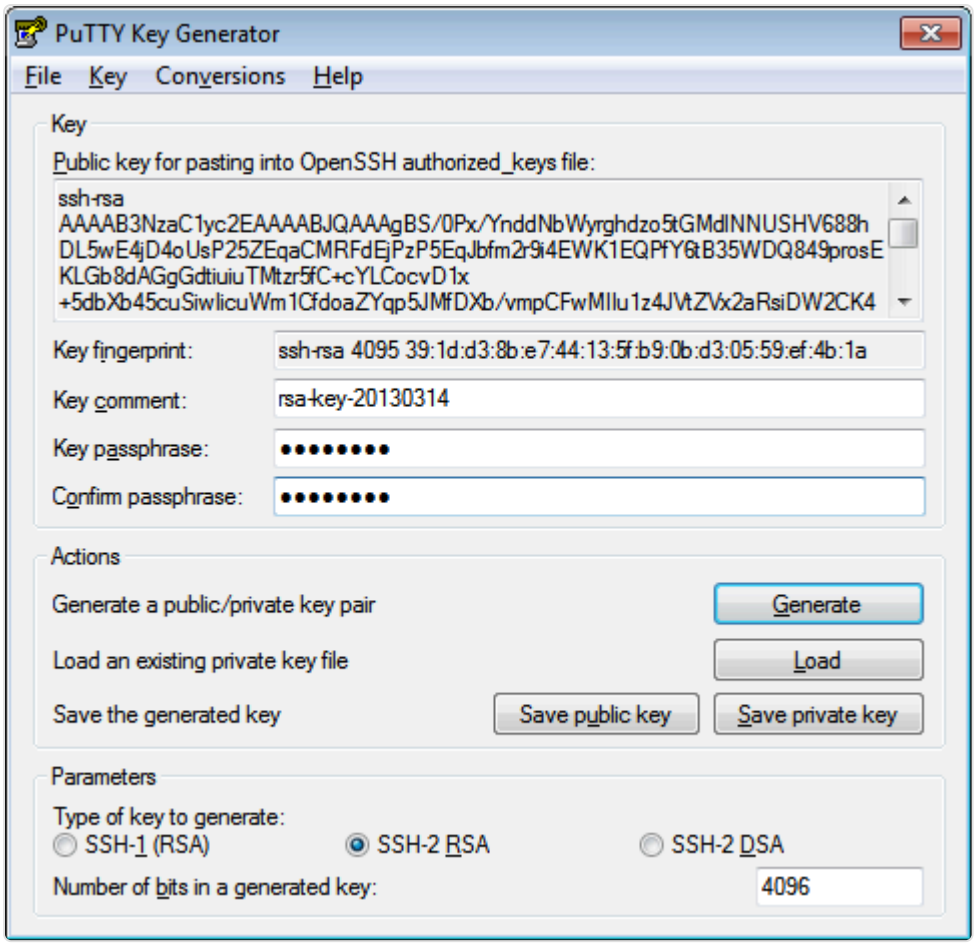
The single person controlling temcocontrols repo would be able to review the work done by developer and also able to accept and merge or reject the pull request.

Let’s see how this can be done.

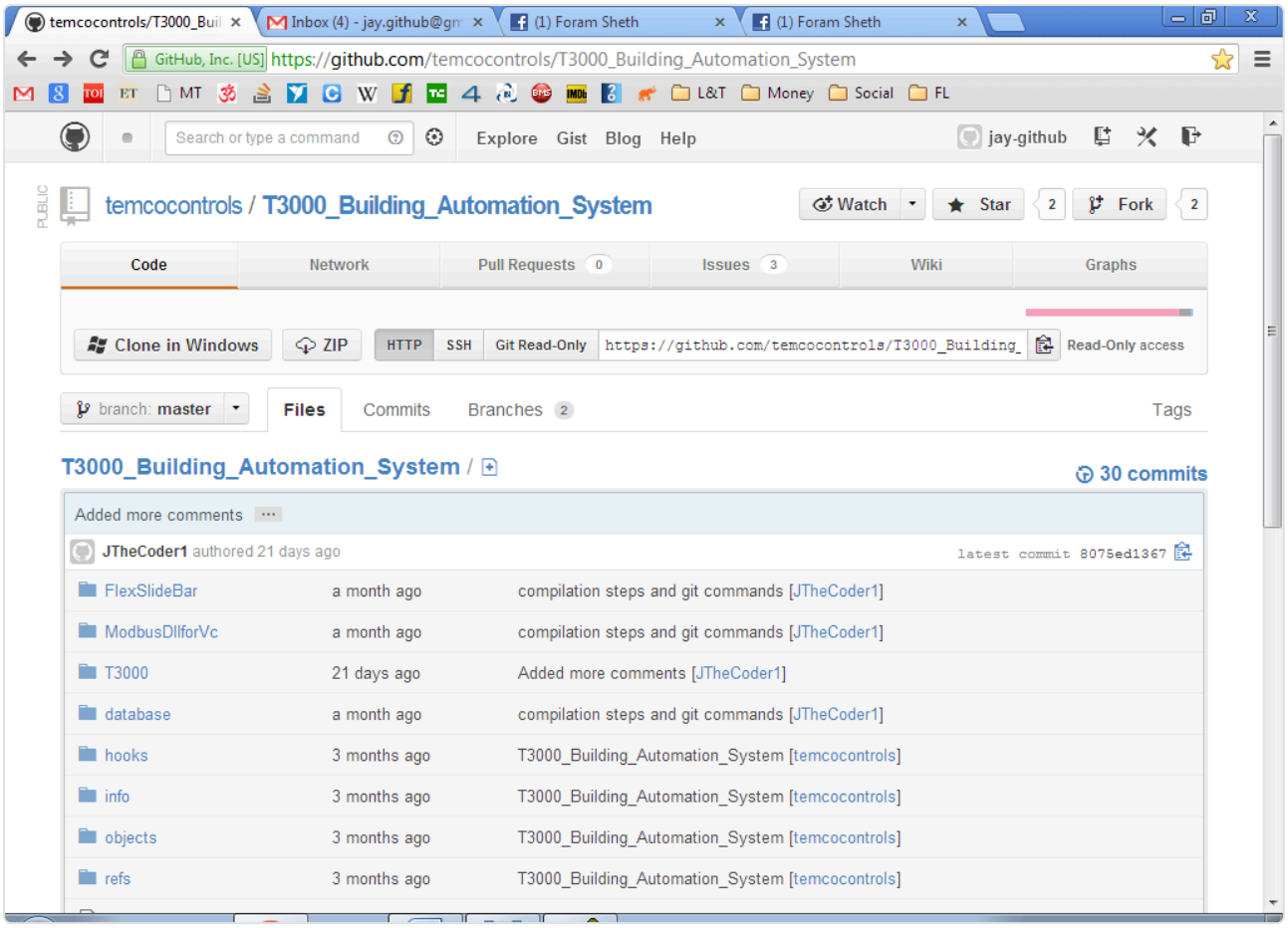
Sign in to the developer account.



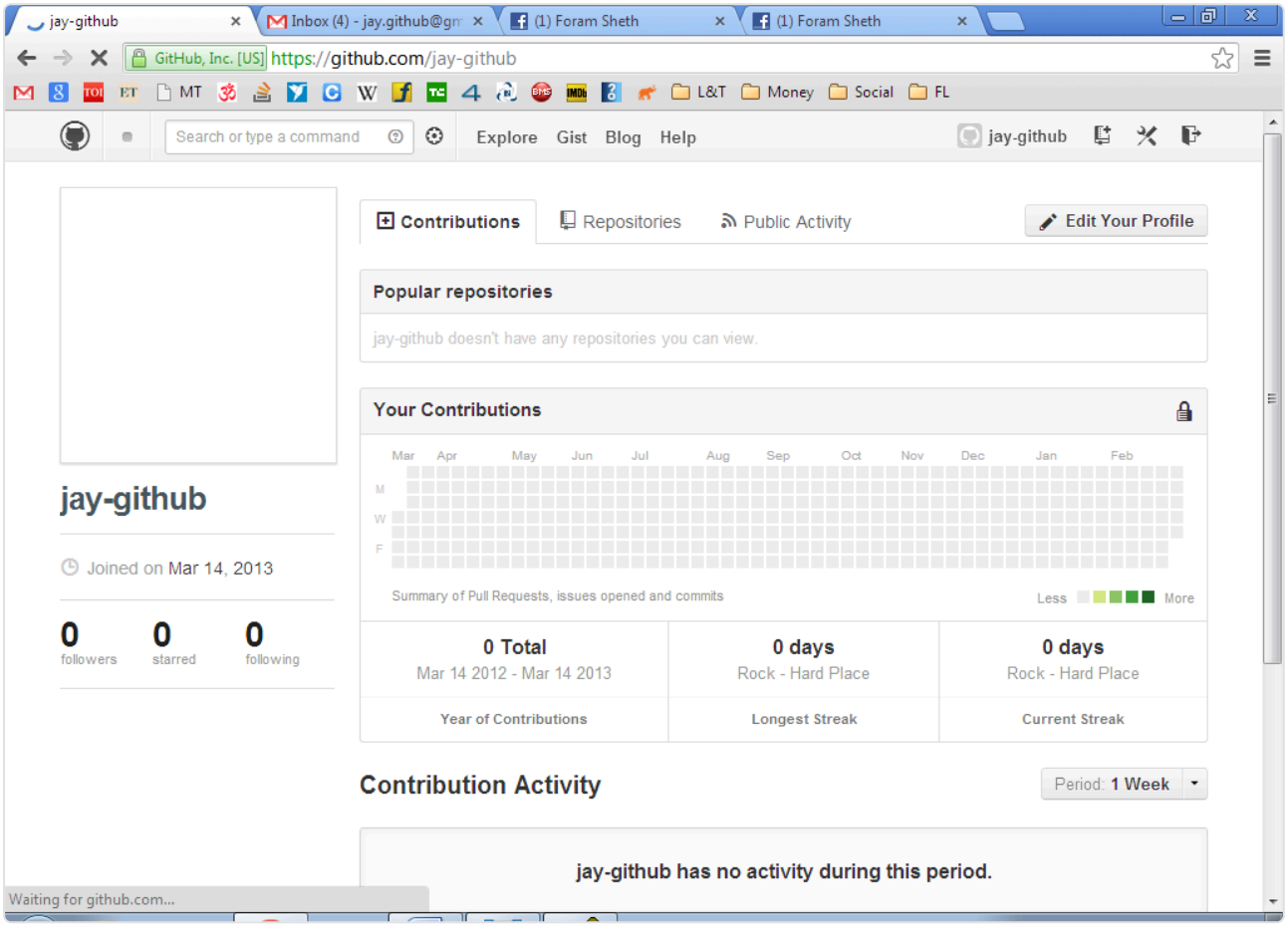
Generate SSH key.



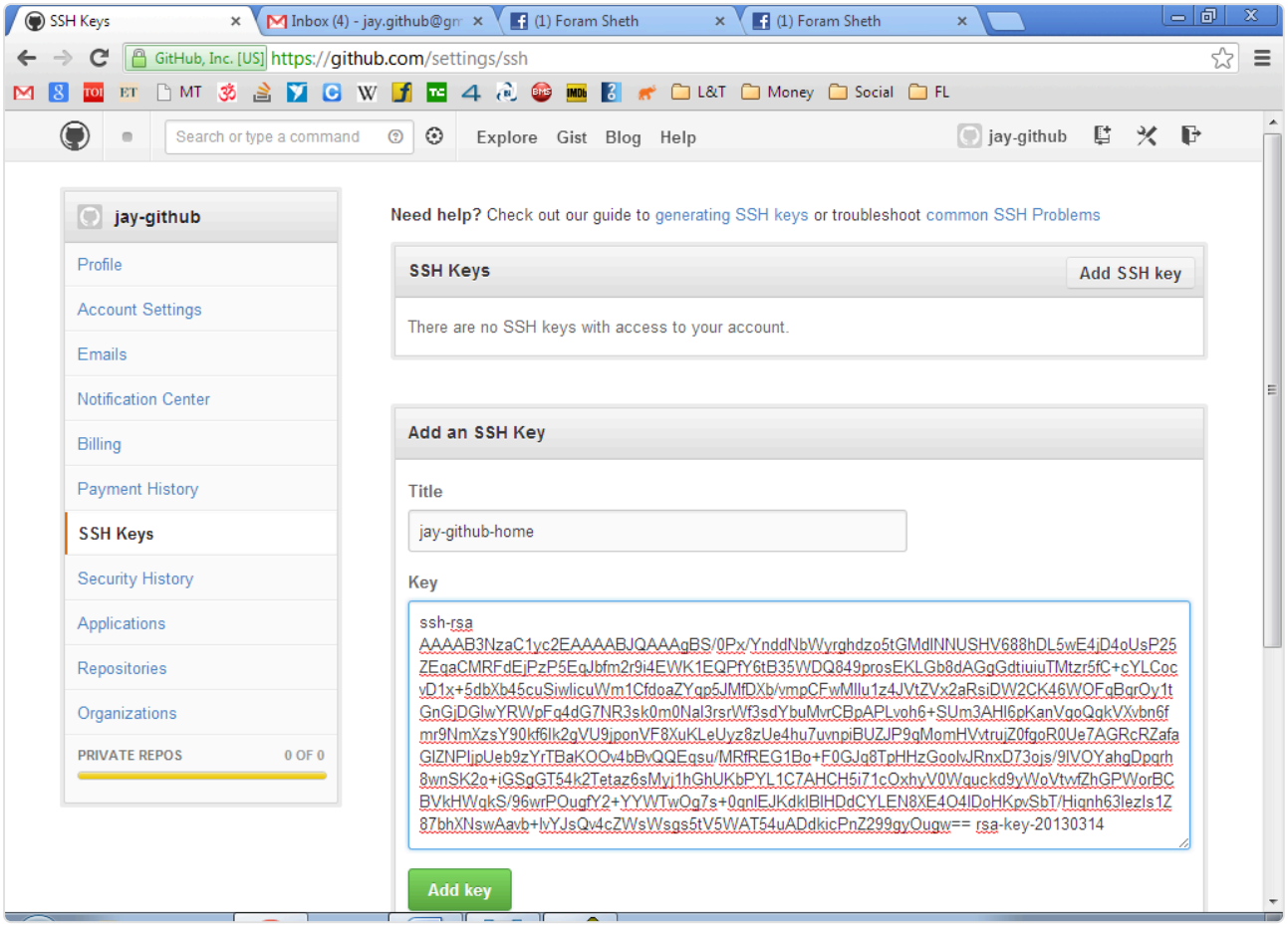
Screen below shows signed in home page for developer.



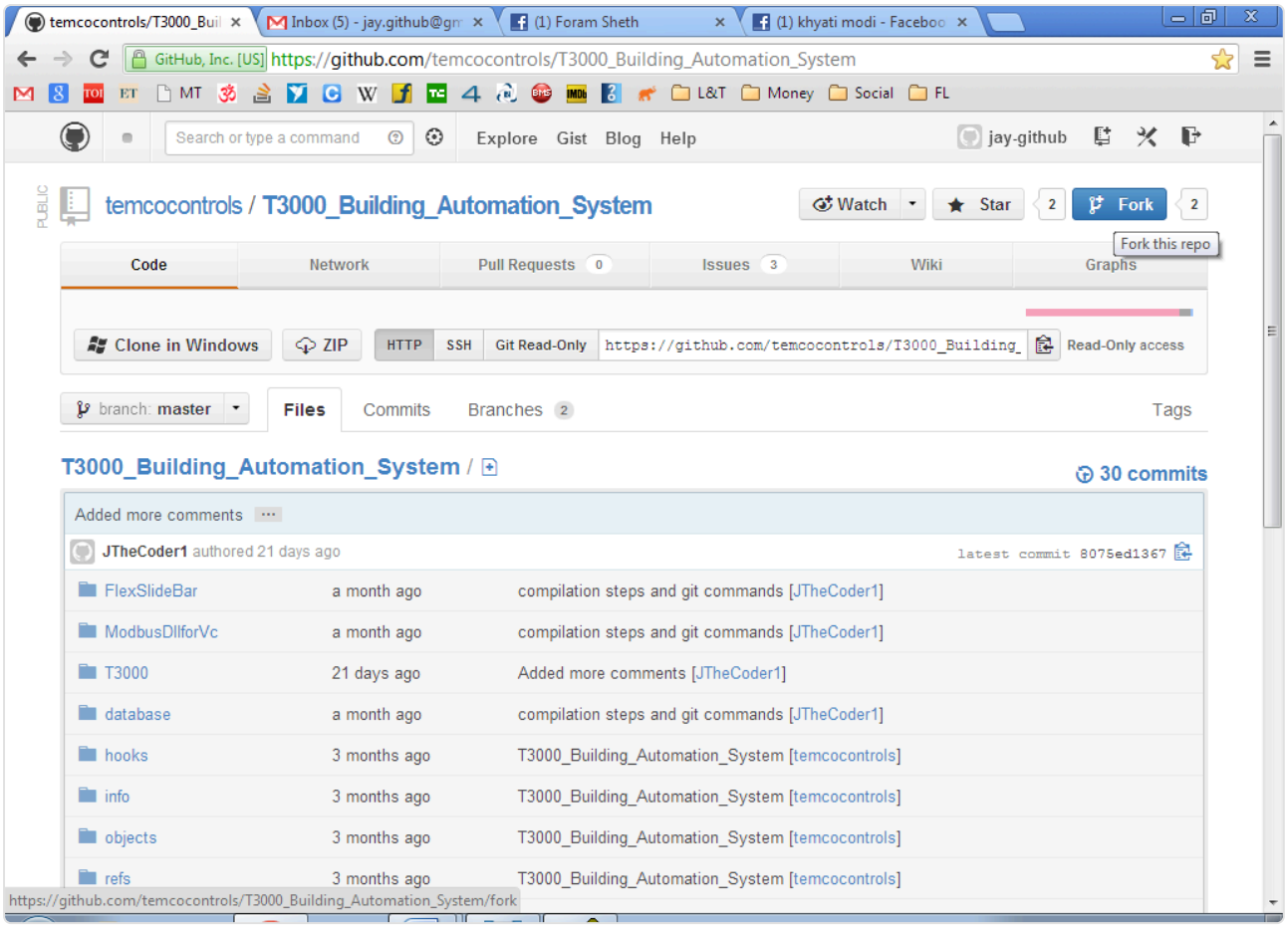
Go to Edit Your Profile.



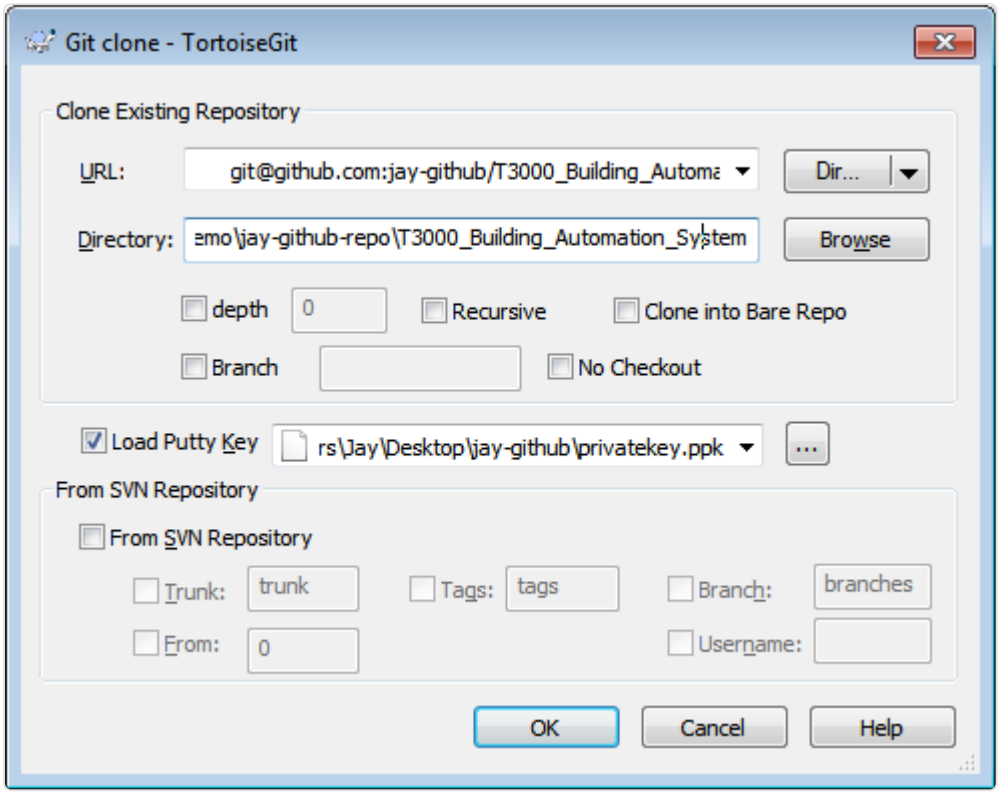
Add SSH key.



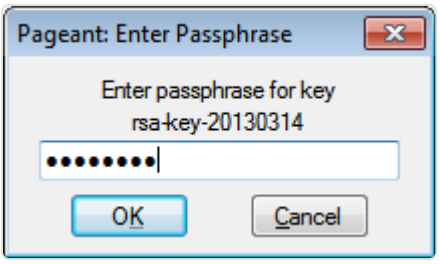
Fork temcocontrols/T3000... project



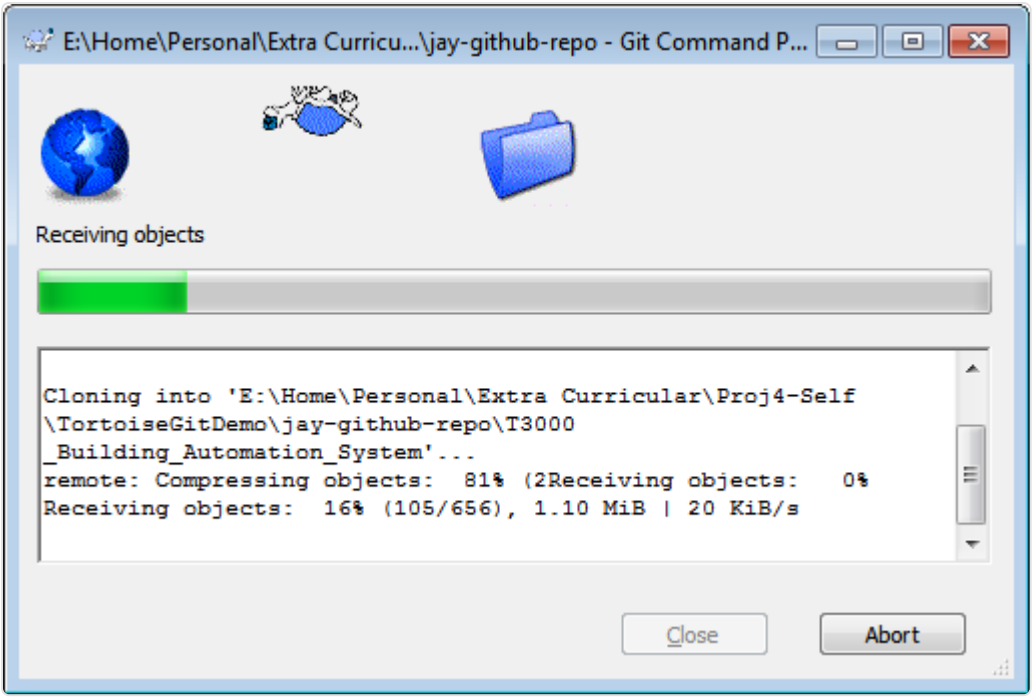
Clone fork into local PC using TortoiseGit.



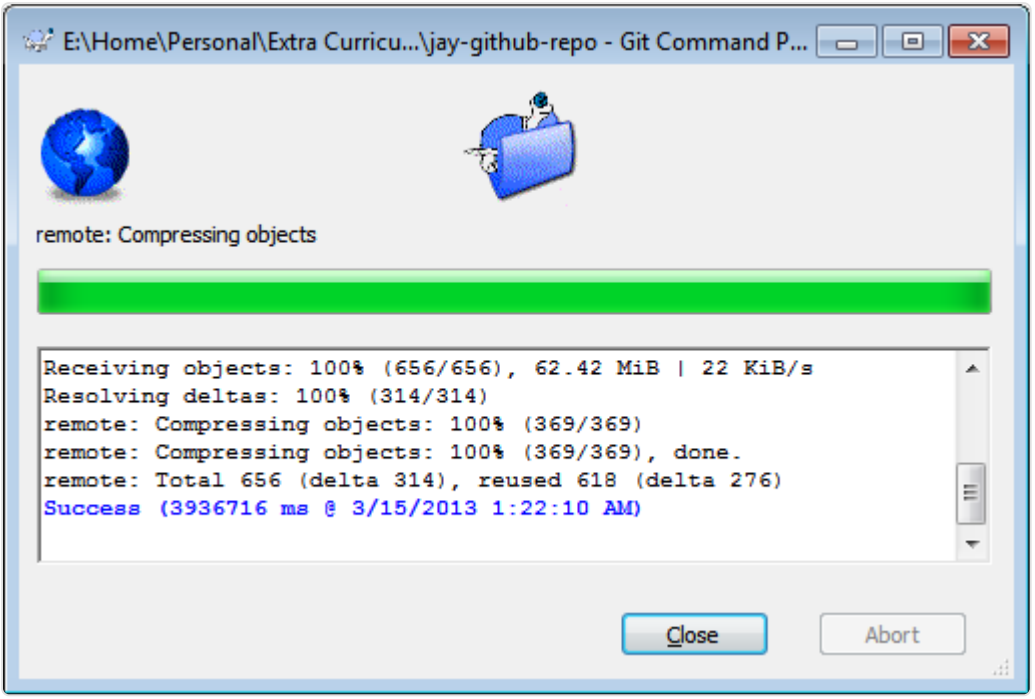
Enter passphrase.



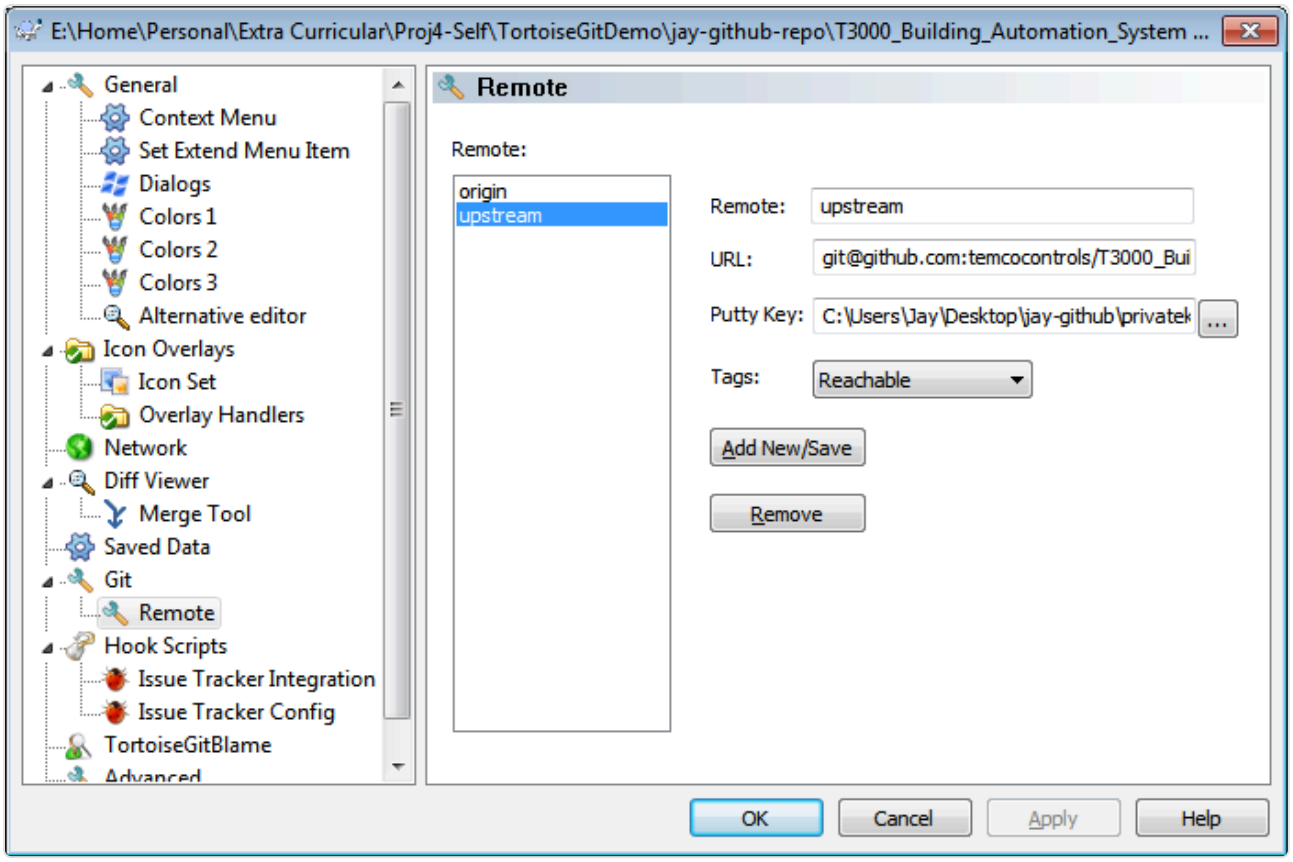
Cloning may take some time.



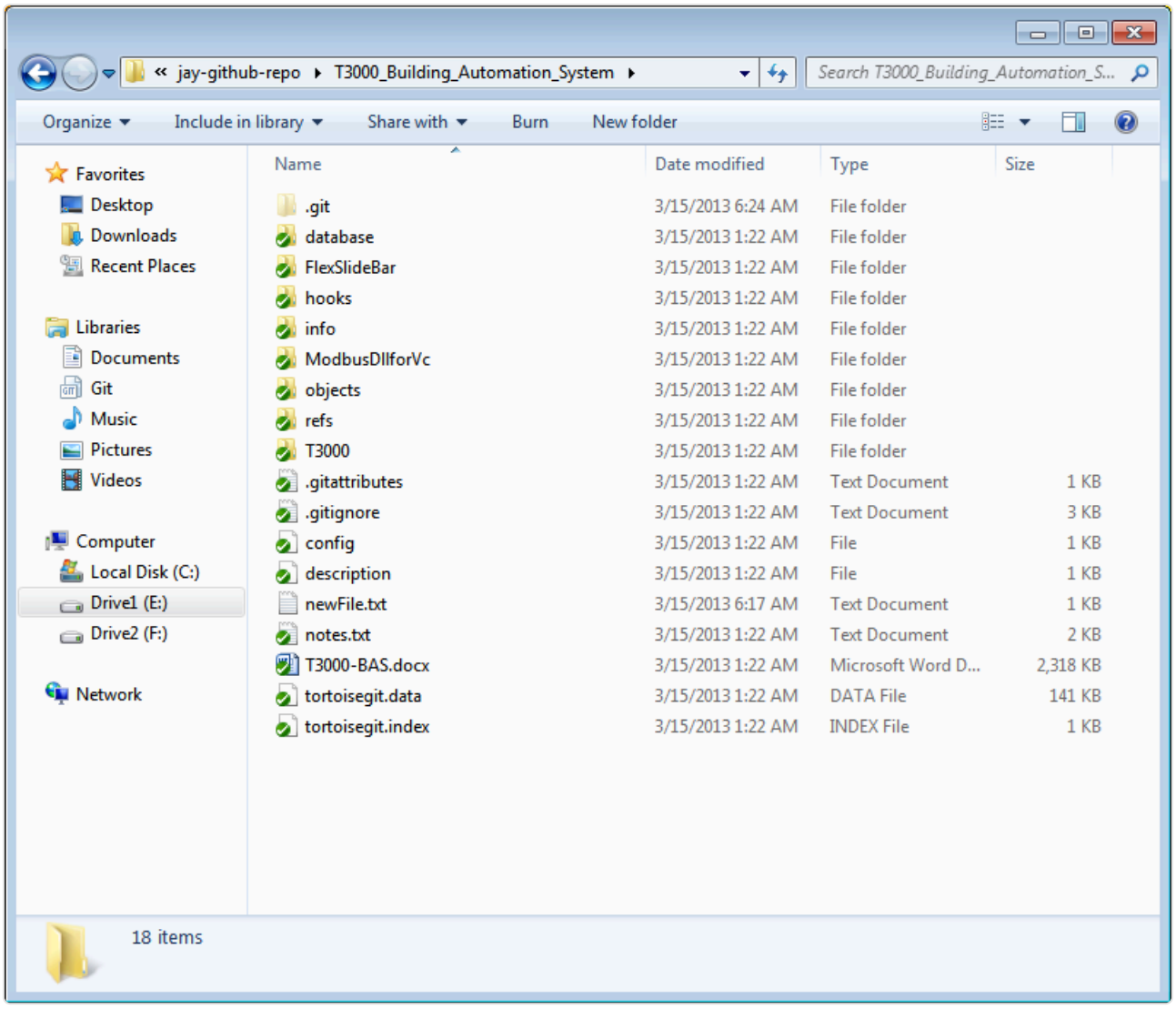
Eventually it will get over.



Enter details of main repo (temcocontrols/T3000... repo)

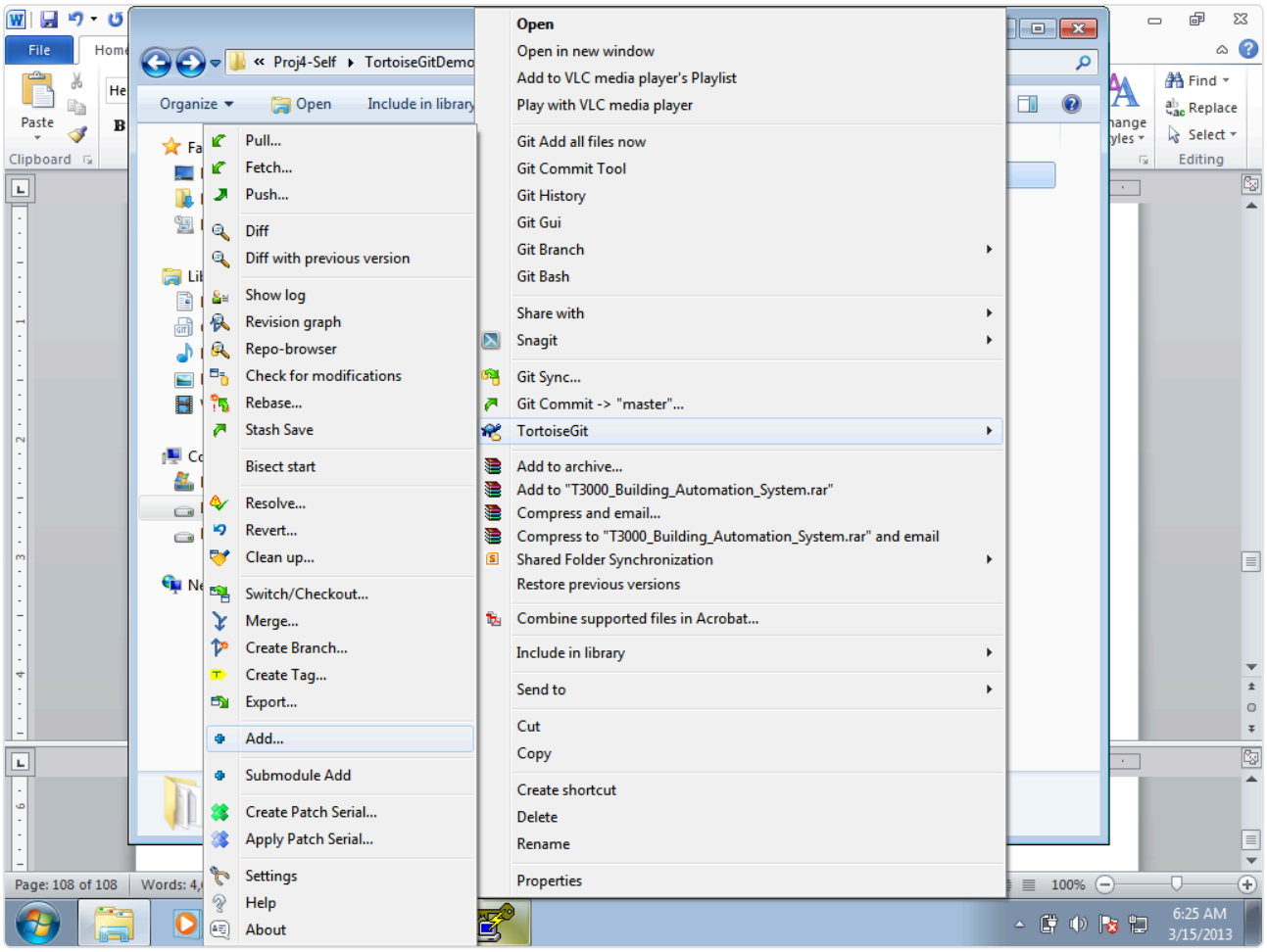


Here, for demo purpose, I have added newFile.txt in the repo directory on my local PC.

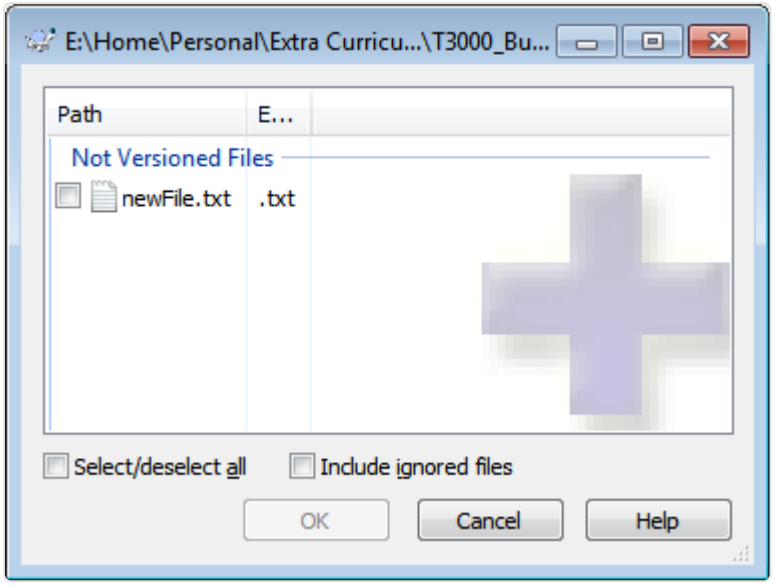


Now, it's time to send my changes to single person controlling temcocontrols repo. This is done by first committing changes, pushing them to own fork and then sending a pull request to temcocontrols.

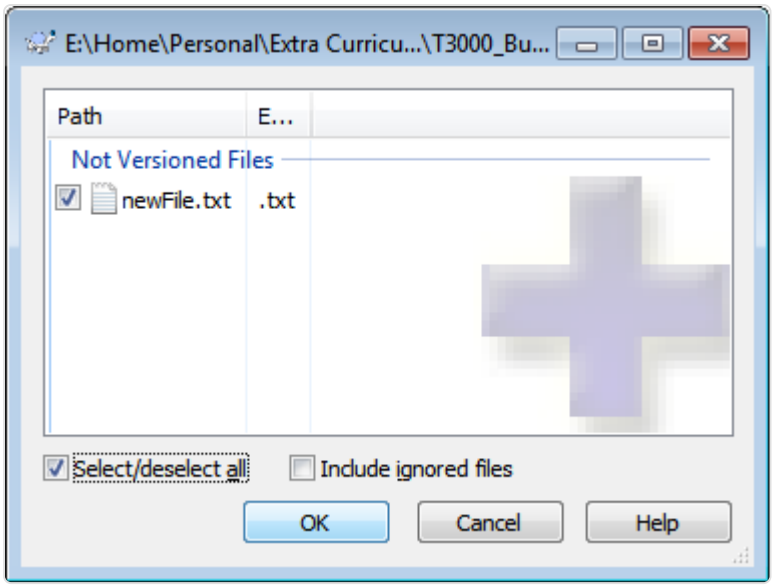
Add... is shown below.



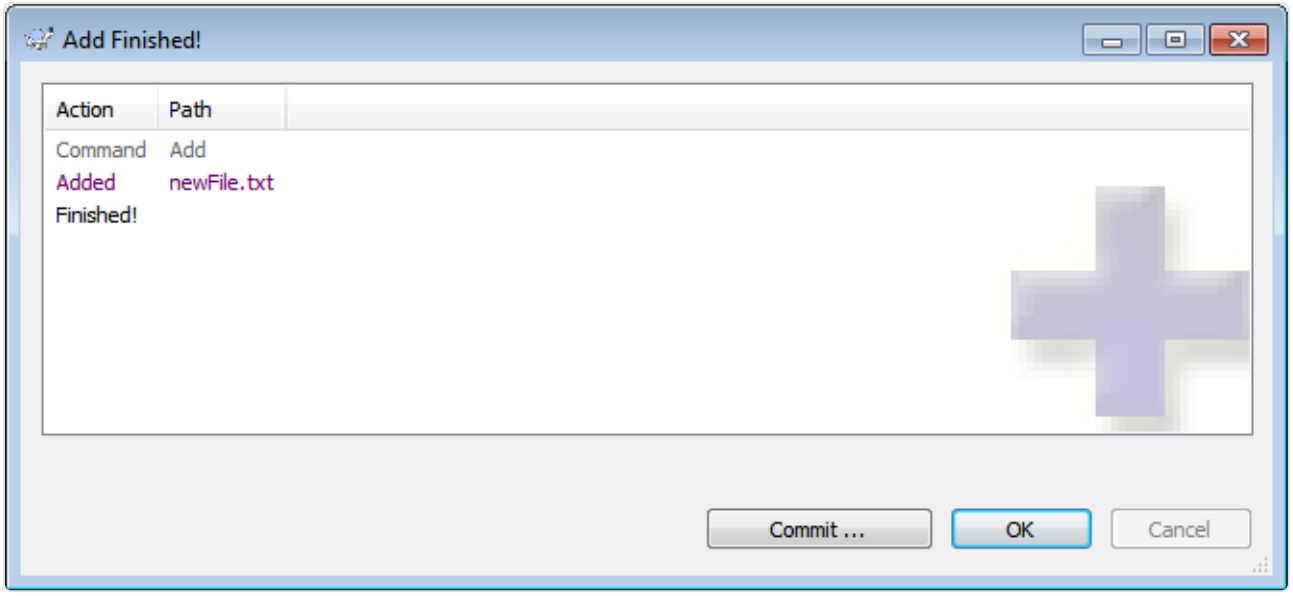
TortoiseGit confirms what needs to be added.



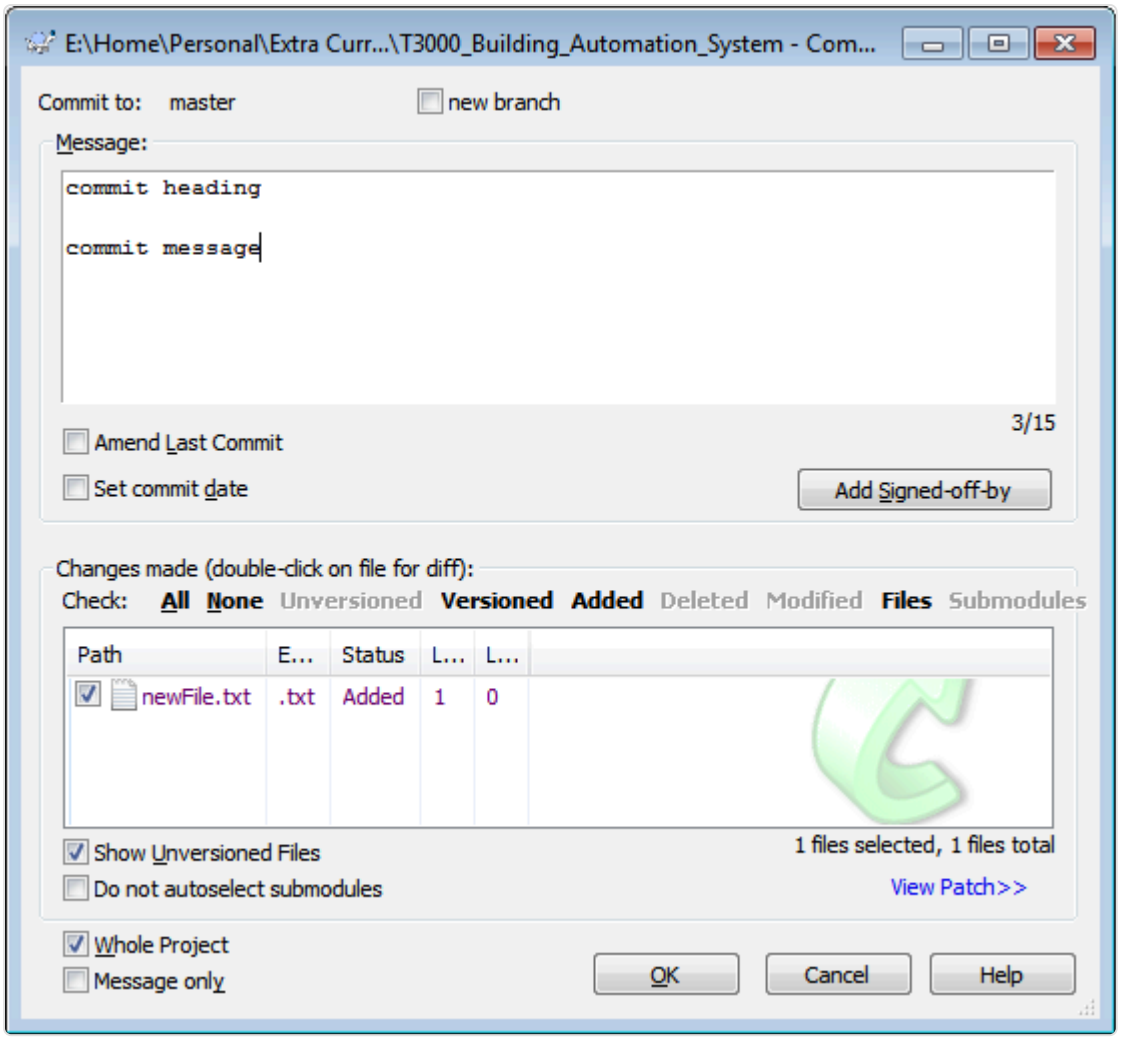
Select files to be added and click ok.



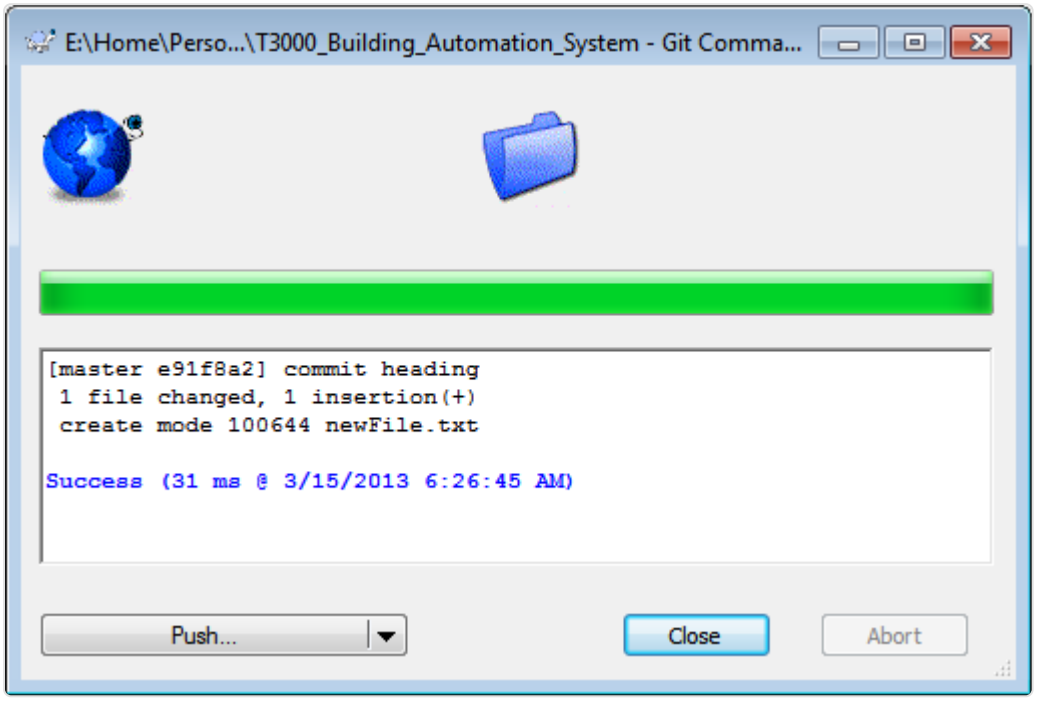
File is added and click on Commit... to commit changes on local PC.



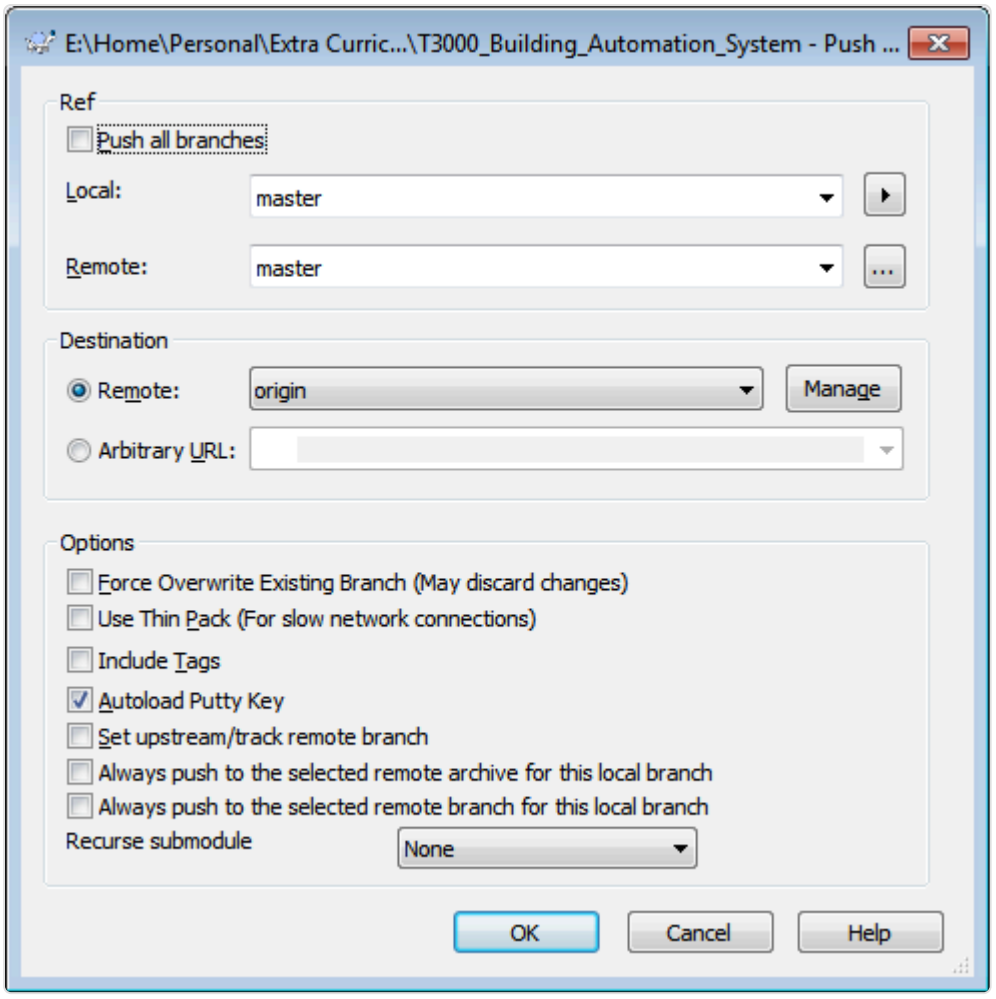
Commit dialog box appears. Enter commit message and click ok.



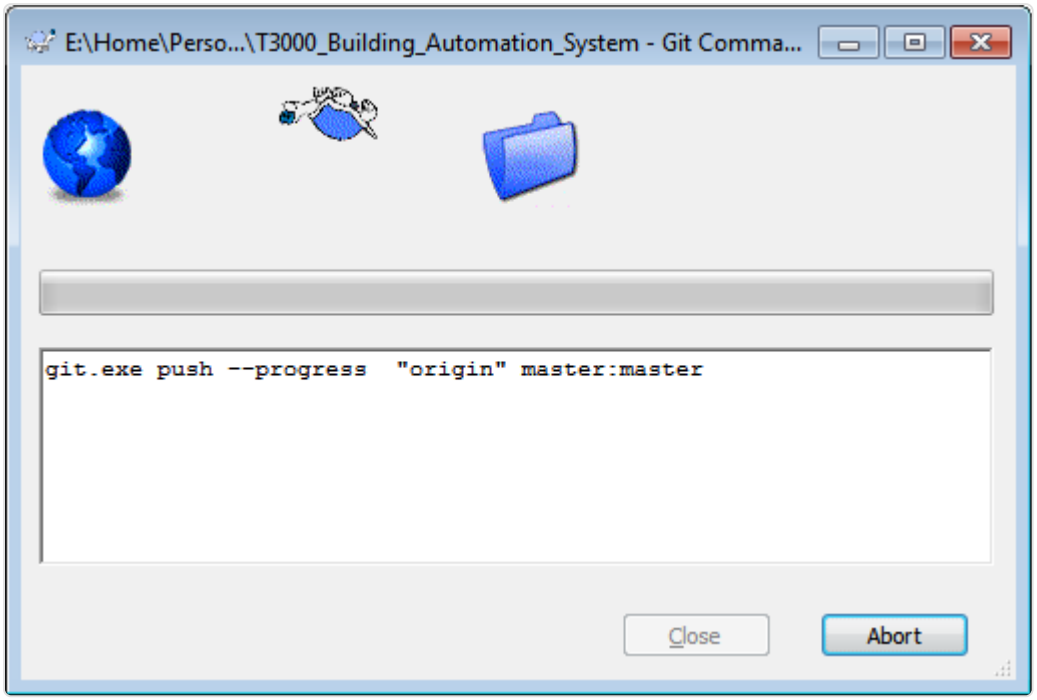
Once committed, if you want to send changes to your fork on github, click Push... or click Close.



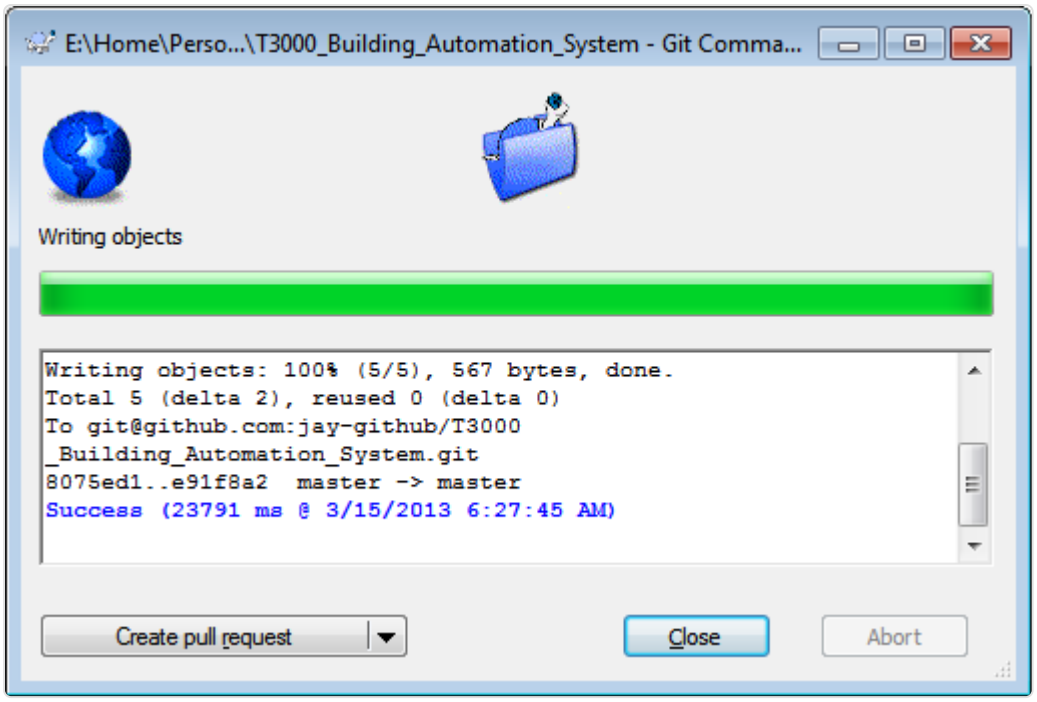
Select Destination as origin. Not upstream. Else changes would go to main repo instead of your fork and click ok.



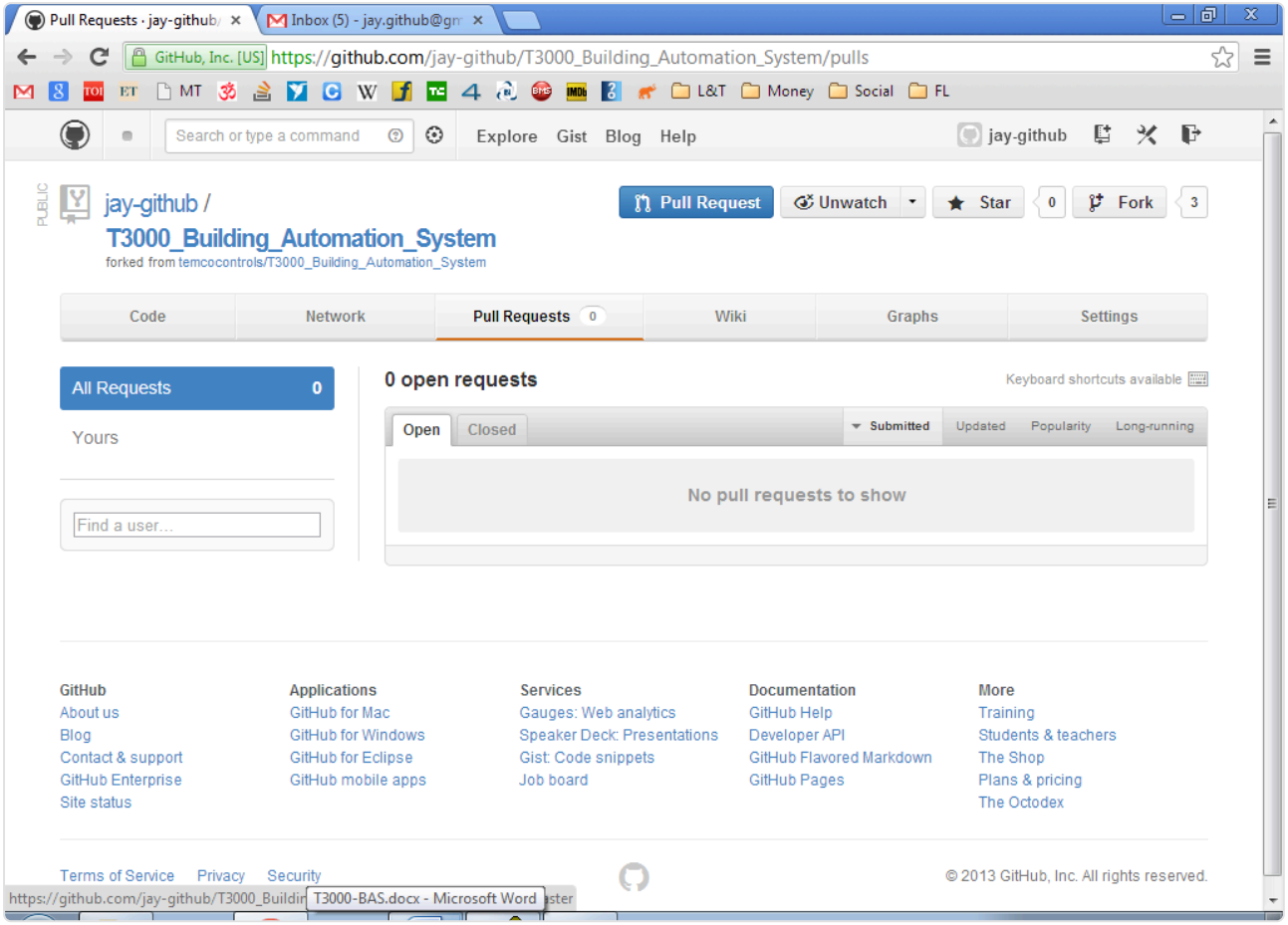
Some waiting...



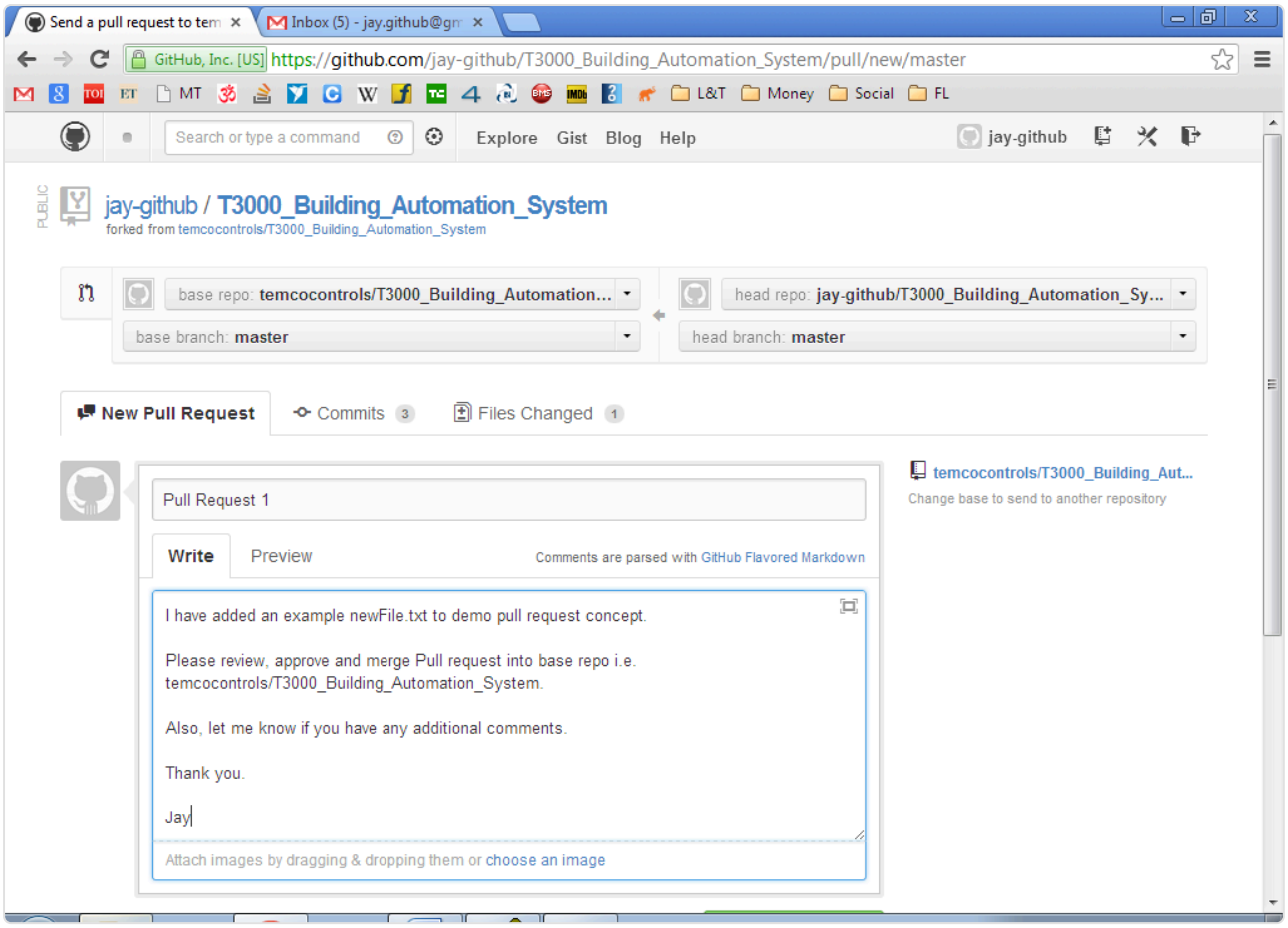
And pushing is done.



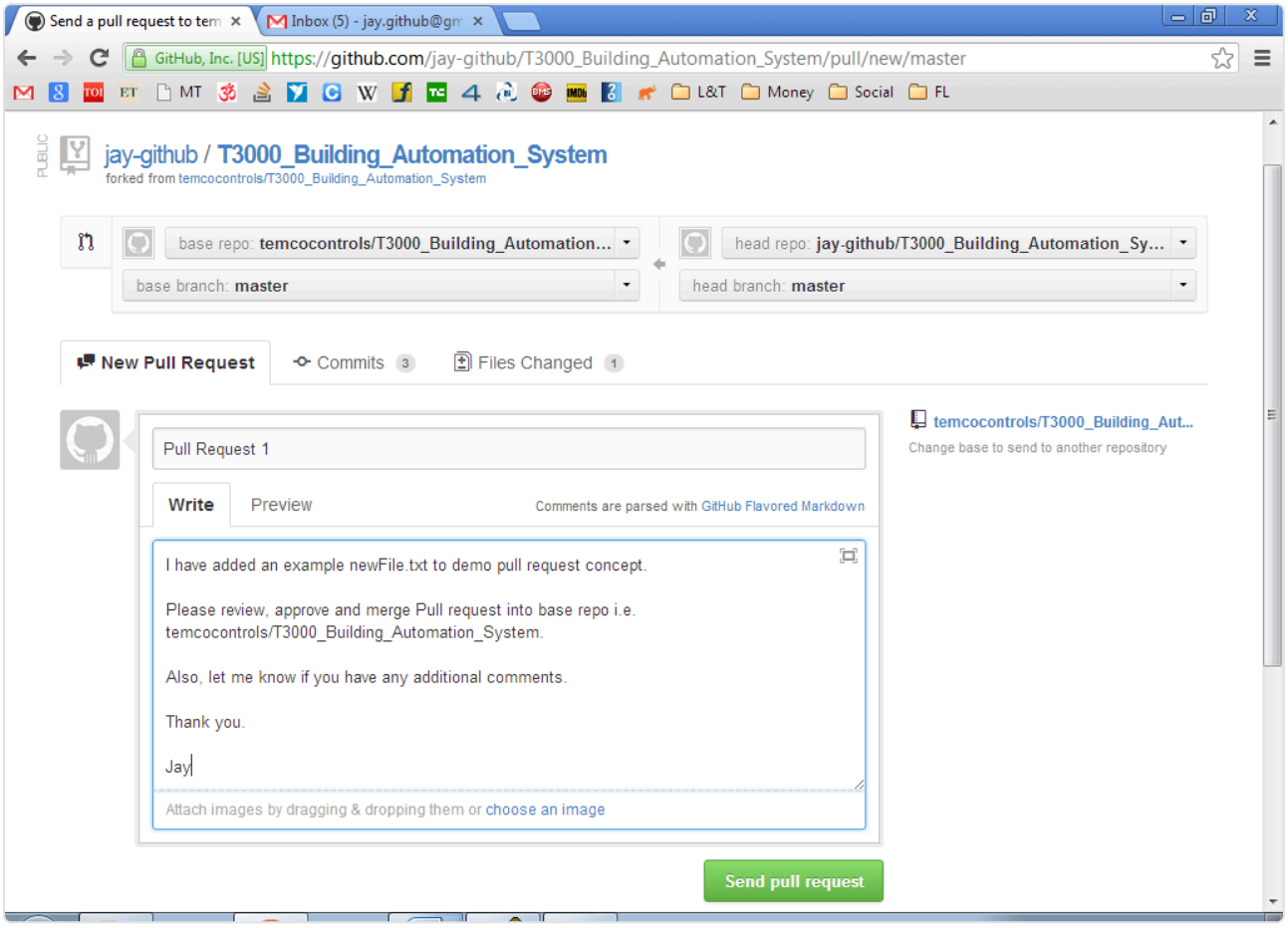
Now notify temcontrols of the changes by clicking Pull Request on github website.



Enter a title and a message.

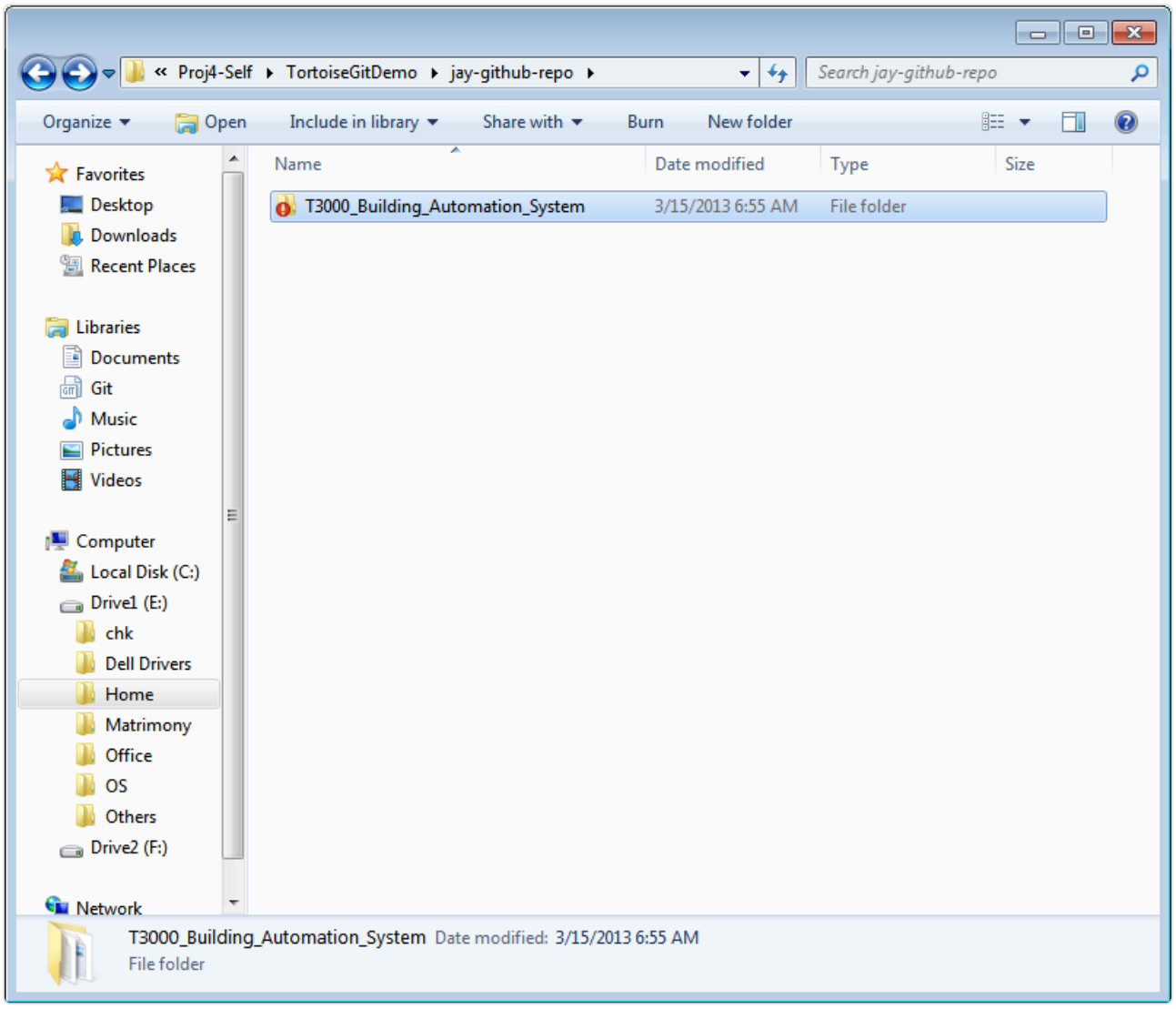


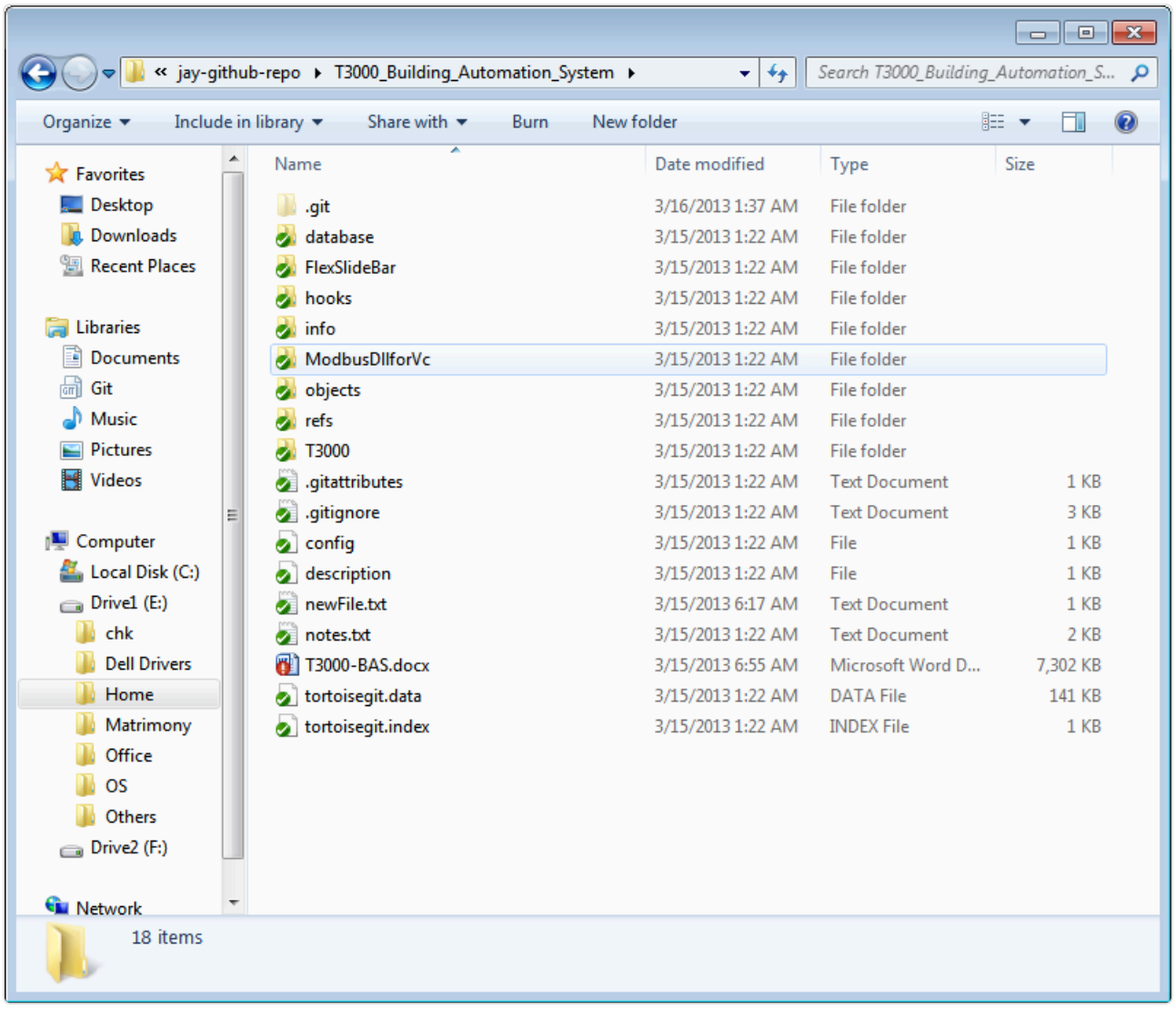
Click on Send Pull Request.



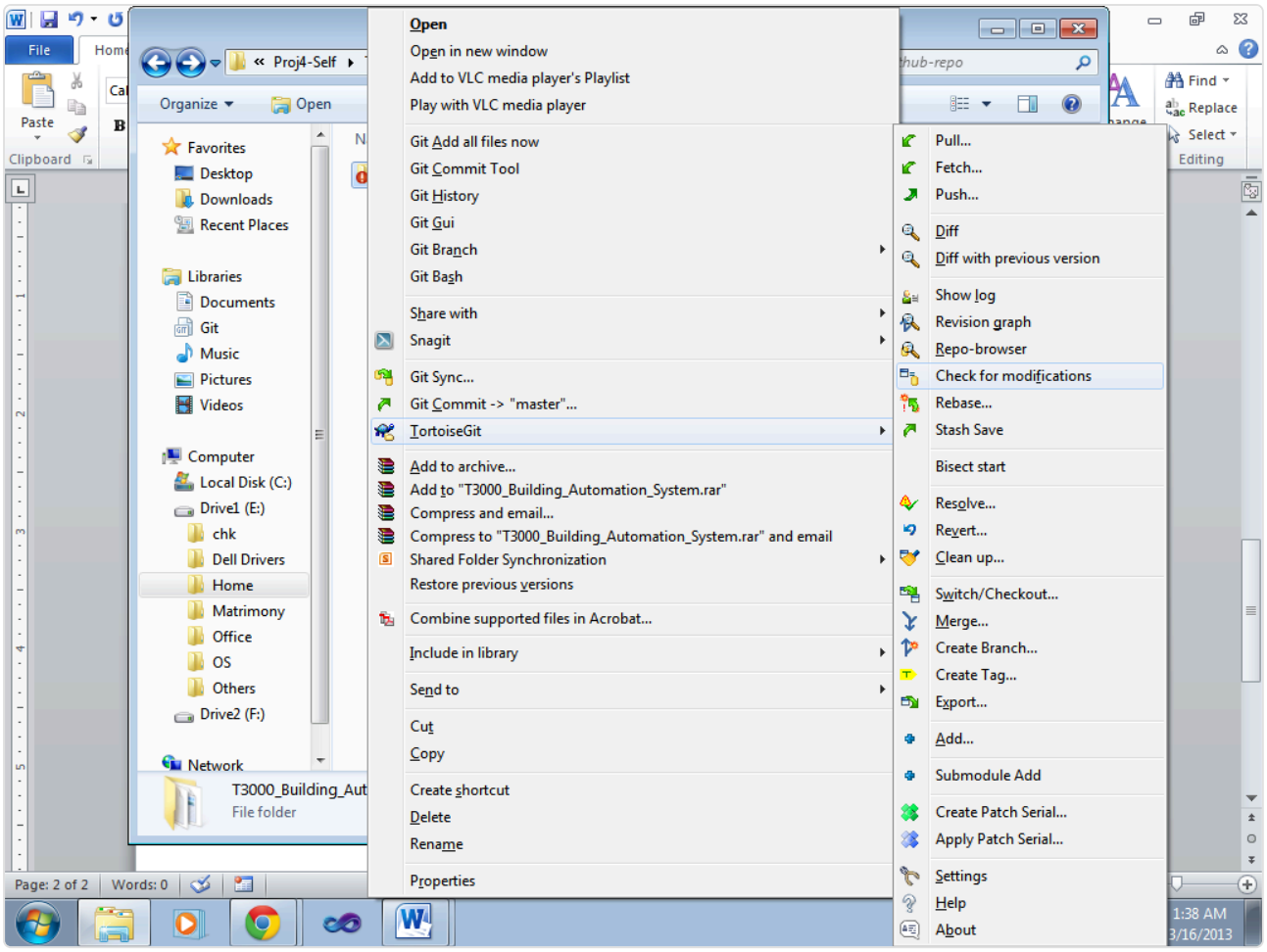
Now, single person controlling main repo shall be able to merge the pull request as described in last section of chapter 6.

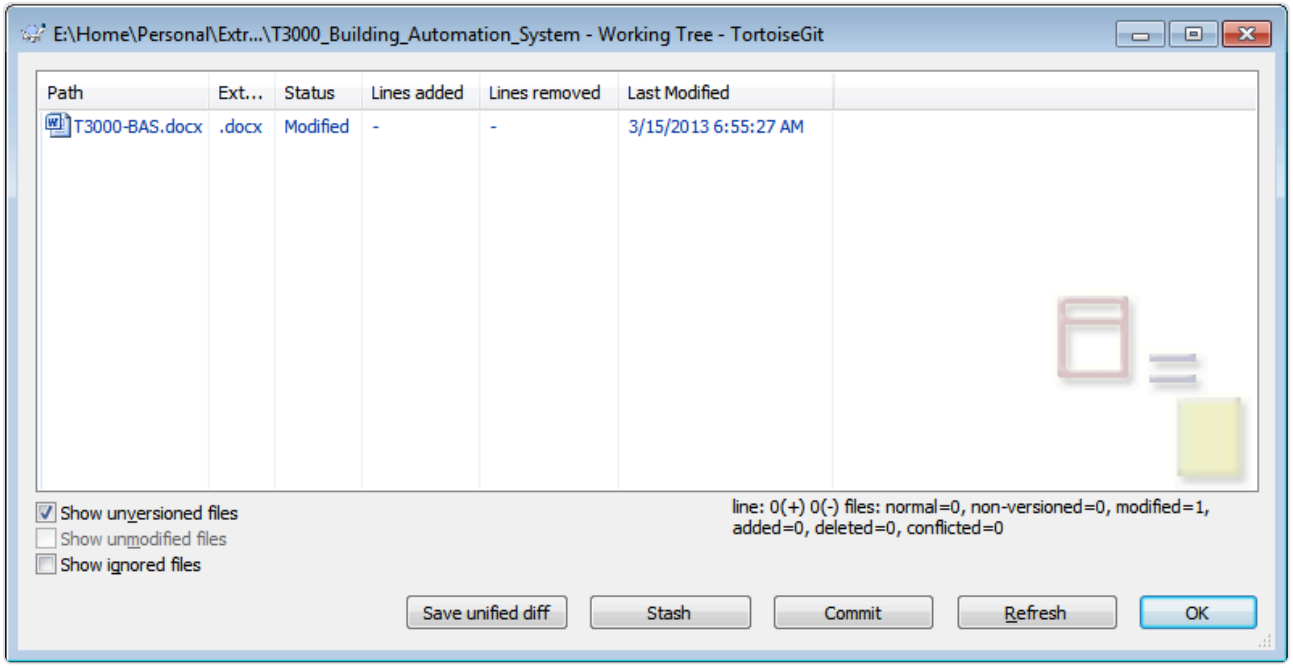
Following are screen shots of adding modifications. (In case Add... does not work)



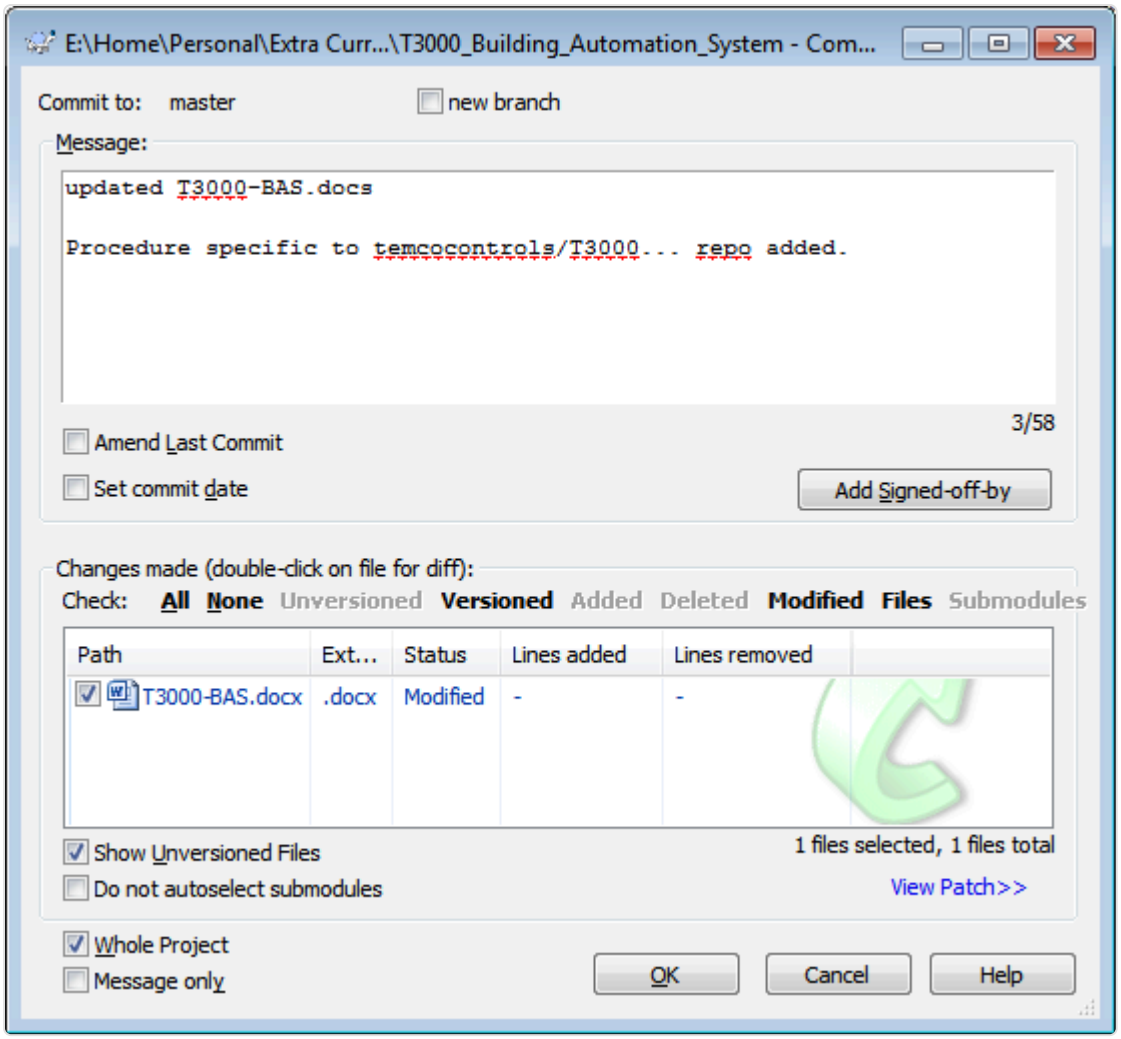


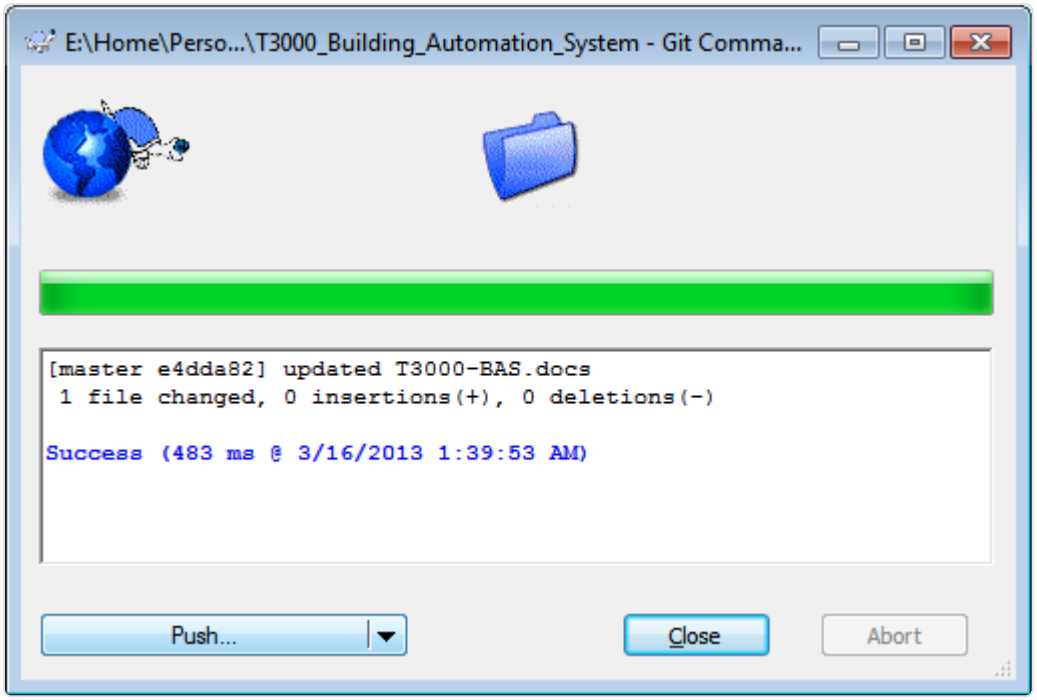
Click on TortoiseGit -> Check for modifications

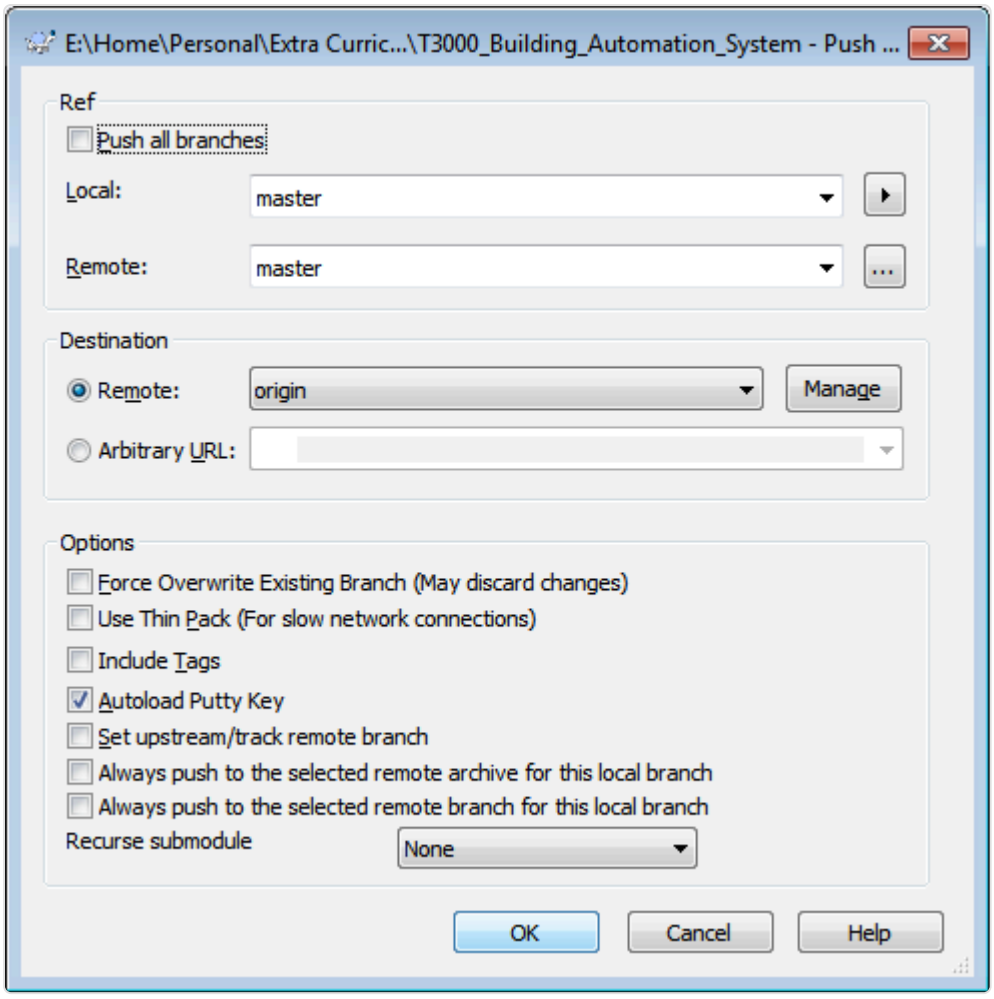


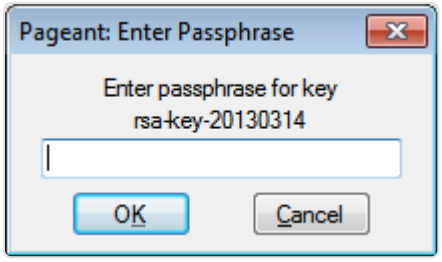


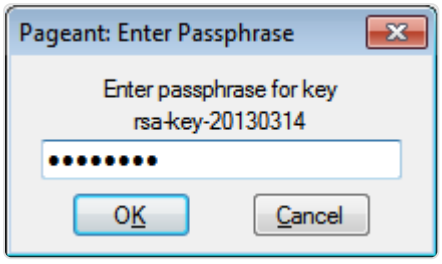
Click on Commit.

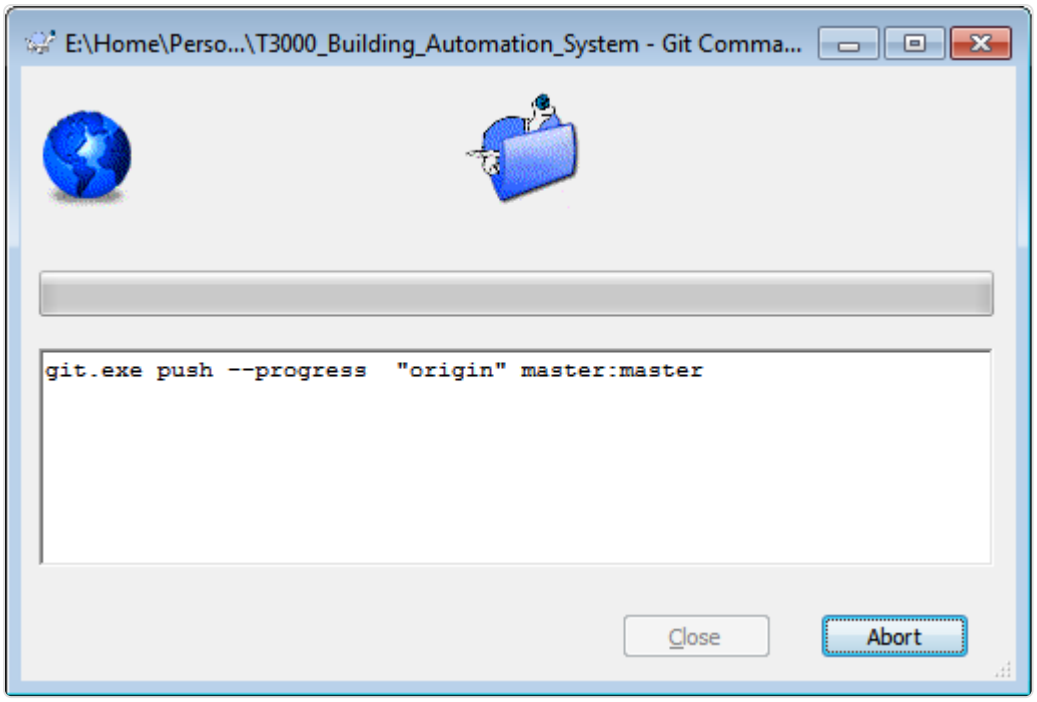

E:\Home\Personal\Extra Curr...\T3000_Building_Automation_System - Com...
Commit to: master
new branch
Message:
updated T3000-BAS.docs
Procedure specific to temcocontrols/T3000... repo added.
3/58
Amend Last Commit
Set commit date
Add Signed-off-by
Changes made (double-click on file for diff):
Check: All None Unversioned Versioned Added Deleted Modified Files Submodules
Path
Ext...
Status
Lines added
Lines removed
T3000-BAS.docx
.docx
Modified
-
-
Show Unversioned Files
Do not autoselect submodules
1 files selected, 1 files total
View Patch>>
Whole Project
Message only
OK
Cancel
Help

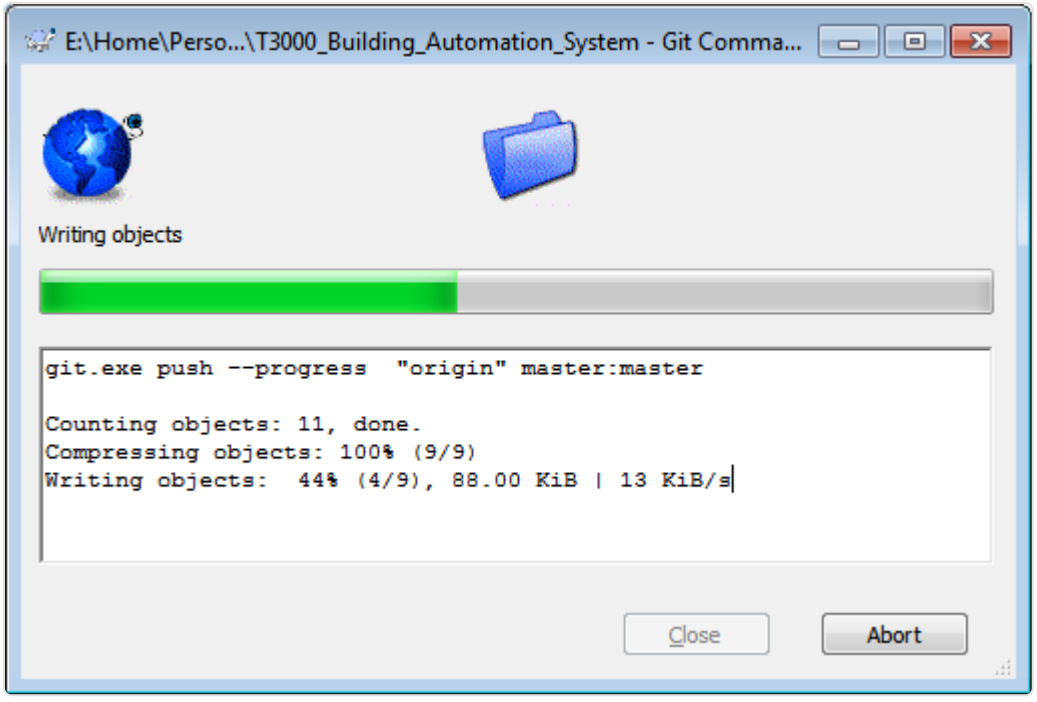

E:\Home\Perso...\T3000_Building_Automation_System - Git Comma...
[master e4dda82] updated T3000-BAS.docs
1 file changed, 0 insertions(+), 0 deletions(-)
Success (483 ms @ 3/16/2013 1:39:53 AM)
Push...
Close
Abort

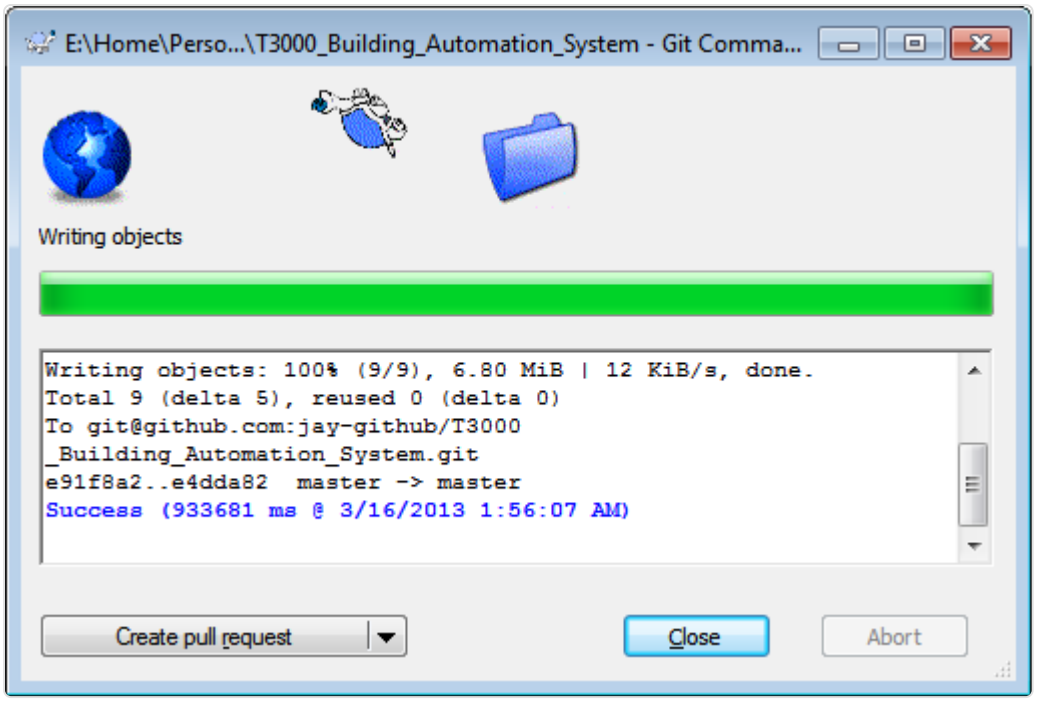

E:\Home\Personal\Extra Curric...\T3000_Building_Automation_System - Push ...
Ref
Push all branches
Local:
master
Remote:
master
Destination
Remote:
origin
Manage
Arbitrary URL:
Options
Force Overwrite Existing Branch (May discard changes)
Use Thin Pack (For slow network connections)
Include Tags
Autoload Putty Key
Set upstream/track remote branch
Always push to the selected remote archive for this local branch
Always push to the selected remote branch for this local branch
Recurse submodule
None
OK
Cancel
Help


Pageant: Enter Passphrase
Enter passphrase for key
rsa-key-20130314
OK
Cancel

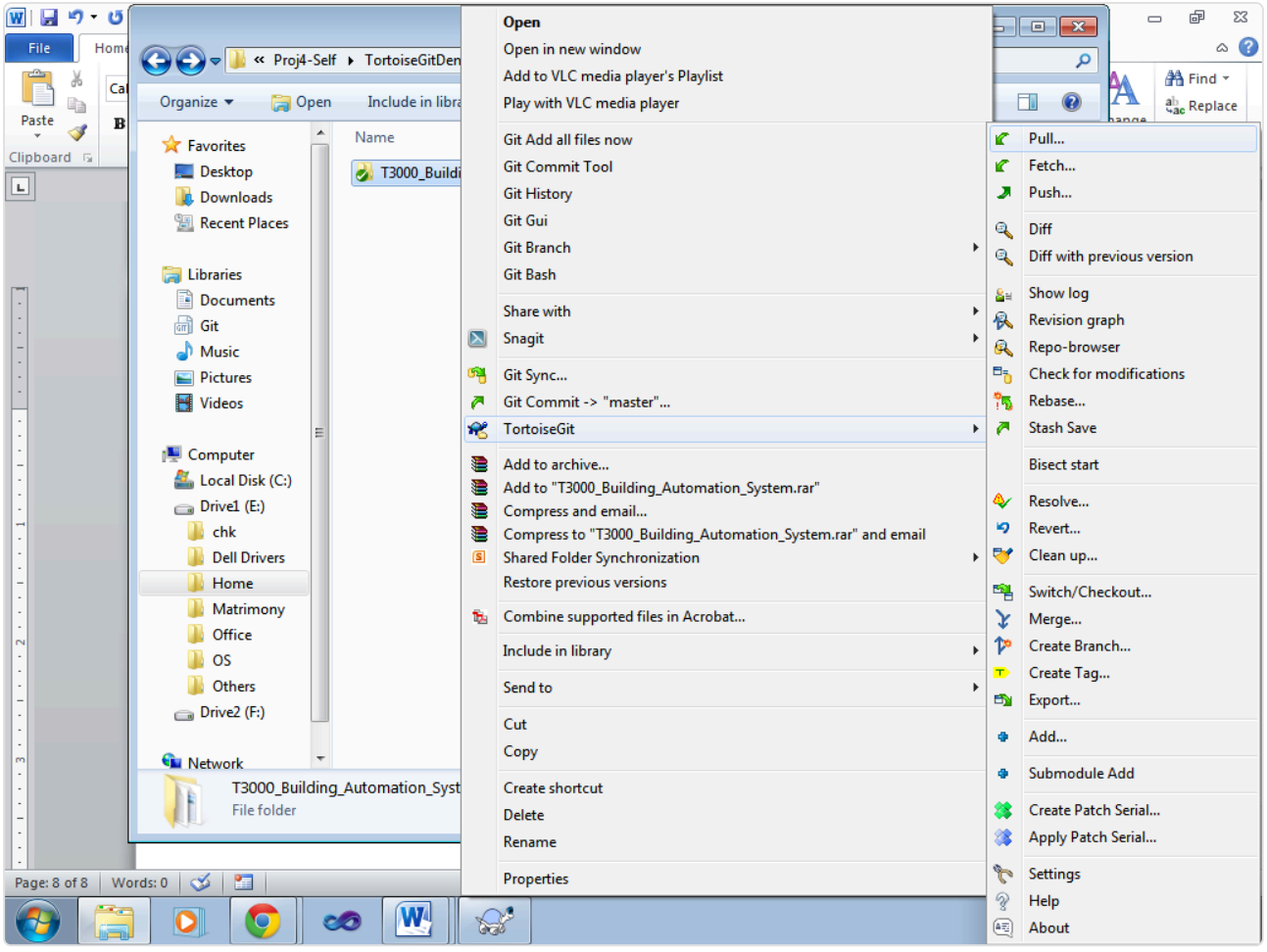

Pageant: Enter Passphrase
Enter passphrase for key
rsa-key-20130314
OK
Cancel

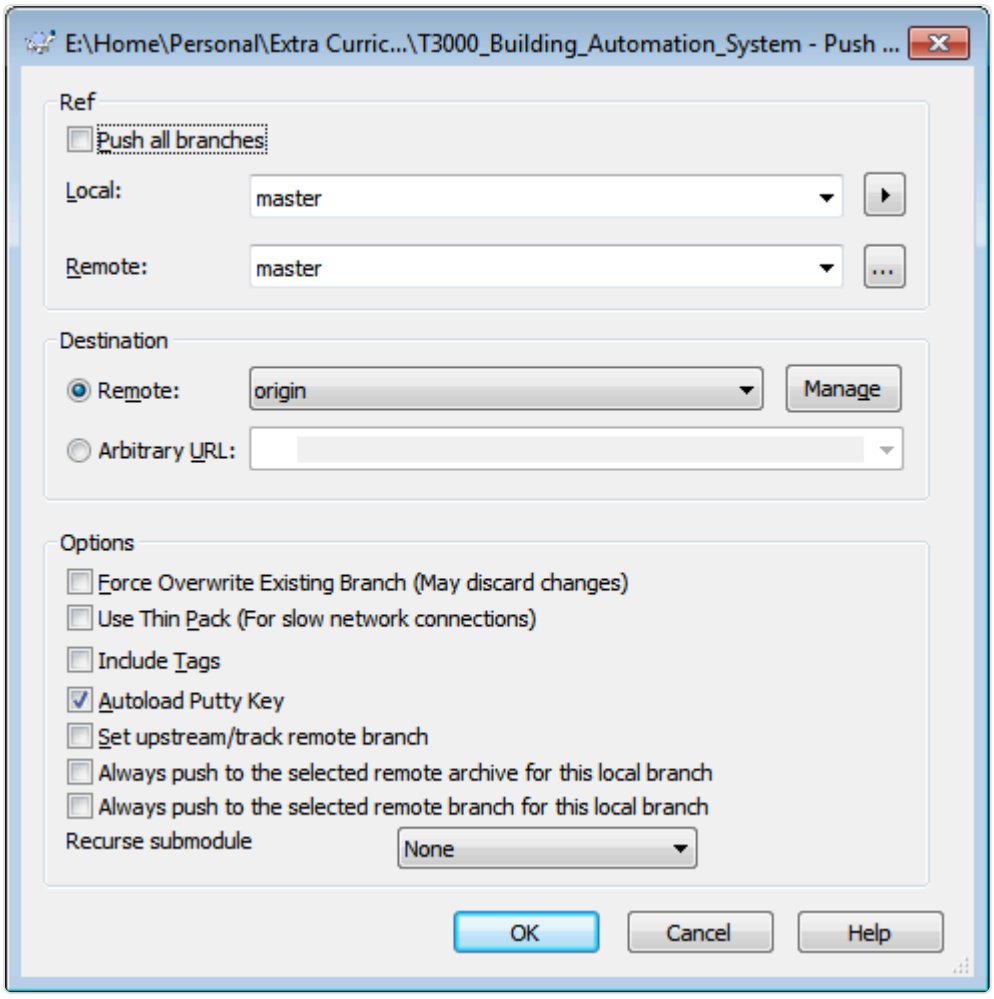

E:\Home\Perso...\T3000_Building_Automation_System - Git Comma...
git.exe push --progress "origin" master:master
Close
Abort

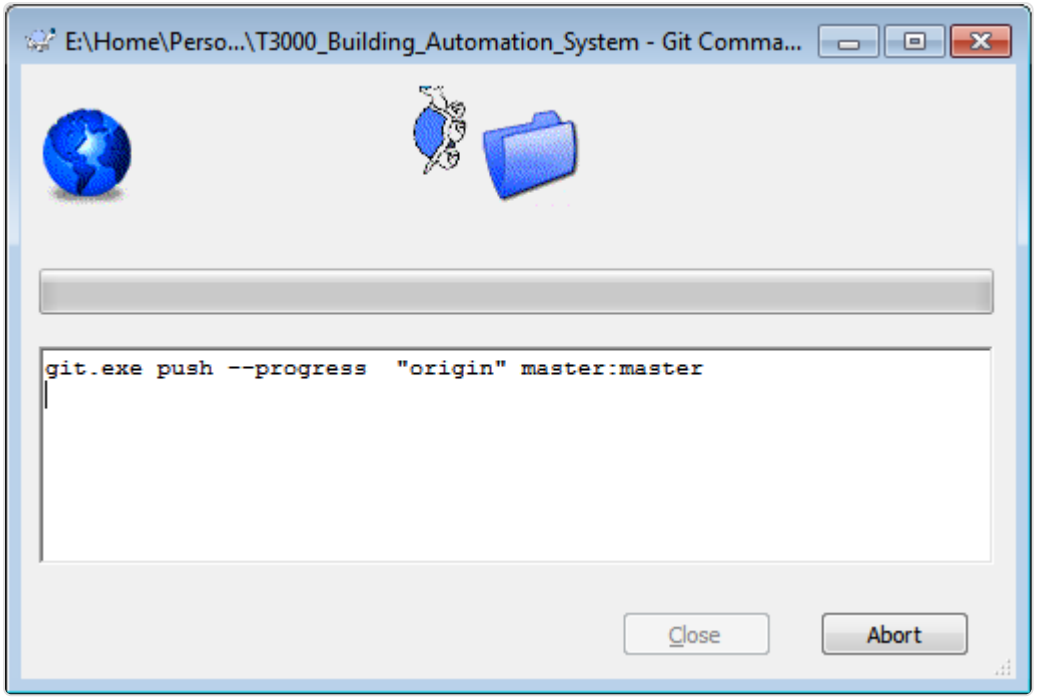

E:\Home\Perso...\T3000_Building_Automation_System - Git Comma...
Writing objects
git.exe push --progress "origin" master:master
Counting objects: 11, done.
Compressing objects: 100% (9/9)
Writing objects: 44% (4/9), 88.00 KiB | 13 KiB/s
Close
Abort

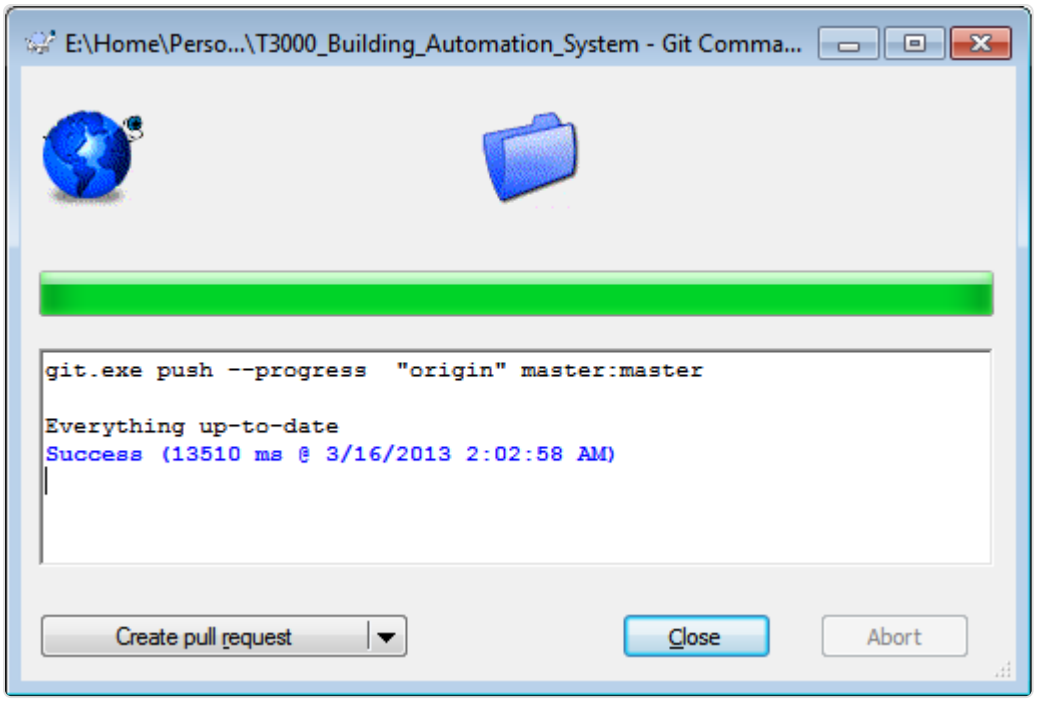


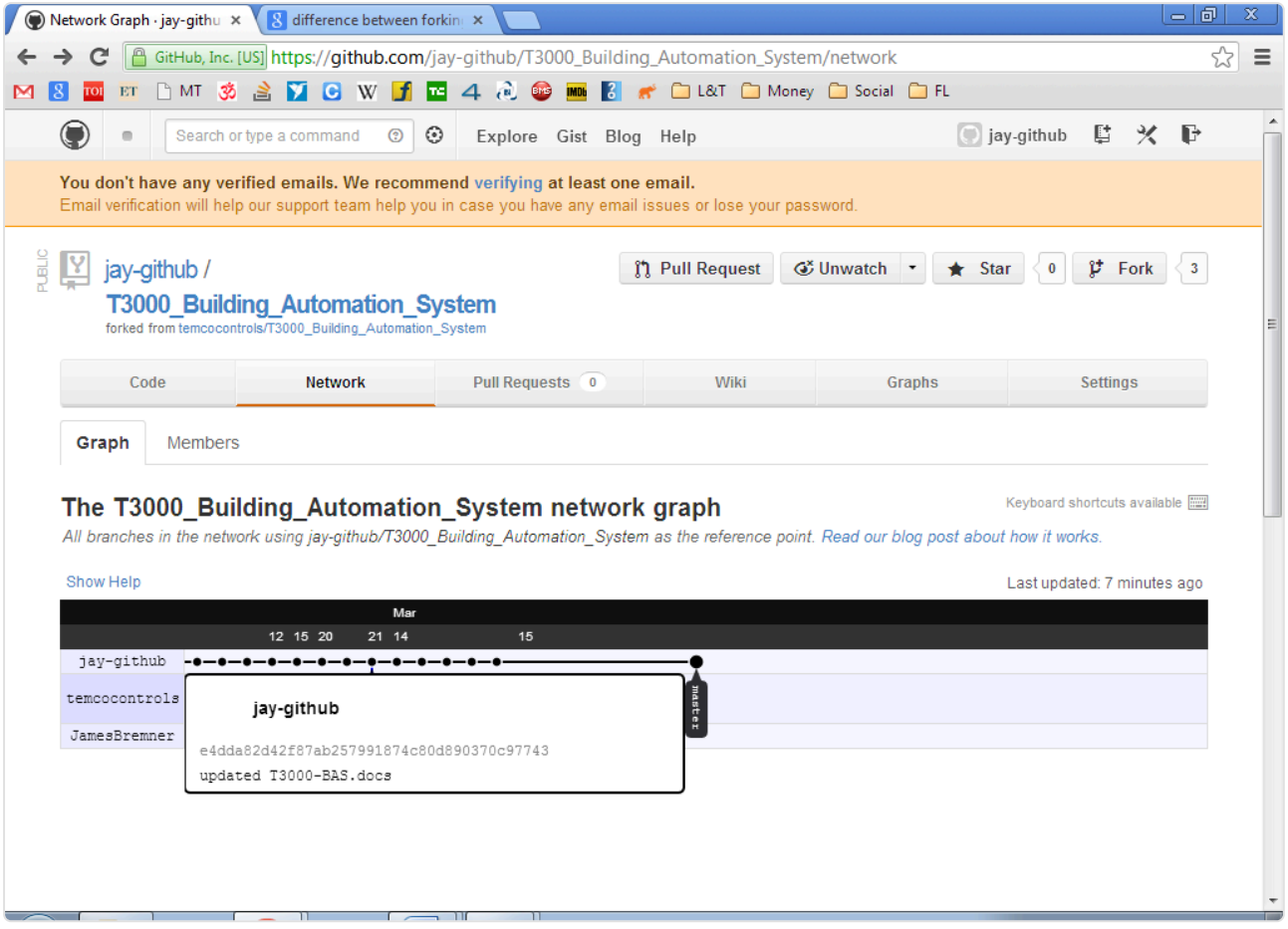
Push to github. TortoiseGit -> Push...

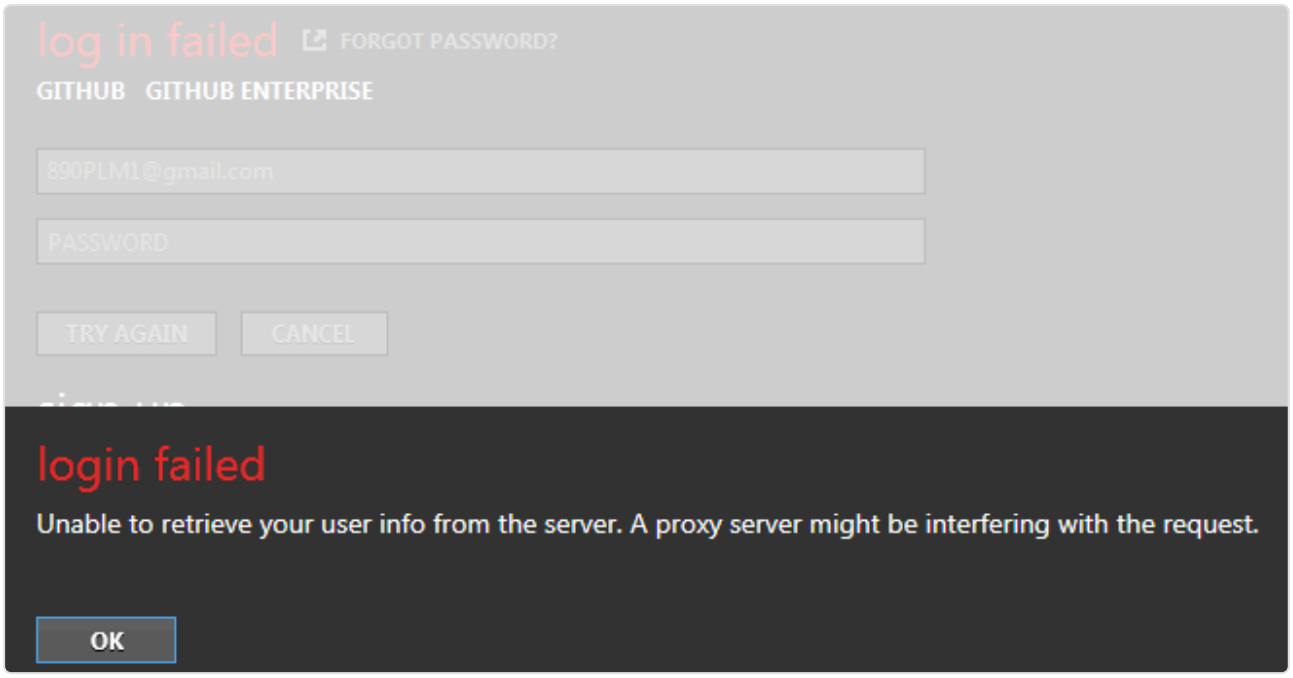



E:\Home\Personal\Extra Curric...\T3000_Building_Automation_System - Push ...
Ref
Push all branches
Local:
master
Remote:
master
Destination
Remote:
origin
Manage
Arbitrary URL:
Options
Force Overwrite Existing Branch (May discard changes)
Use Thin Pack (For slow network connections)
Include Tags
Autoload Putty Key
Set upstream/track remote branch
Always push to the selected remote archive for this local branch
Always push to the selected remote branch for this local branch
Recurse submodule
None
OK
Cancel
Help


E:\Home\Perso...\T3000_Building_Automation_System - Git Comma...
git.exe push --progress "origin" master:master
Close
Abort


E:\Home\Perso...\T3000_Building_Automation_System - Git Comma...
git.exe push --progress "origin" master:master
Everything up-to-date
Success (13510 ms @ 3/16/2013 2:02:58 AM)
Create pull request
Close
Abort


Network Graph · jay-githu
difference between forkin
GitHub, Inc. [US] https://github.com/jay-github/T3000_Building_Automation_System/network
Search or type a command
Explore Gist Blog Help
jay-github
You don't have any verified emails. We recommend verifying at least one email.
Email verification will help our support team help you in case you have any email issues or lose your password.
jay-github / T3000_Building_Automation_System
forked from temcocontrols/T3000_Building_Automation_System
Pull Request Unwatch Star 0 Fork 3
Code Network Pull Requests 0 Wiki Graphs Settings
Graph Members
The T3000_Building_Automation_System network graph
Keyboard shortcuts available
All branches in the network using jay-github/T3000_Building_Automation_System as the reference point. Read our blog post about how it works.
Show Help
Last updated: 7 minutes ago
Mar
12 15 20 21 14 15
jay-github
temcocontrols
JamesBremner
jay-github
e4dda82d42f87ab257991874c80d890370c97743
updated T3000-BAS.docs
master



https://github.com/gitextensions/gitextensions/issues/2044

download and install

http://download.microsoft.com/download/2/B/F/2BF4D7D1-E781-4EE0-9E4F-FDD44A2F8934/NDP40-KB2468871-v2-x86.exe